

Министерство образования Республики Беларусь

Учреждение образования «БЕЛОРУССКИЙ
ГОСУДАРСТВЕННЫЙ ТЕХНОЛОГИЧЕСКИЙ УНИВЕРСИТЕТ»

Факультет Информационных технологий

Кафедра Информационных систем и технологий

Специальность 1-40 05 01 Информационные системы и технологии

Специализация 1-40 05 01-03 Информационные системы и технологии (изда-
тельско-полиграфический комплекс)

**ПОЯСНИТЕЛЬНАЯ ЗАПИСКА
ДИПЛОМНОГО ПРОЕКТА НА ТЕМУ:**

Веб-приложение «Маркетплейс фриланс-заказов для студентов»

Обучающийся 4 курс, 2 группа Д.В. Глухова
курс, группа дата подпись И. О. Ф.

Руководитель
дипломного проекта асс. Н.И. Уласевич
должность, ученая степень, ученое звание дата подпись И. О. Ф.

Заведующий
кафедрой К.Т.Н. Е.А. Блинова
должность, ученая степень, ученое звание дата подпись И. О. Ф.

Консультант ст. преп. А.С. Соболевский
должность, ученая степень, ученое звание дата подпись И. О. Ф.

Нормоконтролер асс. А.Н. Николайчук
должность, ученая степень, ученое звание дата подпись И. О. Ф.

Дипломный проект защищен с оценкой _____

Председатель ГЭК к.т.н., доцент В.К. Дюбков
должность, ученая степень, ученое звание дата подпись И. О. Ф.

Минск 2026

Лист задания

Оглавление

Реферат	5
Abstract	6
Введение	7
1 Постановка задачи и обзор аналогичных решений	8
1.1 Постановка задачи	8
1.2 Обзор аналогичных решений	8
1.3 Выводы по разделу	13
2 Проектирование веб-приложения	14
2.1 Выбор средств реализации	14
2.2 Разработка функциональных требований	15
2.3 Разработка базы данных	17
2.4 Архитектура веб-приложения	24
2.5 Проектирование алгоритмов	25
2.6 Выводы по разделу	28
3 Разработка веб-приложения	29
3.1 Разработка серверной части	29
3.1.1 Аутентификация и авторизация пользователей	31
3.1.2 Реализация отправки сообщений	32
3.1.3 Реализация создания заказа	33
3.1.4 Реализация создания отклика на заказ или услугу	35
3.1.5 Реализация функций администратора	35
3.2 Разработка клиентской части	37
3.2.1 Система маршрутизации клиентской части	39
3.2.2 Сервисы для взаимодействия с API	39
3.2.4 WebSocket-коммуникация и управление состоянием	39
3.2.6 Стилизация интерфейса	40
3.2.7 Обработка блокировки пользователей	40
3.3 Сборка проекта в Docker-контейнер	41
3.4 Выводы по разделу	41
4 Тестирование веб-приложения	43
4.1 Тестирование модуля регистрации и аутентификации	43
4.2 Тестирование создания заказов и услуг	46
4.3 Тестирование системы рейтингов	48
4.4 Тестирование системы откликов	49
4.5 Тестирование системы избранных	50
4.6 Тестирование каталога	51

				ДП 00.00.ПЗ			
	ФИО	Подпись	Дата	Оглавление			
Разраб.	Глухова Д.В.						
Пров.	Уласевич Н.И.						
Н. контр.	Николайчук А.Н.						
Утв.	Блинова Е.А.			БГТУ 1-40 05 01, 2026			
					Лит.	Лист	Листов
					У	1	2

4.7 Тестирование функционала администратора	53
4.8 Вывод по разделу.....	56
5 Руководство пользователя.....	57
5.1 Руководство пользователя для роли Исполнитель	57
5.2 Руководство пользователя для роли Заказчик.....	65
5.3 Руководство пользователя для роли Администратор	69
5.4 Руководство пользователя для роли Менеджер	73
5.5 Выводы по разделу	75
6 Техничко-экономическое обоснование проекта	76
6.1 Общая характеристика программного средства.....	76
6.2 Исходные данные для проведения расчетов	77
6.3 Методика обоснования цены.....	78
6.4 Расчет затрат времени на разработку программного средства.....	78
6.5 Расчет основной заработной платы	79
6.6 Расчет дополнительной заработной платы	80
6.7 Расчет отчислений в Фонд социальной защиты населения и по обязательному страхованию.....	80
6.8 Расчет расходов на материалы.....	81
6.9 Расходы на специальное оборудование и платные услуги	82
6.10 Расчет расходов на оплату машинного времени.....	82
6.11 Расчет прочих прямых затрат.....	83
6.12 Расчет накладных расходов.....	83
6.13 Сумма расходов на разработку программного средства.....	83
6.14 Расходы на сопровождение и адаптацию	83
6.15 Полная себестоимость.....	84
6.16 Определение цены и оценка эффективности.....	84
6.17 Конкурентные преимущества программного средства	86
6.18 Выводы по разделу	87
Заключение	88
Список использованных источников	89
ПРИЛОЖЕНИЕ А Схема архитектуры приложения	90
ПРИЛОЖЕНИЕ Б Диаграмма вариантов использования	91
ПРИЛОЖЕНИЕ В Логическая схема базы данных.....	92
ПРИЛОЖЕНИЕ Г Блок-схема обработки и маршрутизации сообщения	93
ПРИЛОЖЕНИЕ Д Блок-схема процесса создания заказа	94
ПРИЛОЖЕНИЕ Е Модели серверной части	95
ПРИЛОЖЕНИЕ Ж Реализация создания отклика на заказ или услугу	96
ПРИЛОЖЕНИЕ И Система маршрутизации клиентской части	97
ПРИЛОЖЕНИЕ К Сервис администратора	98
ПРИЛОЖЕНИЕ Л WebSocket-коммуникация и управление состоянием	99
ПРИЛОЖЕНИЕ М Docker-файлы для сборки проекта.....	100
ПРИЛОЖЕНИЕ Н Экранная форма главной страницы.....	101
ПРИЛОЖЕНИЕ П Таблица технико-экономических показателей.....	102

Реферат

Пояснительная записка дипломного проекта содержит 89 страниц, 20 таблиц, 84 иллюстрации, 12 использованных источников, 13 приложений.

КРОССПЛАТФОРМЕННОЕ ВЕБ-ПРИЛОЖЕНИЕ, NODE.JS, MYSQL, REACT, VISUAL STUDIO CODE 1.118.1

Целью дипломного проекта является разработка системы для интеграции студентов в профессиональную IT-среду. Программный комплекс обеспечивает условия для выполнения коммерческих задач обучающимися. Работа направлена на формирование функциональной экосистемы взаимодействия между учащимися и представителями малого бизнеса.

Пояснительная записка включает введение, шесть полноценных разделов, отражающих все этапы разработки, и заключение.

В первом разделе содержится аналитический обзор аналогичных решений по теме дипломного проекта и постановка задачи.

Второй раздел посвящен процессу проектирования веб-приложения.

В третьем разделе описывается процесс программной реализации проекта и приведены листинги написанного кода.

В четвертом разделе рассматриваются методы тестирования и приведены результаты проверки работоспособности системы.

В пятом разделе описано руководство пользователя, направленное на объяснение принципов работы с приложением.

В шестом разделе представлен расчет экономических показателей, связанных с разработкой программного продукта.

В заключении приведены результаты проделанной работы.

Объем графического материала в пересчете на формат A1 – 1,5 листа.

				ДП 00.00.ПЗ			
	ФИО	Подпись	Дата	Реферат			
Разраб.	Глухова Д.В.						
Пров.	Уласевич Н.И.						
Н. контр.	Николайчук А.Н.						
Утв.	Блинова Е.А.						
				Лит.			
				Лист			
				Листов			
				У		1	1
				БГТУ 1-40 05 01, 2026			

Abstract

The abstract of the diploma project consists of 89 pages, 20 tables, 84 illustrations, 21 literature sources and 13 appendices.

CROSS-PLATFORM WEB APPLICATION, NODE.JS, MYSQL, REACT, VISUAL STUDIO CODE 1.118.1

The purpose of the diploma project is to develop a system for integrating students into a professional IT environment. The software package provides conditions for students to perform commercial tasks. The work is aimed at creating a functional ecosystem of interaction between students and representatives of small businesses.

The explanatory note consists of an introduction, six chapters covering all stages of development, and a conclusion.

Chapter One contains an analytical review of the literature on the topic of the diploma project and the formulation of the task for the development of the web application.

Chapter Two is dedicated to the process of designing the web application.

Chapter Three describes the software implementation process of the project and includes listings of the written code.

Chapter Four thoroughly discusses testing methods and presents the results of checking the system's performance.

Chapter Five provides a user manual aimed at explaining the principles of working with the application.

Chapter Six presents the calculation of economic indicators related to the development of the software product.

The conclusion presents the results of the work done.

The volume of graphic material in terms of A1 format is 1,5 sheets.

				ДП 00.00.ПЗ						
	ФИО	Подпись	Дата							
Разраб.	Глухова Д.В.			Abstract	Лит.		Лист		Листов	
Пров.	Уласевич Н.И.				У		1	1		
					БГТУ 1-40 05 01, 2026					
Н. контр.	Николайчук А.Н.									
Утв.	Блинова Е.А.									

Введение

Целью дипломного проекта является формирование инфраструктуры для профессиональной интеграции студентов. Веб-приложение предусматривает создание цифровой среды для накопления практического опыта в период обучения, обеспечивает условия для перехода от теоретической подготовки к выполнению коммерческих задач, выступает инструментом связи между вузом и рынком труда. Веб-приложения на базе платформы Node.js [1] и языка программирования JavaScript [2].

Целевая аудитория включает студентов высших учебных заведений очной и заочной формы обучения. Основной сегмент составляют учащиеся технических специальностей со второго по четвертый курсы. Данные пользователи имеют базовые знания в области программирования, дизайна и системного администрирования. Они испытывают потребность в получении первого практического опыта для резюме. Веб-приложение предоставляет им инструменты для монетизации учебных навыков и создания портфолио.

Вторую группу пользователей составляют представители малого и среднего бизнеса. Заказчики нуждаются в реализации разовых задач. Бюджет подобных проектов часто ограничен. Предприниматели ищут исполнителей для выполнения работы по доступной цене. Веб-приложение упрощает поиск кадров для поручений без необходимости найма сотрудников в штат компании.

Для достижения цели необходимо решить следующие задачи:

- провести анализ существующих фриланс-платформ и выявить их достоинства и недостатки применительно к целевой аудитории;
- спроектировать архитектуру веб-приложения, включая разработку UML-диаграммы вариантов использования и логической схемы базы данных;
- разработать программное обеспечение серверной и клиентской частей веб-приложения в соответствии с функциональными требованиями;
- реализовать взаимодействие с базой данных и обеспечить разделение ролей (гость, исполнитель, заказчик, менеджер, администратор).
- провести тестирование разработанного веб-приложения для проверки его функциональности и корректности работы;
- разработать руководство пользователя в соответствии с требованиями.

				ДП 00.00.ПЗ						
	ФИО	Подпись	Дата							
Разраб.	Глухова Д.В.			Введение	Лит.		Лист		Листов	
Пров.	Уласевич Н.И.				У		1	1		
					БГТУ 1-40 05 01, 2026					
Н. контр.	Николайчук А.Н.									
Утв.	Блинова Е.А.									

1 Постановка задачи и обзор аналогичных решений

1.1 Постановка задачи

Рынок IT-труда предъявляет высокие требования к молодым специалистам, ожидая от них не только теоретических знаний, но и практического опыта. Студенты профильных специальностей часто сталкиваются с проблемой «холодного старта»: отсутствием коммерческого опыта и портфолио, что затрудняет поиск первой работы или стажировки.

Проблема заключается в отсутствии специализированной нишевой платформы, которая бы эффективно связывала заказчиков, ищущих исполнителей для различных задач, и студентов, ищущих практический опыт и возможность формирования портфолио с информацией о себе.

1.2 Обзор аналогичных решений

Для определения оптимальной модели функционирования проектируемого веб-приложения был проведен анализ платформ Fiverr [3], Freelancer [4] и Upwork [5] – мировых лидеров в сфере фриланс-услуг.

В отличие от традиционных фриланс-бирж, где основой является заказ, Fiverr оперирует концепцией «Гига» – стандартизированной, заранее упакованной услуги с фиксированной ценой и четким набором результатов. Исполнитель выступает в роли магазина, а заказчик – в роли потребителя, выбирающего готовый товар. На рисунке 1.1 представлен обзор главной страницы.

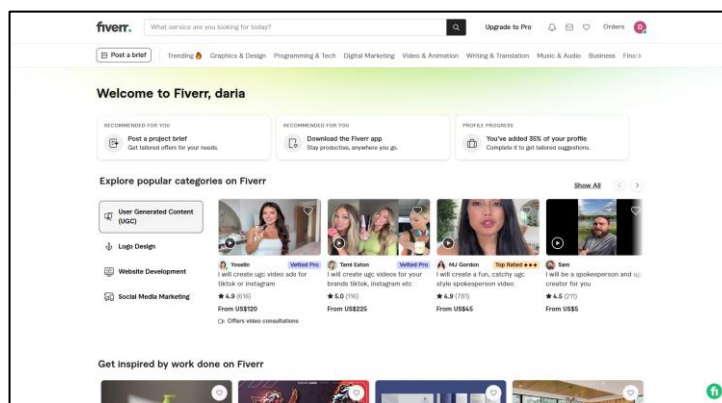


Рисунок 1.1 – Обзор главной страницы «Fiverr»

Система репутации на Fiverr является мощным инструментом мотивации и построения доверия, что крайне важно для маркетплейса, где работают начинающие исполнители и заказчики.

				ДП 01.00.ПЗ						
	ФИО	Подпись	Дата							
Разраб.	Глухова Д.В.			1 Постановка задачи и обзор аналогичных решений			Лит.	Лист	Листов	
Пров.	Уласевич Н.И.						У	1	6	
							БГТУ 1-40 05 01, 2026			
Н. контр.	Николайчук А.Н.									
Утв.	Блинова Е.А.									

Fiverr использует механизм геймификации, где исполнитель продвигается от «New Seller» до «Top Rated Seller» на основе таких метрик, как своевременность сдачи, качество работы и объем заработка. Каждый новый уровень исполнителя открывает дополнительные привилегии.

Далее рассмотрим профиль пользователя, представленный на платформе Fiverr, вид профиля продемонстрирован на рисунке 1.2.

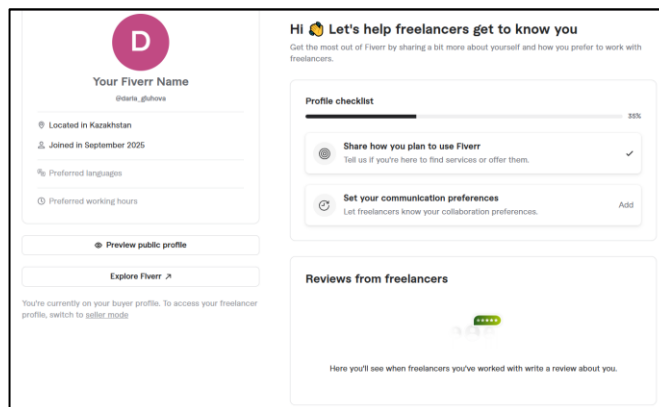


Рисунок 1.2 – Обзор профиля пользователя

Функционал и визуальная составляющая профиля исполнителя созданы для быстрого формирования доверия и демонстрации компетенций. Сделан упор на репутацию пользователя, который предоставляет социальное доказательство качества работы. Fiverr использует пятизвездочную систему оценки, отображение среднего бала, количество отзывов четко соответствует числу выполненных работ, по которым были оставлены отзывы. Исполнитель использует свой профиль как центр для управления всей деятельностью на платформе. Этот функционал обычно скрыт от публичного просмотра и доступен через личный кабинет. Поддерживаются настройки: адрес электронной почты, номер телефона, пароль, оповещения о заказах, сообщениях, отзывах.

Далее рассмотрим категории услуг, представлено на рисунке 1.3.

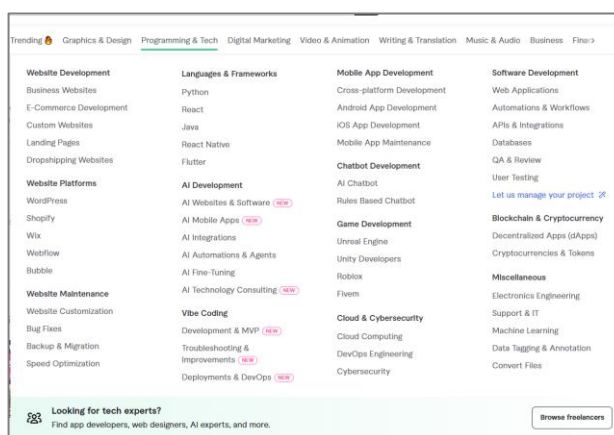


Рисунок 1.3 – Обзор категорий

Система категоризации на Fiverr является ключевым элементом, который трансформирует фриланс-биржу в эффективный интернет-магазин услуг.

Категории организованы по принципу глубокого иерархического дерева, что позволяет заказчику максимально быстро и точно найти узкоспециализированную услугу. Преимущество такой структуры в ее масштабируемости и интуитивности: она не только ускоряет поиск, но и позволяет платформе охватывать широчайший спектр цифровых и креативных услуг, включая нишевые и трендовые направления. Для проектируемого веб-приложения этот подход важен, так как он позволяет эффективно структурировать предложения студентов по управлению и добавлению новых категорий и подкатегорий.

Главная страница Freelancer, показанная на рисунке 1.4, представляет собой классический пример биржи заказов, ориентированной на исполнителя.

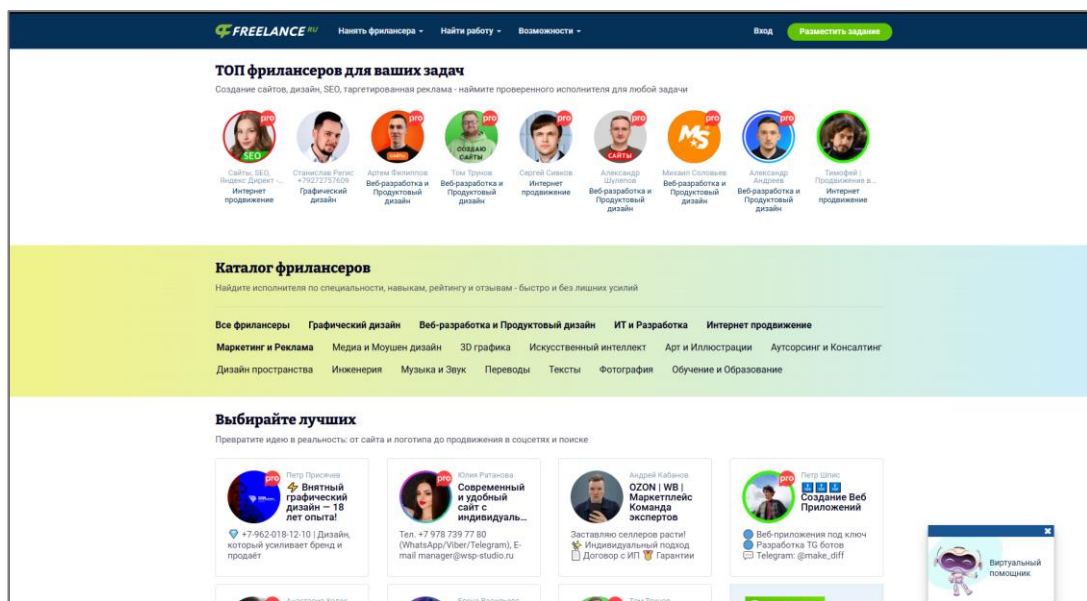


Рисунок 1.4 – Обзор главной страницы «Freelancer»

В отличие от витрины Fiverr, здесь ключевым элементом является динамическая лента проектов. Эта лента агрегирует и в реальном времени отображает все релевантные заказы, опубликованные заказчиками. Для проектируемого веб-приложения данная модель является основополагающей для реализации функционала со стороны исполнителя.

Функционал этой страницы значительно шире простого отображения данных; он неразрывно связан с мощными инструментами поиска и фильтрации. Хотя на рисунке 1.4 они могут быть не полностью видны, любая подобная лента обязательно подразумевает возможность сортировки по категориям, бюджету, дате публикации и требуемым навыкам.

Самое важное различие, которое необходимо учесть при адаптации – модель взаимодействия. Freelancer построен на механике аукциона, где исполнители соревнуются, предлагая минимальную цену.

Для заказчика зеркальный функционал этой страницы – панель управления, где он видит свои проекты и список откликнувшихся исполнителей.

Рисунок 1.5 демонстрирует один из важнейших функциональных блоков любой фриланс-платформы – интерфейс управления проектом и коммуникации. Как только заказчик одобрил отклик исполнителя, система создает для

них выделенную комнату проекта, которая выходит далеко за рамки обычного чата и представляет собой полноценную панель управления.

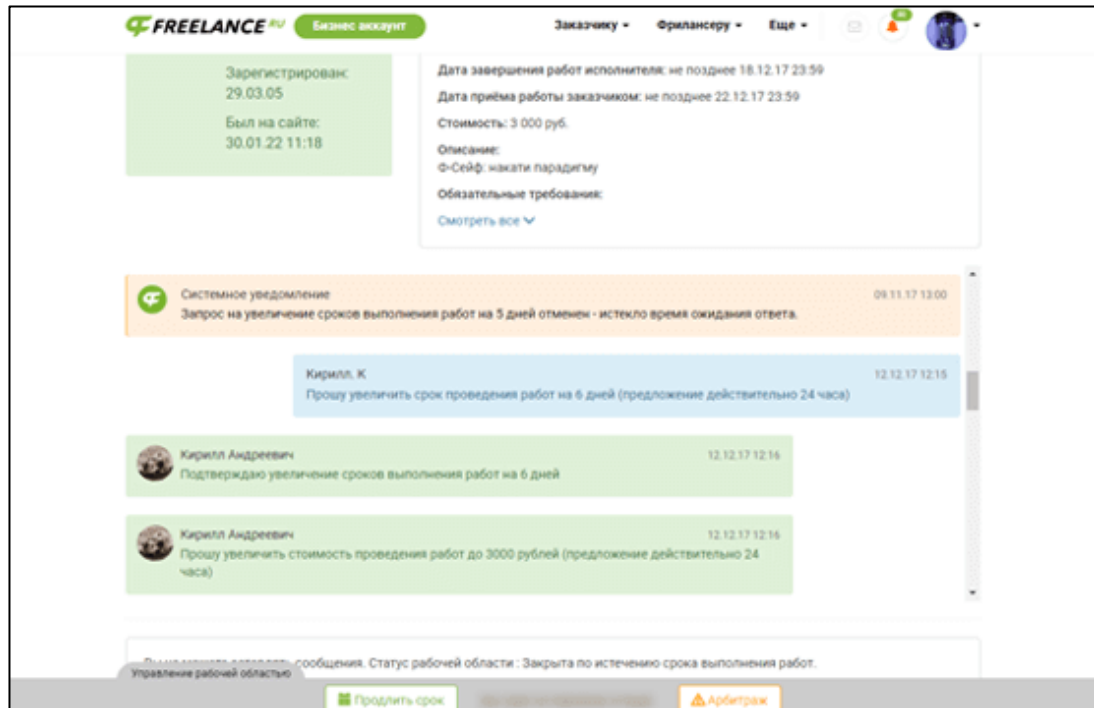


Рисунок 1.5 – Обзор коммуникации заказчика и исполнителя

Ключевой элемент – встроенный мессенджер, интегрированный с бизнес-логикой платформы. Мессенджер не просто чат для обсуждения деталей, а документированный канал связи. Все договоренности, переданные файлы и согласования фиксируются, что важно для разрешения ситуаций.

Далее рассмотрим следующий аналог – платформа Upwork, главная страница которой представлена на рисунке 1.6.

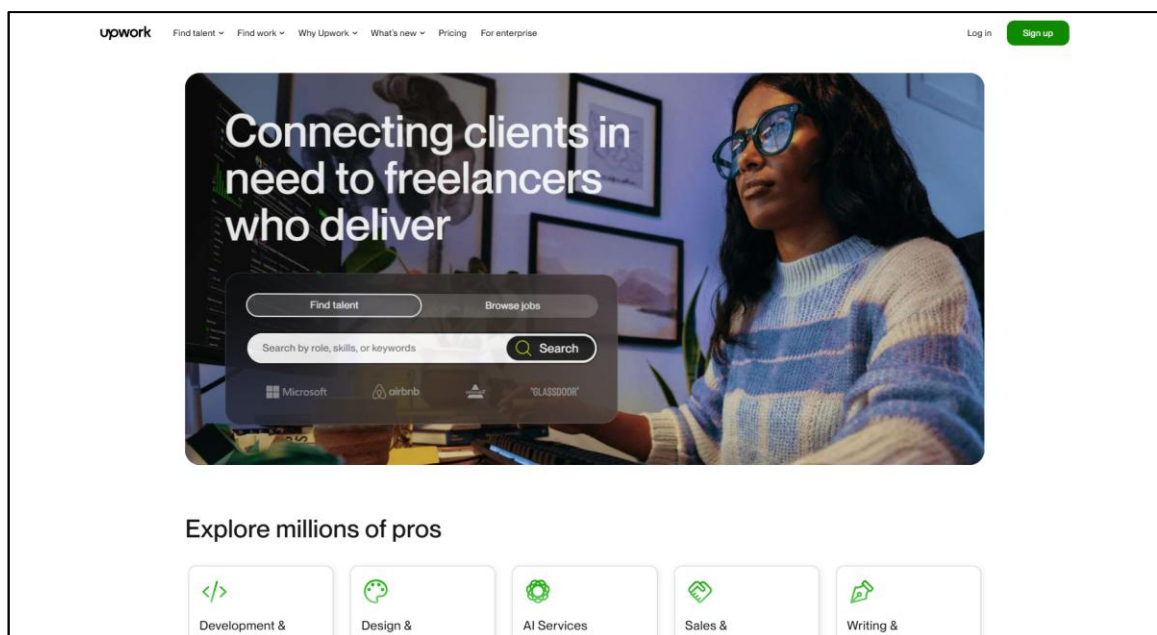


Рисунок 1.6 – Обзор главной страницы «Upwork»

Представляет собой гибридную модель, которая успешно объединяет в себе элементы биржи заказов и витрины услуг, позиционируя себя как решение для поиска высококвалифицированных специалистов и долгосрочного сотрудничества. Главная страница для исполнителя представляет собой ленту проектов, которая по функционалу схожа с Freelancer.

Важнейшим элементом для анализа является экономическая модель Upwork – система коннектов. Для того чтобы исполнитель мог откликнуться на заказ, он должен потратить внутреннюю платную валюту. Эта механика введена для борьбы со спамом и повышения качества откликов.

Процесс регистрации на платформах, подобных Upwork, является критически важным этапом, который определяет весь дальнейший пользовательский опыт. Как видно из примера на рисунке 1.7, регистрация – это не просто сбор учетных данных, а первичный механизм разделения функционала.

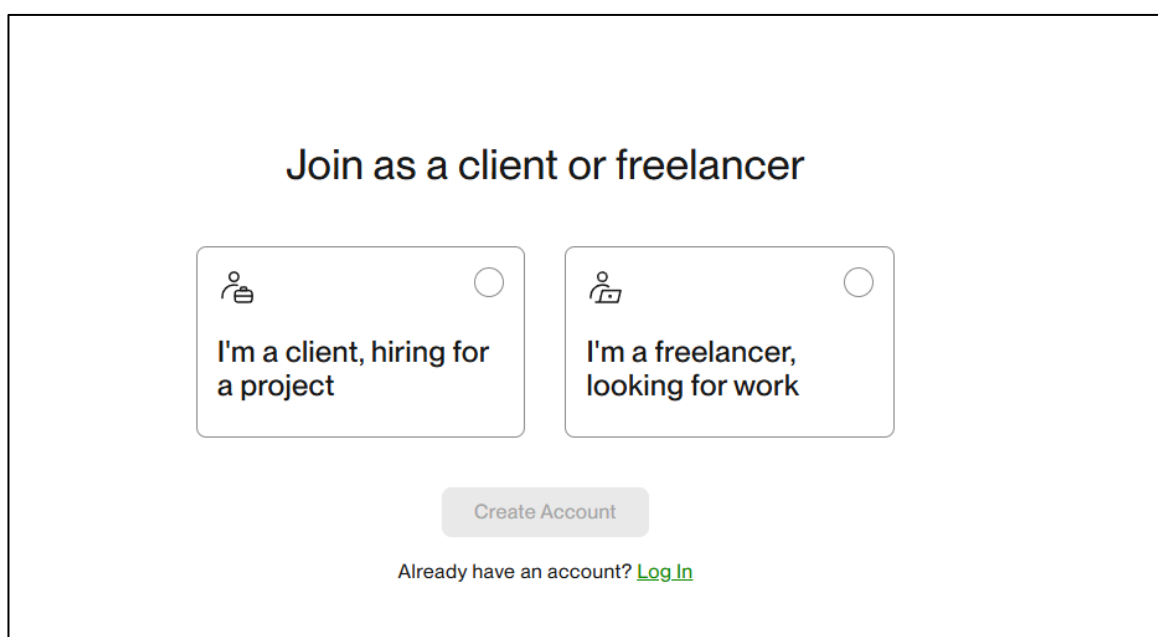


Рисунок 1.7 – Обзор регистрации пользователя

Сразу после ввода базовых данных система ставит пользователя перед следующим ключевым выбором: «Присоединиться как клиент» или «Присоединиться как фрилансер». Этот шаг является фундаментальным для всей архитектуры приложения, поскольку он мгновенно сегментирует пользователей и направляет их в совершенно разные функциональные воронки.

Выбор роли «Клиент» настраивает интерфейс пользователя на выполнение задач, связанных с поиском талантов: создание профиля компании, публикацию заказов, управление платежами и просмотр каталога исполнителей.

В то же время, выбор роли «Фрилансер» запускает сложный, многошаговый процесс создания профиля пользователя, этот шаг, по сути, является созданием резюме. Система Upwork заставляет пользователя заполнить образование, навыки, портфолио и опыт, прежде чем предоставить ему полный доступ к ленте заказов.

1.3 Выводы по разделу

Проведенный анализ функционала ведущих фриланс-платформ Fiverr, Freelancer и Upwork позволил определить оптимальный подход к проектированию веб-приложения. Целью анализа было не копирование, а адаптация существующих бизнес-моделей под специфические нужды нишевого рынка: устранение барьеров для входа студентов и предоставление заказчикам удобного инструмента для поиска квалифицированных новичков.

Критически важной частью адаптации стало принятие решений об отказе от некоторых функциональных элементов. Во-первых, категорически отвергается модель аукциона, присущая Freelancer, поскольку она поощряет демпинг и обесценивает труд начинающих специалистов. Вместо этого, функция «Откликнуться на заказ» будет ориентирована на демонстрацию квалификации, прикрепление резюме и сопроводительного письма. Во-вторых, полностью исключается механизм платных откликов, используемый на Upwork. Платный барьер для студентов-новичков противоречит главной цели проекта – бесплатный и свободный доступ к опыту.

Для обеспечения прозрачности и доверия будет заимствован механизм детализированного профиля, который станет полноценным резюме. Процесс регистрации будет включать первичное разделение ролей для настройки интерфейса. Наконец, будет реализована сквозная репутационная система с оценками и отзывами и встроенный чат для коммуникации. Проектируемое веб-приложение будет функционально богатым и социально-ориентированным, создавая безопасную и эффективную среду для старта своей карьеры.

Выбранная гибридная модель с бесплатным доступом и упором на структурированное резюме решает изначально поставленную проблему отсутствия специализированной нишевой платформы. В результате студенты получают доступное веб-приложение, которое не требует никаких финансовых вложений и не ставит их в прямую ценовую конкуренцию с опытными фрилансерами. Заказчики, в свою очередь, получают возможность целенаправленно искать мотивированных исполнителей по фильтрам образования и навыков, что невозможно на глобальных биржах. Интеграция функционала услуг позволяет студентам-новичкам продавать стандартизированные микро-задачи, тем самым быстро накапливая портфолио и положительные отзывы, что является ключевым для их дальнейшего профессионального роста.

2 Проектирование веб-приложения

Успешное архитектурное решение напрямую влияет на уровень связности компонентов, что важно для дальнейшего развития и внесения изменений без нарушения целостности системы. В контексте веб-приложения «Маркет-плейс фриланс-заказов для студентов» особое внимание уделяется модульности и соответствию принципам асинхронного программирования на Node.js.

2.1 Выбор средств реализации

Для разработки выбран стек технологий, обеспечивающий производительность, масштабируемость и удобство разработки.

Серверная часть реализована на Node.js с использованием Express.js [6]. Node.js подходит для высоконагруженных приложений с множеством одновременных подключений. Express.js предоставляет гибкую архитектуру REST API и упрощает маршрутизацию и обработку запросов.

В качестве системы управления базами данных выбрана MySQL [7]. Она обеспечивает надежное хранение и целостность данных, поддерживает сложные связи между сущностями и хорошо интегрируется с выбранным ORM. Для работы с базой данных используется Sequelize [8], который позволяет описывать модели данных на JavaScript, автоматизирует миграции и упрощает построение запросов с учетом связей между таблицами.

Для аутентификации и авторизации применяется JWT. Токены хранятся в защищенных HTTP-only cookies через cookie-session, что повышает безопасность и снижает риск XSS. Реализована система ролей.

Для обмена сообщениями в реальном времени используется Socket.io. Это позволяет отправлять уведомления, обновлять списки чатов и доставлять сообщения без перезагрузки страницы. Socket.io интегрирован с системой аутентификации через JWT для безопасного подключения.

Frontend разработан на React [9] с TypeScript. TypeScript добавляет статическую типизацию, что снижает ошибки и упрощает поддержку. React обеспечивает компонентную архитектуру и эффективное обновление интерфейса.

В качестве инструмента сборки выбран Vite. Он обеспечивает быструю разработку с горячей перезагрузкой модулей и оптимизированную сборку для production. Vite интегрирован с React через официальный плагин.

Для стилизации интерфейса используется Tailwind CSS. Подход utility-first ускоряет разработку, обеспечивает консистентность дизайна и позволяет создавать адаптивные интерфейсы без написания большого объема CSS.

				ДП 02.00.ПЗ						
	ФИО	Подпись	Дата							
Разраб.	Глухова Д.В.			2 Проектирование веб-приложения			Лит.	Лист	Листов	
Пров.	Уласевич Н.И.						У	1	14	
							БГТУ 1-40 05 01, 2026			
Н. контр.	Николайчук А.Н.									
Утв.	Блинова Е.А.									

Для HTTP-запросов к серверу используется Axios. Он предоставляет удобный API, поддерживает interceptors для автоматической обработки токенов и ошибок, а также упрощает работу с асинхронными запросами.

Маршрутизация на клиенте реализована через React Router DOM. Наличие этой библиотеки позволяет создавать одностраничное приложение с навигацией между разными частями веб-приложения без перезагрузки страницы, защищать маршруты через компоненты и управлять состоянием навигации.

Для отправки email-уведомлений реализована интеграция с Gmail API. Это позволяет отправлять письма для верификации аккаунтов и уведомлений пользователям напрямую через сервис Google.

Выбранный стек обеспечивает разделение ответственности между клиентом и сервером, упрощает разработку и тестирование, а также позволяет масштабировать веб-приложение при росте нагрузки.

2.2 Разработка функциональных требований

На рисунке 2.1 представлена диаграмма вариантов использования [10], на которой указаны пять актеров: гость, заказчик и исполнитель, менеджер, администратор, а также обобщение заказчика и исполнителя в пользователь.

В основе системы лежит гость, являющийся незарегистрированным посетителем. Его функционал максимально ограничен и направлен на вовлечение. Гость может просматривать каталог доступных предложений для ознакомления с рынком. Самые важные для него действия – регистрация в системе и вход в систему, которые являются необходимыми шагами для получения статуса пользователя и доступа к расширенным функциям.

После успешного входа гость становится пользователем. Эта роль представляет собой базового зарегистрированного члена сообщества, и, что важно, она является обобщающей для ролей заказчика и исполнителя, что означает, что некоторые их возможности наследуются от пользователя.

Пользователь имеет доступ к основной навигации и коммуникациям: он может просматривать категории услуг и просматривать профили других участников. Для общения предусмотрена работа с чатом, которая обязательно включает просмотр чата и может дополнительно расширяться отправкой сообщения. Управление уведомлениями включает в себя работу с уведомлениями, которая требует просмотр уведомлений и может быть расширена возможностью пометить как прочитанное, удалить уведомление или прочитать все. Работа с профилем – ключевой элемент персонализации, включающий обязательный просмотр профиля и расширенное редактирование профиля. Важная часть системы – это управление репутацией: работа с отзывами, которая обязательно включает просмотр отзывов и может быть расширена добавлением отзыва. Для удобства поиска и сохранения интересных предложений реализована работа с избранным, которая всегда включает просмотр избранного и может быть расширена добавлением понравившегося пользователя в избранное.

Заказчик – это ключевая специализированная роль в системе, которая архитектурно наследует базовый набор прав и возможностей стандартного

пользователя (аутентификация, доступ к личному кабинету, настройки профиля), но при этом функционально ориентирована на формирование спроса. Основная деятельность заказчика концентрируется на потреблении услуг, а также реализуется через расширенный интерфейс управления заявками. Данный функционал предоставляет пользователю контроль над жизненным циклом услуги: от инициализации потребности до финализации сотрудничества.

Исполнитель – вторая специализированная роль, также наследующая все от пользователя, но с фокусом на предоставлении услуг и выполнении заказов. Его функционал расширен в двух ключевых областях. Во-первых, это работа с заказ, позволяющая ему создавать заказ, удалять заказ и редактировать заказ. А также возможность откликов на услуги заказчиков.

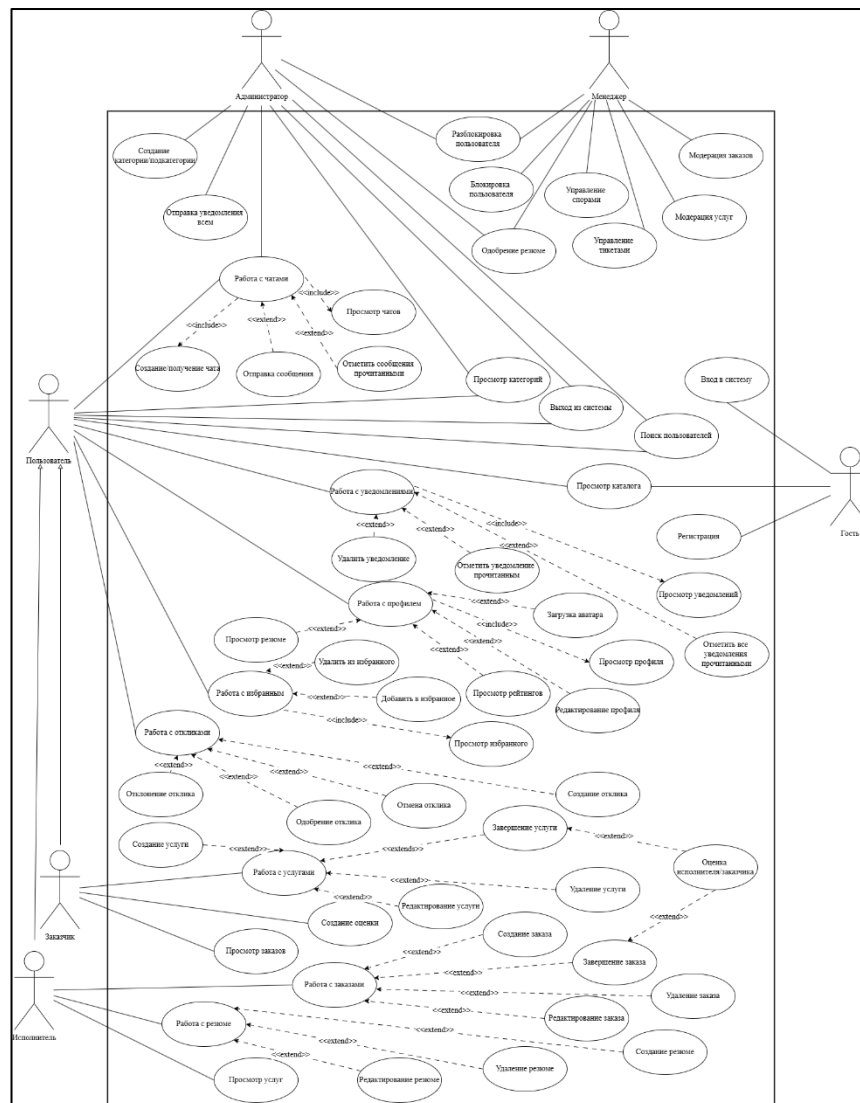


Рисунок 2.1 – Диаграмма вариантов использования

Роль менеджера сфокусирована на контроле качества контента и разрешении конфликтов между участниками. Данный актер выполняет модерацию заказов и услуг для поддержания порядка на платформе. Менеджер управляет доступом к системе через механизмы блокировки и разблокировки аккаунтов. В обязанности роли входит проверка и одобрение резюме исполнителей для

подтверждения квалификации студентов. Для решения технических и организационных проблем предусмотрено управление тикетами и спорами. Деятельность менеджера снижает риски для всех сторон и обеспечивает безопасность взаимодействия внутри маркетплейса.

Диаграмма моделирует фриланс-маркетплейс с разграниченными ролями и процессом взаимодействия, основанном на услугах, заказах и оценках.

2.3 Разработка базы данных

Логическая схема базы данных – это визуальное представление структуры базы данных и отношений между таблицами, которые хранятся в базе.

Для реализации веб-приложения была спроектирована логическая схема базы данных. База данных построена на основе реляционной модели и включает в себя набор взаимосвязанных таблиц, предназначенных для эффективного хранения и управления информацией о пользователях, их профилях, ролях, фриланс-услугах, категориях, заказах, заявках, сообщениях, уведомлениях и рейтингах. Логическая схема базы данных для фриланс-маркетплейса представлена на рисунке 2.2.

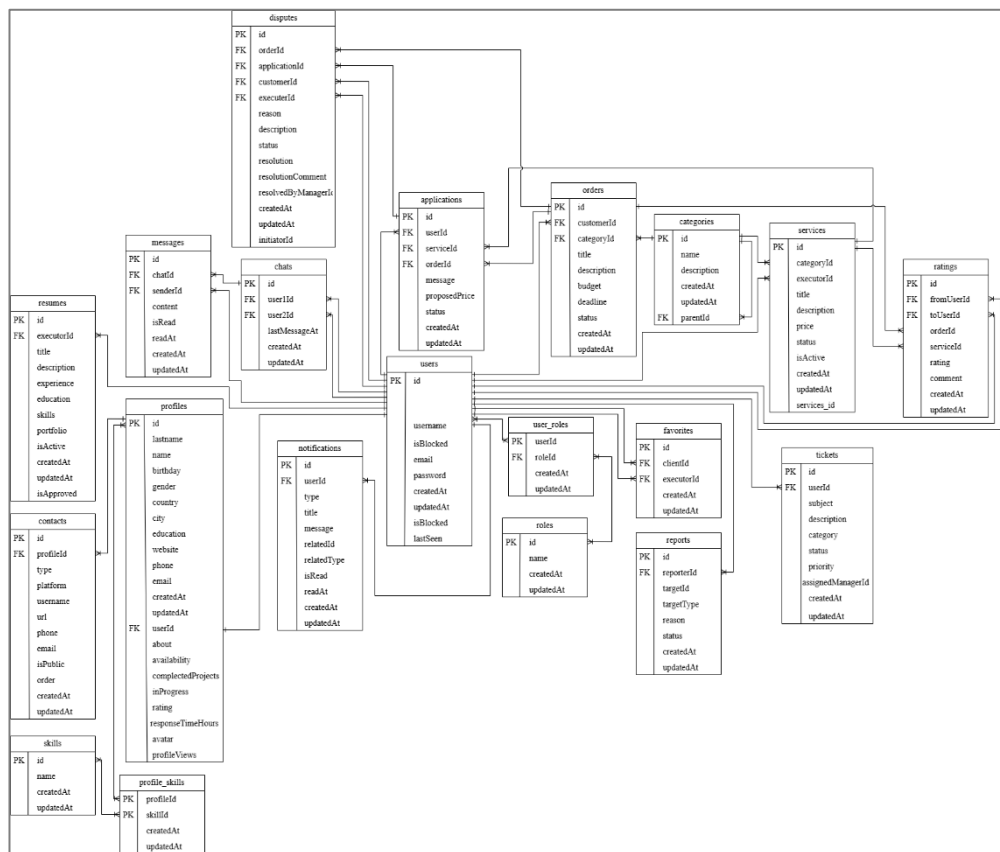


Рисунок 2.2 – Логическая схема базы данных таблиц

В базе данных находится 17 основных таблиц и 2 промежуточные, разработанные для обеспечения полного функционала маркетплейса, включая разделение ролей, управление проектами, услугами и репутацией.

Таблица users хранит данные для идентификации пользователей, зарегистрированных в приложении. Структура таблицы представлена в таблице 2.1.

Таблица 2.1 – Структура таблицы «users»

Название столбца	Тип, ограничение целостности	Описание столбца
id	INT, PRIMARY KEY, AUTO INCREMENT	Уникальный идентификатор пользователя
username	VARCHAR(255)	Имя пользователя (логин)
email	VARCHAR(255)	Адрес электронной почты
password	VARCHAR(255)	Хэш-сумма пароля
createdAt, updatedAt	DATETIME, NOT NULL	Временные метки создания/обновления записи
isBlocked	TINYINT(1), DEFAULT '0'	Флаг глобальной блокировки пользователя
lastSeen	DATETIME	Дата и время последнего посещения

Таблица roles, представленная в таблице 2.2, содержит список разрешенных ролей в системе. Столбец name содержит наименование роли: «Customer» для заказчика, «Executer» для исполнителя, «Administrator» для администратора, а также «Manager» для менеджера.

Таблица 2.2 – Структура таблицы «roles»

Название столбца	Тип, ограничение целостности	Описание столбца
id	INT, PRIMARY KEY	Уникальный идентификатор роли
name	VARCHAR(255)	Название роли
createdAt, updatedAt	DATETIME, NOT NULL	Временные метки создания/обновления записи

Таблица 2.3 – user_roles является промежуточной таблицей для реализации связи «многие-ко-многим» между users и roles. Соединяет userId и roleId, определяя, какие права доступа имеет пользователь.

Таблица 2.3 – Структура таблицы «user_roles»

Название столбца	Тип, ограничение целостности	Описание столбца
roleId	INT, PRIMARY KEY, FOREIGN KEY (roles.id)	Уникальный идентификатор роли
userId	INT, PRIMARY KEY, FOREIGN KEY (users.id)	Уникальный идентификатор пользователя
createdAt, updatedAt	DATETIME, NOT NULL	Временные метки создания/обновления записи

Таблица profiles хранит расширенные персональные данные пользователей. Столбец userId содержит внешний ключ, ссылающийся на users, обеспечивая связь 1:1. Структура таблицы представлена в таблице 2.4.

Таблица 2.4 – Структура таблицы «profiles»

Название столбца	Тип, ограничение целостности	Описание столбца
id	INT, PRIMARY KEY, AUTO_INCREMENT	Уникальный идентификатор профиля
userId	INT, FOREIGN KEY (users.id)	Идентификатор связанного пользователя
lastname, name	VARCHAR(255)	Фамилия и имя
birthday	DATETIME	Дата рождения
gender, country, city	VARCHAR(255)	Личные и географические данные
about	TEXT	Биография/общее описание профиля
rating	FLOAT, DEFAULT '0'	Общий рейтинг пользователя
completedProjects	INT, DEFAULT '0'	Количество выполненных проектов
responseTimeHours	INT, DEFAULT '0'	Среднее время ответа (в часах)
avatar	LONGTEXT	Бинарные данные или ссылка на изображение профиля
profileViews	INT, DEFAULT '0'	Количество просмотров профиля

Таблица contacts предназначена для хранения разнообразных и дополнительных контактных данных, которые не входят в базовый набор информации профиля, таких как ссылки на социальные сети, мессенджеры или альтернативные номера. Столбец profileId является внешним ключом, связывающим контакт с профилем. Структура таблицы contacts представлена в таблице 2.5.

Таблица 2.5 – Структура таблицы «contacts»

Название столбца	Тип, ограничение целостности	Описание столбца
id	INT, PRIMARY KEY, AUTO_INCREMENT	Идентификатор контакта
profileId	INT, NOT NULL, FOREIGN KEY (profiles.id)	Идентификатор профиля
type	ENUM, NOT NULL	Тип контакта ('messenger', 'social', 'other')
platform	VARCHAR(255), NOT NULL	Название платформы (например, 'Telegram')
url	VARCHAR(255)	Ссылка на контакт
isPublic	TINYINT(1), DEFAULT '1'	Флаг публичной видимости контакта

Таблица categories содержит список категорий для классификации фриланс-заказов и услуг. Структура таблицы представлена в таблице 2.6.

Таблица 2.6 – Структура таблицы «categories»

Название столбца	Тип, ограничение целостности	Описание столбца
1	2	3
id	INT, PRIMARY KEY, AUTO_INCREMENT	Идентификатор категории

Продолжение таблицы 2.6

1	2	3
name	VARCHAR(255), UNIQUE, NOT NULL	Название категории
description	TEXT	Описание категории
parentId	INT, FOREIGN KEY (categories.id)	Ссылка на родительскую категорию

Таблица services содержит информацию о карточках услуг, опубликованных Исполнителями. Структура таблицы представлена в таблице 2.7.

Таблица 2.7 – Структура таблицы «services»

Название столбца	Тип, ограничение целостности	Описание столбца
id	INT, PRIMARY KEY, AUTO_INCREMENT	Идентификатор услуги
executorId	INT, NOT NULL, FOREIGN KEY (users.id)	Идентификатор исполнителя
categoryId	INT, NOT NULL, FOREIGN KEY (categories.id)	Идентификатор категории услуги
title	VARCHAR(255), NOT NULL	Название услуги
description	TEXT, NOT NULL	Описание услуги
price	DECIMAL(10,2)	Бюджет проекта
status	ENUM, DEFAULT 'open'	Статус услуги ('open', 'in_progress', 'completed', 'cancelled')

Таблица 2.8 – orders является центральной транзакционной сущностью со стороны заказчика. Она хранит всю информацию о созданных проектах или заданиях, которые требуют выполнения. Ключевые поля включают customerId, budget и deadline. Поле status отслеживает текущее состояние заказа в процессе работы, начиная с open и заканчивая completed или cancelled. Эта таблица является основной целью для исполнителей, подающих заявки, и используется для отслеживания всех активных и завершенных сделок.

Таблица 2.8 – Структура таблицы «orders»

Название столбца	Тип, ограничение целостности	Описание столбца
1	2	3
id	INT, PRIMARY KEY, AUTO_INCREMENT	Идентификатор заявки
customerId	INT, NOT NULL, FOREIGN KEY (users.id)	Идентификатор исполнителя, подавшего заявку
categoryId	INT, FOREIGN KEY (orders.id)	Идентификатор категории
title	INT, FOREIGN KEY (services.id)	Заголовок услуги
description	TEXT	Сопроводительное сообщение/предложение
budget	DECIMAL(10,2)	Бюджет заказа

Продолжение таблицы 2.8

1	2	3
deadline	DATETIME	Срок окончания заказа
status	ENUM, DEFAULT 'pending'	Статус заявки ('pending', 'approved', 'rejected')

Таблица applications фиксирует все заявки и предложения от исполнителей в ответ на открытые заказы или на услуги, предлагаемые другими исполнителями. Она содержит сопроводительное message и proposedPrice, являясь официальной формой выражения интереса к работе. Поле status (например, pending, approved, rejected) критически важно для управления процессом найма. Эта таблица является мостом, связывающим заказчиков и исполнителей в начале рабочего процесса. Структура представлена в таблице 2.9.

Таблица 2.9 – Структура таблицы «applications»

Название столбца	Тип, ограничение целостности	Описание столбца
id	INT, PRIMARY KEY, AUTO INCREMENT	Идентификатор заявки
userId	INT, NOT NULL, FOREIGN KEY (users.id)	Идентификатор исполнителя, подавшего заявку
orderId	INT, FOREIGN KEY (orders.id)	Ссылка на связанный заказ
serviceId	INT, FOREIGN KEY (services.id)	Ссылка на связанную услугу
message	TEXT	Сопроводительное сообщение/предложение
proposedPrice	DECIMAL(10,2)	Предложенная цена
status	ENUM, DEFAULT 'pending'	Статус заявки ('pending', 'approved', 'rejected')

Таблица 2.10 – chats предназначена для организации диалогов между двумя пользователями. Таблица содержит поле id, которое хранит айди диалога, а также таблица хранит пары user1Id и user2Id, уникально идентифицируя каждый разговор. Уникальный индекс на этой паре гарантирует, что между двумя пользователями может быть создан только один чат. Поле lastMessageAt используется для быстрой сортировки чатов по актуальности, позволяя пользователю видеть самые свежие диалоги вверху списка.

Таблица 2.10 – Структура таблицы «chats»

Название столбца	Тип, ограничение целостности	Описание столбца
id	INT, PRIMARY KEY, AUTO INCREMENT	Идентификатор чата
user1Id, user2Id	INT, NOT NULL, FOREIGN KEY (users.id)	Идентификаторы двух участников чата
lastMessageAt	DATETIME	Время последнего сообщения (для сортировки)

Таблица `messages` хранит фактическое содержимое каждого сообщения, отправленного в чате. Каждое сообщение обязательно привязывается к `chatId` и имеет ссылку на `senderId`. Поля `isRead` и `readAt` обеспечивают функциональность уведомлений о прочтении, которая является критически важной для оперативной коммуникации. Таблица поддерживает функцию работы с чатом из диаграммы вариантов использования. Структура представлена в таблице 2.11.

Таблица 2.11 – Структура таблицы «messages»

Название столбца	Тип, ограничение целостности	Описание столбца
<code>id</code>	INT, PRIMARY KEY, AUTO INCREMENT	Идентификатор сообщения
<code>chatId</code>	INT, NOT NULL, FOREIGN KEY (chats.id)	Идентификатор чата
<code>senderId</code>	INT, NOT NULL, FOREIGN KEY (users.id)	Идентификатор отправителя
<code>content</code>	TEXT, NOT NULL	Текст сообщения
<code>isRead</code>	TINYINT(1), DEFAULT '0'	Флаг прочтения сообщения
<code>readAt</code>	DATETIME	Время прочтения сообщения

Таблица `notifications` служит для информирования пользователей о системных событиях. Она привязывает уведомление к конкретному `userId` и использует поля `relatedId` и `relatedType` для создания контекстной ссылки на объект, который вызвал событие. Структура представлена в таблице 2.12.

Таблица 2.12 – Структура таблицы «notifications»

Название столбца	Тип, ограничение целостности	Описание столбца
<code>id</code>	INT, PRIMARY KEY, AUTO INCREMENT	Идентификатор уведомления
<code>userId</code>	INT, NOT NULL, FOREIGN KEY (users.id)	Идентификатор получателя
<code>type</code>	ENUM, NOT NULL	Тип события (например, 'new_application', 'new_message')
<code>title, message</code>	VARCHAR(255), TEXT, NOT NULL	Заголовок и текст уведомления
<code>relatedId</code>	INT	Идентификатор связанной сущности (например, <code>orderId</code>)
<code>relatedType</code>	ENUM	Тип связанной сущности ('order', 'service', 'application' и т.д.)
<code>isRead</code>	TINYINT(1), DEFAULT '0'	Флаг прочтения

Таблица 2.13 – `ratings` является основой системы репутаций маркетплейса. Она фиксирует каждую оценку (`rating`) и отзыв (`comment`), выставленный пользователем (`fromUserId`) другому пользователю (`toUserId`) по завершении сделки. Каждая запись может быть связана с конкретным `orderId` или `serviceId`, обеспечивая прозрачность и контекст оценки. Данные из этой таблицы агрегируются для расчета общего рейтинга в таблице `profiles`.

Таблица 2.13 – Структура таблицы «ratings»

Название столбца	Тип, ограничение целостности	Описание столбца
id	INT, PRIMARY KEY, AUTO INCREMENT	Идентификатор оценки
fromUserId, toUserId	INT, NOT NULL, FOREIGN KEY (users.id)	Идентификаторы пользователя, который оценил, и того, кого оценили
orderId, serviceId	INT, FOREIGN KEY	Ссылка на связанный заказ или услугу
rating	INT, NOT NULL	Числовая оценка (например, от 1 до 5)
comment	TEXT	Текст отзыва

Таблица skills представляет собой справочник всех профессиональных навыков, доступных для выбора исполнителями.

Таблица profile_skills реализует отношение «многие-ко-многим» между профилями и навыками, позволяя одному исполнителю указать несколько навыков и одному навыку быть связанным со многими исполнителями. Эта связка является ключевой для эффективного поиска и фильтрации исполнителей по их компетенциям. Структура – таблица 2.14.

Таблица 2.14 – Структура таблиц «skills» и «profile_skills»

Название столбца	Таблица	Тип, ограничение целостности	Описание столбца
id	skills	INT, PRIMARY KEY, AUTO INCREMENT	Идентификатор навыка
name	skills	VARCHAR(255), UNIQUE, NOT NULL	Название навыка
profileId	profile_skills	INT, PRIMARY KEY, FOREIGN KEY (profiles.id)	Идентификатор профиля
skillId	profile_skills	INT, PRIMARY KEY, FOREIGN KEY (skills.id)	Идентификатор навыка

Таблица favorites хранит связи между заказчиками и выбранными ими исполнителями. Заказчики сохраняют профили студентов в личный список для быстрого доступа к информации. Инструмент исключает необходимость повторного поиска специалистов в общем каталоге. Это сокращает время на подбор исполнителя для новых задач. Техническая реализация предотвращает дублирование данных через уникальный индекс на полях customer_id и executor_id. Индекс гарантирует уникальность каждой записи в рамках аккаунта заказчика. Структура полей и типы данных описаны в таблице 2.15.

Для предотвращения дублирования записей в таблице базы данных используется уникальный индекс на комбинации двух полей customerid и executorid. Такой уникальный индекс гарантирует, что один и тот же исполнитель не может быть добавлен в избранное более одного раза для заказчика. Структура таблиц представлена в таблице 2.15.

Таблица 2.15 – Структура таблиц «skills» и «profile_skills»

Название столбца	Тип, ограничение целостности	Описание столбца
id	INT, PRIMARY KEY, AUTO INCREMENT	Идентификатор записи избранного
customerId	INT, NOT NULL, FOREIGN KEY (users.id)	Идентификатор Заказчика
executerId	INT, NOT NULL, FOREIGN KEY (users.id)	Идентификатор избранного Исполнителя

Проектирование базы данных для фриланс-маркетплейса было выполнено с использованием MySQL и реляционной модели, обеспечивающей высокую целостность данных и эффективную обработку транзакций.

2.4 Архитектура веб-приложения

Дипломный проект построен на клиент-серверной архитектуре. Клиент-серверная архитектура – это архитектура, которая подразумевает две компоненты: клиент и сервер. Клиент является инициатором соединения. В качестве сервера будет выступать приложение на Node.js. В качестве клиента будет выступать приложение с асинхронным UI (React). Схема архитектуры представлена в приложении А. Схема развёртывания [11] представлена на рисунке 2.3.

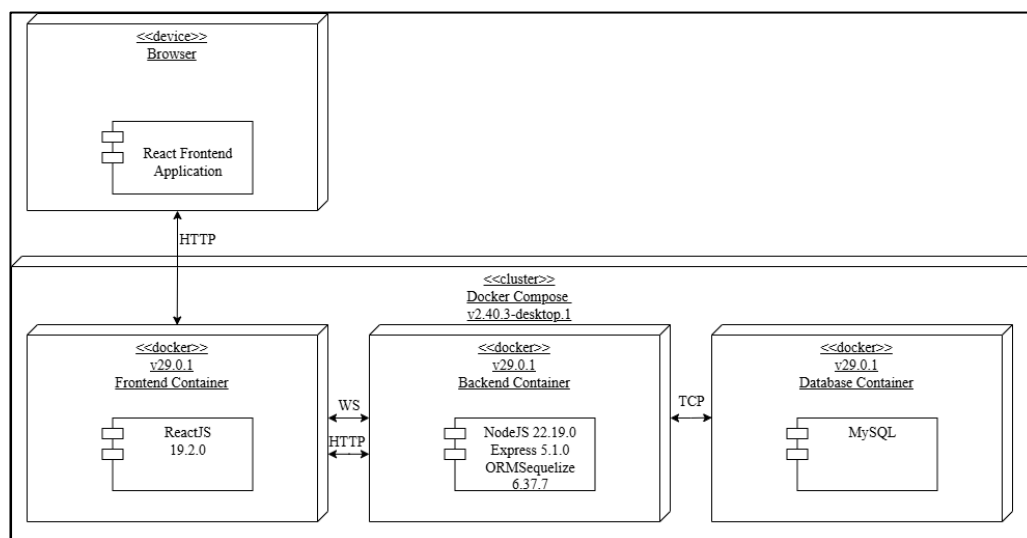


Рисунок 2.3 – Схема развёртывания приложения

Схема описывает многоуровневую архитектуру развёртывания веб-приложения, которая использует контейнеризацию для изоляции и управления компонентами. Верхний уровень представлен пользователем, на котором работает Фронтенд-приложение. Это приложение взаимодействует с остальной частью по сети, выступая в роли клиентского интерфейса.

Основная часть развёртывания реализована как кластер Docker Compose [12], что позволяет определить и запустить многоконтейнерное приложение с помощью одного файла конфигурации. Этот подход обеспечивает

воспроизводимость среды и упрощает управление зависимостями между сервисами. Внутри кластера выделяются три основных контейнера, соответствующих типичному стеку web-разработки: фронтенд, бэкенд и база данных.

Первый контейнер, Frontend Container, предназначен для размещения и запуска интерфейсной части приложения, построенной на ReactJS. Контейнер изолирует среду выполнения фронтенда и взаимодействует с бэкенд-сервисом. Взаимодействие происходит по протоколам HTTP для стандартных запросов и, вероятно, по WS (WebSocket) для организации двунаправленного, постоянного соединения, что используется для обмена в реальном времени.

Второй контейнер, Backend Container, содержит серверную логику приложения, реализованную с использованием Node.js и фреймворка Express. Для работы с данными здесь также задействован ORM Sequelize, который абстрагирует операции с базой данных и обеспечивает более безопасный и структурированный доступ к данным. Бэкенд-контейнер выступает в роли посредника, обрабатывая запросы от фронтенда, применяя бизнес-логику и взаимодействуя с хранилищем данных.

Третий контейнер, Database Container, предназначен для хранения данных и использует систему управления базами данных MySQL. Этот контейнер изолирует базу данных от логики приложения и обеспечивает персистентность данных. Взаимодействие бэкенд-контейнера с базой данных происходит по протоколу TCP, который является стандартным транспортным протоколом для большинства СУБД, обеспечивая надежную передачу данных между сервером приложений и сервером базы данных.

2.5 Проектирование алгоритмов

В рамках главы, посвященной проектированию веб-приложения, ключевым этапом является формализация логики обработки пользовательских запросов. Данный раздел описывает алгоритм создания и отправки сообщения, который представляет одну из основных функций разрабатываемой системы. Целью построения данной схемы является визуализация последовательности действий системы, начиная от получения запроса и заканчивая сохранением данных или возвратом ошибки, с особым акцентом на проверку прав доступа и валидацию входных данных перед выполнением целевого действия.

Согласно представленной на рисунке 2.4 блок-схеме, выполнение алгоритма отправки сообщений начинается с проверки статуса авторизации пользователя. Если система определяет, что пользователь не авторизован, процесс немедленно прерывается возвратом статуса «401 Unauthorized», что предотвращает несанкционированный доступ к функционалу. В случае успешной аутентификации система переходит к извлечению необходимых данных из запроса, а именно идентификатора чата (chatid) и текстового содержимого (content). Далее следует этап валидации, на котором проверяется, не является ли тело сообщения пустым. Если контент отсутствует, алгоритм завершается ошибкой «400», информирующей о невозможности отправки пустого сообщения. При успешном прохождении валидации инициируется поиск

соответствующего чата в базе данных. Если чат с указанным идентификатором не найден, возвращается ошибка «404». Однако существование чата не гарантирует успешную отправку: критически важным является следующий шаг – проверка принадлежности пользователя к данному чату. Если пользователь не является участником, система блокирует действие ошибкой «403 Forbidden». Только после успешного прохождения всех этапов верификации происходит непосредственное создание записи сообщения в базе данных и формирование соответствующего уведомления. На финальном этапе алгоритм проверяет доступность Socket.io; если сервис активен, инициируется отправка событий для мгновенной доставки сообщения подключенным клиентам, после чего процесс завершается возвратом успешного статуса и объекта сообщения.

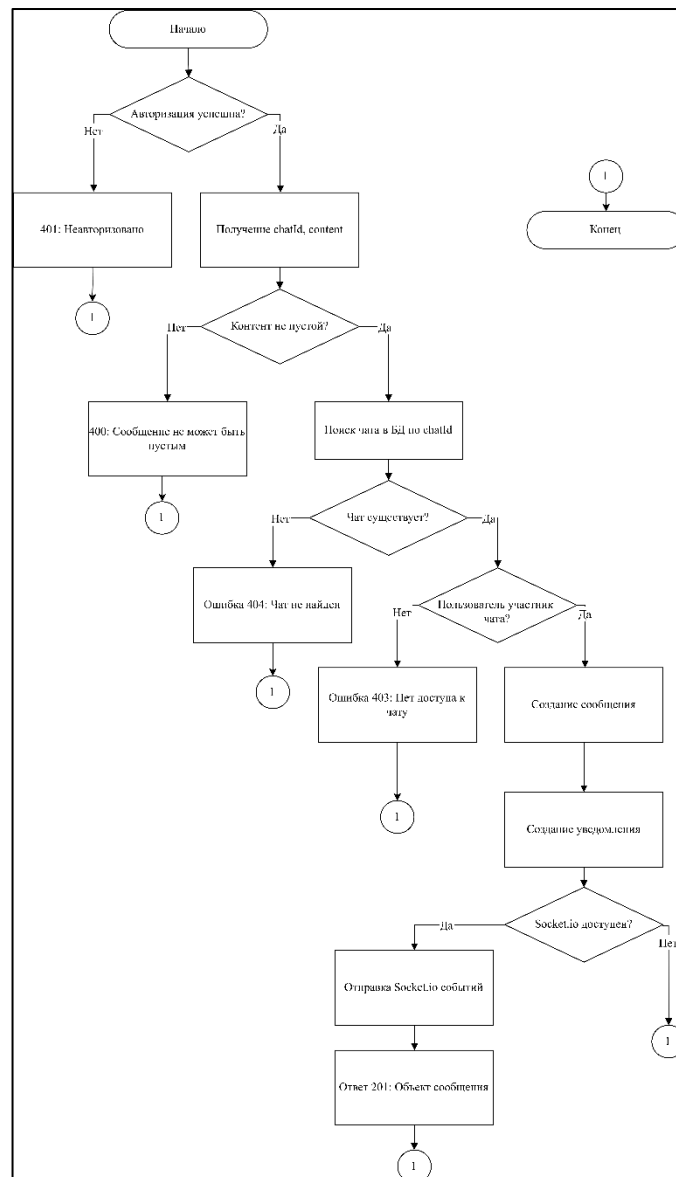


Рисунок 2.4 – Блок-схема обработки и маршрутизации сообщений

Анализ приведенного алгоритма позволяет сделать вывод о высокой надежности спроектированного процесса обработки данных. Строгая последовательность проверок (авторизация, валидация, существование ресурса, права

доступа) обеспечивает безопасность системы, отсекая некорректные или вредоносные запросы до момента обращения к операциям записи. Кроме того, интеграция условной проверки доступности Socket.io делает архитектуру гибкой, позволяя системе корректно функционировать и сохранять данные даже в случае временной недоступности сервиса реального времени, возвращая стандартизированный ответ клиенту.

Алгоритм, представленный на рисунке 2.5, описывает многоуровневый процесс обработки запроса на публикацию заказа в рамках серверной логики.

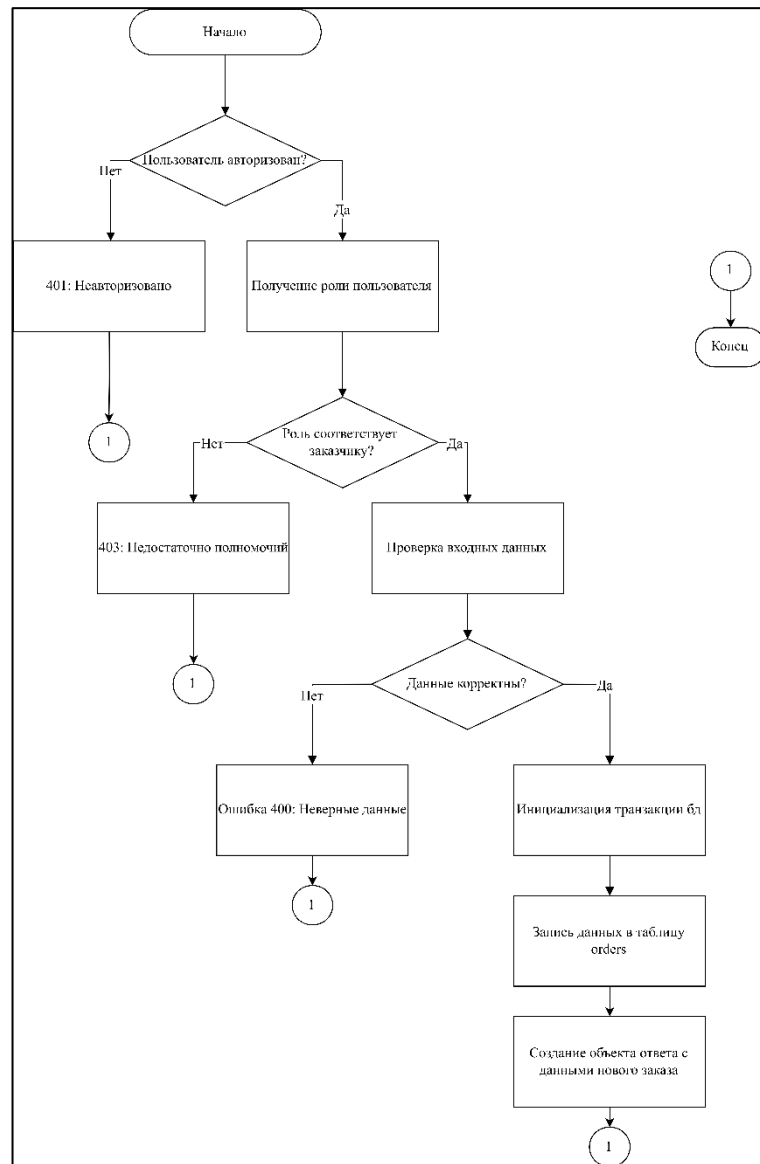


Рисунок 2.5 – Блок-схема процесса создания заказа

Работа начинается с этапа аутентификации, где проверяется наличие активной сессии пользователя, и в случае неудачи выполнение прерывается с возвратом ошибки 401 unauthorized. Далее следует критически важный блок инспекции прав доступа (RBAC check), который обращается к ролевой модели пользователя для подтверждения статуса заказчика; при отсутствии необходимых полномочий система генерирует исключение 403 forbidden. После успешной верификации субъекта запускается этап валидации входных данных, где

проверяется корректность заполнения полей заголовка, бюджета и категории, а любые несоответствия приводят к ответу 400 bad request. Заключительная фаза алгоритма включает инициализацию транзакции базы данных для персистенции сущности в таблицу orders, последующую агрегацию связанных метаданных и формирование финального http-ответа 201, что официально завершает процесс создания заявки.

2.6 Выводы по разделу

Разработка архитектуры веб-приложения «Маркетплейс фриланс-заказов для студентов» выполнена с акцентом на модульность, высокую связность компонентов и использование асинхронных технологий, что является критически важным для обеспечения масштабируемости системы и удобства ее дальнейшей модификации. Серверная часть, реализованная на платформе Node.js с использованием фреймворка Express, берет на себя обработку бизнес-логики и маршрутизацию API, в то время как клиентская часть на ReactJS обеспечивает отзывчивый пользовательский интерфейс. Взаимодействие с данными организовано через реляционную базу данных MySQL с применением ORM Sequelize, что создает необходимый слой абстракции и гарантирует целостность хранимой информации.

Существенным этапом проектирования стала детальная проработка алгоритмической базы приложения, в частности, процесса обработки пользовательских сообщений. Разработанная блок-схема демонстрирует строгий подход к безопасности: алгоритм включает последовательные этапы авторизации, валидации входных данных и проверки прав доступа к чату до момента внесения записи в базу данных. Включение в логику работы проверки доступности сервиса Socket.io подтверждает ориентацию системы на работу в режиме реального времени, обеспечивая мгновенную доставку уведомлений пользователям при сохранении надежности транзакций.

Завершает архитектурное решение схема развертывания, основанная на принципах DevOps с использованием контейнеризации Docker и оркестрации Docker Compose. Такой подход позволяет изолировать фронтенд, бэкенд и базу данных в независимые контейнеры, что упрощает перенос приложения между средами разработки и эксплуатации. В совокупности спроектированная структура данных, алгоритмы обработки запросов и среда развертывания формируют надежный технологический фундамент для реализации функционального и отказоустойчивого программного продукта.

3 Разработка веб-приложения

В рамках проекта реализована структура, разделяющая серверную и клиентскую части, что значительно упрощает процессы разработки, сопровождения и последующего масштабирования приложения.

3.1 Разработка серверной части

Серверная часть приложения расположена в директории backend и включает в себя компоненты, необходимые для реализации REST API и функциональности WebSocket. Приложение разработано на платформе Node.js с использованием фреймворка Express.js 5.1.0 и языка программирования JavaScript. Для обеспечения работы с базой данных используется ORM Sequelize 6.37.7 в связке с MySQL. Реализация real-time коммуникации, критически важной для чатов и уведомлений, достигнута с помощью Socket.IO 4.8.1. Аутентификация пользователей построена на механизме JWT (jsonwebtoken 9.0.2) с использованием cookie-session 2.1.1 для управления сессиями. Для безопасного хеширования паролей применяется bcryptjs 3.0.2, а отправка email-уведомлений реализована с помощью nodemailer 6.10.1.

Архитектура серверной части организована по модульному принципу, где каждый функциональный аспект приложения выделен в отдельную директорию. Это обеспечивает четкое разделение обязанностей, упрощая поддержку и расширение системы. В данном разделе рассмотрим основные директории серверной части.

На рисунке 3.1 представлена структура серверной части, демонстрирующая архитектуру серверной логики.

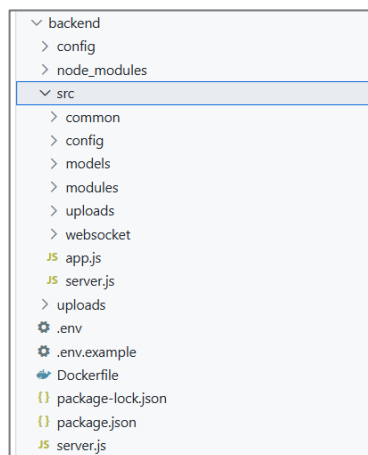


Рисунок 3.1 – Структура серверной части

				ДП 03.00.ПЗ				
	ФИО	Подпись	Дата					
Разраб.	Глухова Д.В.			3 Разработка веб-приложения	Лит.		Лист	Листов
Пров.	Уласевич Н.И.				У		1	15
					БГТУ 1-40 05 01, 2026			
Н. контр.	Николайчук А.Н.							
Утв.	Блинова Е.А.							

Точкой входа служит файл `src/server.js`: в нём создаётся HTTP-сервер, к нему подключается приложение `Express`, инициализируется `Socket.IO` в `src/websocket/socket.js`, при запуске выполняется синхронизация схемы базы и посев ролей. Общие параметры (порт, CORS, лимиты тела запроса, настройки `cookie-session`, каталог загрузок, почта) сосредоточены в каталоге `src/config`.

Каталог `src/modules` объединяет прикладную логику по областям. Типичный модуль содержит маршруты (файлы с суффиксом `.routes.js`), контроллер (`.controller.js`) и при необходимости сервис (`.service.js`). Модуль `auth` отвечает за регистрацию, вход, выход и верификацию email (отдельный JWT для ссылки в письме). Модуль `profile` за чтение и обновление профиля, аватар, поиск пользователей и формирование рабочего профиля. Модули `order` и `service` реализуют полный жизненный цикл заказов и услуг с фильтрацией и сортировкой. Модуль `application` обрабатывает отклики на заказы и услуги.

Отправка почты вынесена в `src/common/mail/mail.transporter.js`: на базе `nodemailer` создаётся транспорт с сервисом Gmail, учётные данные берутся из конфигурации. В текущей реализации транспорт используется для писем с подтверждением email; тот же механизм при необходимости может обслуживать восстановление пароля и прочие почтовые уведомления.

В каталоге `src/models` описаны сущности `User`, `Role`, `Profile`, `Order`, `Service`, `Application`, `Rating`, `Favorite`, `Resume`, `Chat`, `Message`, `Notification`, `Category`, `Skill`, `Contact`, а также `Attachment`, `Ticket`, `Dispute` и `Report`. Файл `src/models/index.js` связывает модели отношениями «один к одному», «один ко многим» и «многие ко многим» через промежуточные таблицы. Для части связей явно задано поведение `onDelete` со значением `CASCADE`; для остальных действует сочетание настроек `Sequelize` и правил СУБД по умолчанию.

Контроллеры и сервисы модулей `order` и `service` поддерживают создание, выборку списков и детальных записей, изменение и удаление заказов и услуг. Реализована расширенная фильтрация по категории с учётом подкатегорий, статусу, ценовому диапазону и текстовому поиску, а также сортировка результатов. Смена статусов и сопутствующая бизнес-логика сопровождается уведомлениями пользователей там, где это заложено в сервисном слое.

Модуль `chat` отвечает за создание чатов, отправку и получение сообщений, пагинацию выборок, отметку прочтений, а также обновление и удаление сообщений. В файле `src/websocket/socket.js` при подключении клиента выполняется проверка JWT из объекта `handshake.auth` или заголовка авторизации; пользователь подписывается на комнаты, привязанные к его идентификатору и к идентификатору чата, через которые в реальном времени доставляются события чата и связанные уведомления.

В каталоге `src/common` сосредоточены переиспользуемые части: `middleware` аутентификации и регистрации, обработчик ошибок `errorHandler`, обёртка `asyncHandler` для асинхронных обработчиков, а также `middleware` загрузки файлов, связанный с модулем вложений. Централизованная обработка ошибок завершает цепочку `middleware` в `app.js`.

Параметры подключения к базе задаются в `src/config/db.config.js` (при развёртывании может дублироваться файл в каталоге `config`). Секрет для

подписи JWT и прочие чувствительные значения, такие как переменная окружения SECRET, ожидаются из переменных окружения.

Структура каталога src логически делится на app.js и server.js как точки сборки приложения и сервера, каталог config с настройками, common с middleware и почтой, models с описанием сущностей и ассоциаций, представленные в приложении И, modules с доменной логикой по разделам API и подкаталог websocket с инициализацией Socket.IO поверх того же HTTP-сервера.

3.1.1 Аутентификация и авторизация пользователей

Для аутентификации и авторизации используется контроллер auth.controller.js на Express.js. Используется JSON Web Token (JWT) для stateless-аутентификации без хранения сессий на сервере. Токен хранится в cookie-session на клиенте. Регистрация выполняется через метод signup.

Код метода представлен в листинге 3.2. Метод signup принимает данные регистрации. Перед созданием пользователя выполняются проверки через middleware, сперва идет проверка уникальность username и email, checkRolesExisted валидирует указанные роли. При успешной валидации пароль хешируется через argon2, создается пользователь через Sequelize, и ему присваивается роль (по умолчанию «executer», id=1). Если роли указаны явно, они назначаются пользователю через связь many-to-many с таблицей ролей.

```
exports.signup = async (req, res) => {
  try {
    const user = await User.create({
      username: req.body.username,
      email: req.body.email,
      password: await argon2.hash(req.body.password),
    });

    if (req.body.roles && req.body.roles.length > 0) {
      const roles = await Role.findAll({
        where: {
          name: req.body.roles,
        },
      });
      await user.setRoles(roles);
    }
  } else {
    await user.setRoles([1]);
  }
  res.send({ message: "Успешная регистрация пользователя!" });
}
catch (error) {
  console.error(error);
  res.status(500).send({ message: error.message });
}
};
```

Листинг 3.1 – Реализация регистрации пользователей

Аутентификация выполняется методом `signin`. Он проверяет учетные данные, наличие блокировки и генерирует JWT-токен. Код метода аутентификации представлен в листинге 3.2.

Метод `signin` выполняет проверки безопасности. Сначала ищется пользователь по `username`. Если пользователь не найден, возвращается статус 404. Затем проверяется пароль через `argon2.verify`. При неверном пароле возвращается статус 401. После успешной проверки пароля генерируется JWT-токен через `jwt.sign` с `payload`, содержащим идентификатор пользователя (`id`), секретным ключом из переменных окружения, алгоритмом HS256 и сроком жизни 86400 секунд (24 часа). Затем загружаются роли пользователя и формируется массив `authorities` в формате «ROLE_ROLENAME». Токен сохраняется в `cookie-session` через `req.session.token` и возвращается клиенту в совокупности с информацией о пользователе (`id`, `username`, `email`, `roles`, `accessToken`).

```
const passwordIsValid = await argon2.verify(
  user.password,
  req.body.password,
);
const token = jwt.sign({ id: user.id },
  process.env.SECRET,
  {
    algorithm: 'HS256',
    allowInsecureKeySizes: true,
    expiresIn: 86400, // 24 часа
  });
const roles = await user.getRoles();
for (let i = 0; i < roles.length; i++) {
  authorities.push("ROLE_"+roles[i].name.toUpperCase());
}
```

Листинг 3.2 – Реализация метода для аутентификации пользователей

Выход из системы выполняется методом `signout`, который очищает сессию, устанавливая `req.session` в `null`, что удаляет токен из `cookie-session` и завершает сеанс текущего пользователя.

Архитектура аутентификации обеспечивает безопасность через хеширование паролей (`argon2`), использование JWT для `stateless`-аутентификации, проверку блокировок пользователей и валидацию ролей. `Cookie-session` обеспечивает удобное хранение токена на клиенте с автоматической отправкой при каждом запросе, а поддержка заголовка `Authorization` позволяет использовать токен в различных приложениях и других клиентах.

3.1.2 Реализация отправки сообщений

Для отправки сообщений используется контроллер `chat.controller.js` на `Express.js`. Система поддерживает обмен сообщениями между пользователями через REST API и доставку в реальном времени через `WebSocket` (`Socket.IO`).

Отправка сообщения выполняется методом `sendMessage`. Код метода представлен в листинге 3.3.

```
if (chat.userId !== currentUser.id && chat.userId !== current-
  User.id) {
  return res.status(403).send({ message: "Нет доступа к этому
  чату" });
}

const message = await Message.create({ ... });
await chat.update({ lastMessageAt: new Date() });
const notification = await createNotification({ ... });

if (global.io) {
  sendChatMessage(global.io, chatId, messageWithSender, recip-
  ientId);
  updateChatsList(global.io, recipientId);
  if (notification) {
    sendNotification(global.io, recipientId, notification);
  }
}
```

Листинг 3.3 – Реализация отправки сообщений

Метод `sendMessage` принимает идентификатор чата (`chatId`) и содержимое сообщения (`content`). Сначала выполняется валидация: проверяется, что содержимое сообщения не пустое и не состоит только из пробелов. Если валидация не пройдена, возвращается статус 400 с сообщением об ошибке. Затем выполняется проверка существования чата в базе данных через `Chat.findByPk`. Если чат не найден, возвращается статус 404. Далее выполняется проверка прав доступа: проверяется, что текущий пользователь является участником чата (либо `userId`, либо `chatId` совпадает с идентификатором текущего пользователя). Если пользователь не является участником чата, возвращается статус 403 с сообщением «Нет доступа к этому чату».

После успешной проверки прав создается сообщение в базе данных через `Message.create` с полями `chatId`, `senderId` (идентификатор текущего пользователя) и `content` (с удалением пробелов в начале и конце). Затем обновляется время последнего сообщения в чате через `chat.update({ lastMessageAt: new Date() })`, что используется для сортировки чатов по актуальности.

Определяется идентификатор получателя сообщения: если текущий пользователь является `userId`, то получатель – `chatId`, и наоборот. Создается уведомление для получателя через функцию `createNotification` с типом «new_message», заголовком «Новое сообщение» и текстом, содержащим имя отправителя и первые 50 символов сообщения.

3.1.3 Реализация создания заказа

Для создания заказа используется контроллер `order.controller.js` на `Express.js`. Система поддерживает создание заказов заказчиками с указанием

категории, бюджета, дедлайна и описания. Создание заказа выполняется методом `createOrder`. Код метода представлен в листинге 3.4.

```
exports.createOrder = async (req, res) => {
  try {
    const currentUser = req.user;
    const { title, description, budget, deadline, categoryId } =
req.body;
    const user = await User.findPk(currentUser.id, {
      include: [{ model: db.role, as: "roles", through: {
attributes: [] } }]
    });

    const userRoles = user.roles.map(r => r.name);
    if (!userRoles.includes("customer")) {
      return res.status(403).send({ message: "Только за-
казчики могут создавать заказы" });
    }

    const order = await Order.create({
      customerId: currentUser.id,
      title,
      description,
      budget: budget || null,
      deadline: deadline || null,
      categoryId
    });

    const orderWithRelations = await Order.findPk(or-
der.id, {
      include: [
        { model: Category, as: "category" },
        { model: User, as: "customer", attributes:
["id", "username", "email"] }
      ]
    });
  }
};
```

Листинг 3.4 – Реализация создания заказа

Метод `createOrder` принимает данные заказа: `title` (название), `description` (описание), `budget` (бюджет, опционально), `deadline` (срок выполнения, опционально) и `categoryId` (идентификатор категории). Сначала выполняется проверка роли пользователя: загружается текущий пользователь из базы данных вместе с его ролями через Sequelize `include` с использованием связи `many-to-many` через промежуточную таблицу. Роли извлекаются из результата и преобразуются в массив имен ролей. Проверяется наличие роли «customer» в списке ролей пользователя. Если роль «customer» отсутствует, возвращается статус 403 с сообщением «Только заказчики могут создавать заказы». Это обеспечивает, что только пользователи с ролью заказчика могут создавать заказы, что соответствует бизнес-логике системы.

После успешной проверки роли создается заказ в базе данных через `Order.create` с полями `customerId` (идентификатор текущего пользователя), `title`, `description`, `budget` (может быть `null`, если не указан), `deadline` (может быть `null`, если не указан) и `categoryId` (обязательное поле). Поле `status` автоматически устанавливается в значение «open» по умолчанию через определение модели `Order` с использованием `ENUM` типа.

После создания заказа он загружается из базы данных вместе со связанными данными через `Order.findByPk` с включением модели `Category` (для получения информации о категории) и модели `User` (для получения информации о заказчике с ограниченным набором атрибутов: `id`, `username`, `email`). Это обеспечивает возврат полной информации о заказе вместе с данными о категории и заказчике для отображения в интерфейсе.

В конце метод возвращает созданный заказ со статусом 201 (Created).

3.1.4 Реализация создания отклика на заказ или услугу

Для создания отклика используется контроллер `application.controller.js` на `Express.js`. Система поддерживает отклики на заказы и услуги с указанием сообщения и предложенной цены. Реализация создания отклика на заказ или услугу представлена в приложении К.

Создание отклика выполняется методом `createApplication`. Метод принимает данные: `orderId` или `serviceId` (обязательно один из них), `message` (опционально) и `proposedPrice` (опционально). Выполняется валидация: должен быть указан либо `orderId`, либо `serviceId`, но не оба одновременно. Если валидация не пройдена, возвращается статус 400.

Если указан `orderId`, проверяется существование заказа и то, что пользователь не откликается на свой заказ. Если указан `serviceId`, проверяется существование услуги и то, что пользователь не откликается на свою услугу. Затем проверяется, что данный пользователь еще не откликнулся на этот заказ или услугу через поиск существующего отклика.

После проверок создается отклик через `Application.create` с полями `userId`, `orderId` или `serviceId`, `message` и `proposedPrice`. Поле `status` автоматически устанавливается в «pending». Создается уведомление для владельца заказа или услуги через `createNotification` с типом «new_application». Затем метод возвращает созданный отклик со статусом 201.

Одобрение отклика выполняется методом `approveApplication`. Проверяется, что текущий пользователь является владельцем заказа или услуги. Статус отклика обновляется на «approved», создается уведомление для автора.

Отклонение отклика выполняется методом `rejectApplication` аналогично одобрению, но статус устанавливается в «rejected».

3.1.5 Реализация функций администратора

Система поддерживает блокировку и разблокировку пользователей, просмотр списка всех пользователей и отправку массовых уведомлений.

Блокировка пользователя выполняется методом `blockUser`. Метод принимает идентификатор пользователя в параметрах маршрута. Блокировка пользователя представлена в листинге 3.5.

```
exports.blockUser = async (req, res) => {
  try {
    const currentUser = req.user;
    const { userId } = req.params;

    const admin = await User.findByPk(currentUser.id, {
      include: [{ model: db.role, as: "roles", through: {
attributes: [] } }]]
    });
    const adminRoles = admin.roles.map(r => r.name);
    if (!adminRoles.includes("administrator")) {
      return res.status(403).send({ message: "Только адми-
нистраторы могут блокировать пользователей" });
    }

    const user = await User.findByPk(userId);
    if (!user) {
      return res.status(404).send({ message: "Пользователь
не найден" });
    }

    await user.update({ isBlocked: true });
    res.status(200).send({ message: "Пользователь заблокиро-
ван" });
  } catch (error) {
    console.error(error);
    res.status(500).send({ message: error.message });
  }
};
```

Листинг 3.5 – Реализация блокировки пользователя

Сначала выполняется проверка роли администратора: загружается текущий пользователь вместе с его ролями, проверяется наличие роли «administrator». Если роль отсутствует, возвращается статус 403. Затем проверяется существование пользователя для блокировки. Если пользователь не найден, возвращается статус 404. После проверок выполняется обновление поля `isBlocked` в значение `true` через `user.update`. Заблокированный пользователь не сможет войти в систему, так как middleware `getUserFromToken` проверяет это поле и возвращает ошибку при попытке аутентификации. Разблокировка пользователя выполняется методом `unblockUser` аналогично блокировке, но поле `isBlocked` устанавливается в значение `false`, что восстанавливает доступ пользователя к системе.

Отправка уведомлений выполняется методом `sendNotificationToAll` в контроллере уведомлений, реализация представлена в листинге 3.6.

```

exports.sendNotificationToAll = async (req, res) => {
  try {

    const currentUser = req.user;
    const { title, message } = req.body;
    const user = await User.findByIdPk(currentUser.id, {
      include: [{ model: db.role, as: "roles", through: {
attributes: [] } }]
    });
    const userRoles = user.roles.map(r => r.name);
    if (!userRoles.includes("administrator")) {
      return res.status(403).send({ message: "Только адми-
нистраторы могут отправлять уведомления всем пользователям" });
    } const allUsers = await User.findAll({ attributes:
['id'] });

    const notifications = [];
    for (const user of allUsers) {
      const notification = await Notification.create({
        userId: user.id,
        type: 'admin_notification',
        title: title || 'Уведомление от администратора',

        message: message, isRead: false});
      if (global.io) {
        const { sendNotification } = =
require("../../websocket/socket");
        sendNotification(global.io, user.id,
notification);
      } notifications.push(notification);
    }
    res.status(200).send({
      message: `Уведомление отправлено ${notifications.length}
пользователям`,
      count: notifications.length
    });
  }
}

```

Листинг 3.6 – Реализация отправки массовых уведомлений

Маршрутизация административных функций настроена в файле `admin.routes`. Все маршруты защищены middleware `getUserFromToken`, который определяет пользователя по токenu для проверки аутентификации.

3.2 Разработка клиентской части

Клиентская часть реализована как одностраничное приложение (SPA) на React 19.2.0 с TypeScript 5.9.3. Используется Vite 7.2.2 как инструмент сборки, React Router DOM 7.9.5 для маршрутизации, Axios 1.7.9 для HTTP-запросов, Socket.IO Client 4.8.1 для WebSocket, Tailwind CSS 4.1.17 для стилизации.

Клиентская часть приложения организована по модульному принципу: каждый функциональный модуль включает собственные компоненты и

сервисы. Благодаря этому достигаются удобство поддержки, расширяемость и разделение ответственности. Главный файл приложения App.tsx инициализирует React Router с использованием HashRouter, настраивает публичные и защищённые маршруты, а также запускает WebSocket для авторизованных пользователей. Кроме того, здесь реализована проверка блокировки аккаунта, при необходимости отображающая отдельный экран блокировки.

Структура клиентской части представлена на рисунке 3.2.

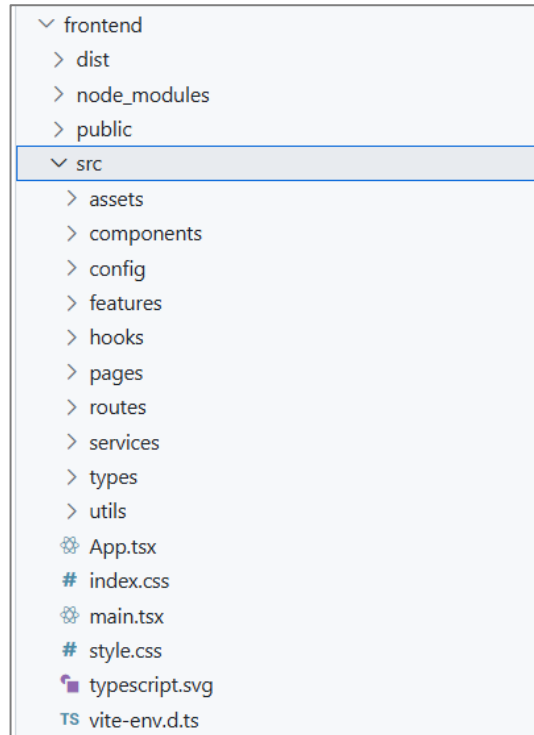


Рисунок 3.2 – Структура клиентской части

Фронтенд собран на React с TypeScript, сборщик Vite, маршрутизация React Router. Корень интерфейса – App.tsx и дерево маршрутов в routes/index.tsx: отдельные страницы лежат в pages (тонкие обёртки под маршрут), а основная разметка и логика экранов сосредоточены в features по предметным областям — авторизация и навигация, главная, каталог, заказы и услуги, отклики, профиль и рабочий профиль, избранное, рейтинги, резюме, чаты, уведомления, блоки администратора, менеджера и модерации. Общие переиспользуемые элементы интерфейса вынесены в components/ui, доступ к закрытым разделам оформлен через ProtectedRoute и сопутствующие компоненты в каталоге features/auth.

Обращение к API вынесено в слой services (отдельные модули под auth, заказы, услуги, категории, чаты, уведомления и т.д.), рядом лежат типы в types, вспомогательные функции в utils (запросы к API, заголовки авторизации, отображение). Для отдельных сценариев используются хуки, например домашняя лента в hooks, а для событий в реальном времени websocket.service совместно с экранами чатов и уведомлений; стили задаются через index.css и типовые классы в духе утилитарного CSS в компонентах.

3.2.1 Система маршрутизации клиентской части

Маршрутизация реализована средствами React Router DOM на базе HashRouter. Публичные маршруты включают страницы логина, регистрации, верификации, каталог, страницы просмотра заказов, услуг и профилей пользователей. Защищённые маршруты доступны только авторизованным пользователям и включают главную страницу, редактирование профиля, создание и редактирование заказов и услуг, работу с избранным, резюме, админ-панель, чаты, уведомления и личные элементы. Реализация системы маршрутизации представлена в приложении Л.

Компонент ProtectedRoute контролирует доступ к защищённым страницам. Он использует AuthService.getCurrentUser() для проверки авторизации. Если пользователь не авторизован, отображается компонент AuthRequired с предложением войти. Параметр allowPublic позволяет открывать некоторые страницы без авторизации.

3.2.2 Сервисы для взаимодействия с API

Сервисы взаимодействуют с сервером через Axios. AuthService реализует регистрацию, вход, выход, получение текущего пользователя и отправку верификационных данных. JWT-токен сохраняется в localStorage после успешной авторизации и используется для всех защищённых запросов. Реализация сервиса администратора представлена в приложении М.

Остальные сервисы, такие как OrderService, ServiceService, ApplicationService, ProfileService, ChatService, NotificationService, CategoryService, FavoriteService, RatingService, ResumeService, AdminService и StatsService обеспечивают полный набор REST-операций по CRUD-логике для соответствующих сущностей.

3.2.4 WebSocket-коммуникация и управление состоянием

Класс WebSocketService отвечает за работу с Socket.IO. Метод connect() выполняет подключение к серверу, передаёт JWT-токен, включает механизм автоматического переподключения и регистрирует события.

Сервис предоставляет методы подписки и отписки от основных событий: новых сообщений, обновления списка чатов, уведомлений и изменений уведомлений. Также реализованы универсальные методы on и off для обработки любых событий. WebSocket соединение создаётся в App.tsx при загрузке приложения и корректно закрывается при размонтировании. Реализация представлена в приложении Н.

Управление состоянием реализовано через React Hooks: useState и useEffect. Информация о пользователе хранится в localStorage и предоставляется AuthService. События WebSocket обновляют состояние интерфейса в реальном времени, например, при получении новых сообщений или уведомлений. Такой подход обеспечивает динамичность и отзывчивость интерфейса.

3.2.6 Стилизация интерфейса

В качестве основной системы стилизации используется Tailwind CSS 4.1.17, конфиг-файл которого представлен в листинге 3.7. Применяются утилитарные классы, что ускоряет разработку и обеспечивает гибкую верстку.

```
/** @type {import('tailwindcss').Config} */
export default {
  content: ['./index.html', './src/**/*.{js,ts,jsx,tsx}'],

  theme: {
    extend: {
      colors: {
        body: '#000000',
        accent: '#ff555b',
        secondary: '#cc0088',
        white: '#ffffff',
        placeholder: '#888888',
        error: '#ff4d4d',
        success: '#ffffff',
      },

      fontFamily: {
        sans: ['Arial', 'Helvetica', 'sans-serif'],
      },

      boxShadow: {
        auth: '0 8px 25px rgba(255,255,255,0.3)',
      },

      borderColor: {
        body: '#000000',
      },
    },
  },
  plugins: [
  ],
}
```

Листинг 3.7 – Базовые стили

Проект использует дополнительные кастомные настройки, а также файлы App.css, index.css и style.css для специфичных стилей.

3.2.7 Обработка блокировки пользователей

В App.tsx реализована проверка статуса пользователя при запуске приложения. Если сервер возвращает ошибку 403 при запросе к ProfileService.getProfile(), это означает блокировку аккаунта. В таком случае

отображается специальный экран блокировки с объяснением и кнопкой выхода из системы. Это позволяет уведомлять пользователя о статусе аккаунта.

3.3 Сборка проекта в Docker-контейнер

Проект контейнеризирован с использованием Docker и Docker Compose для изоляции компонентов и упрощения развертывания. Архитектура состоит из трех контейнеров: база данных MySQL, бэкенд-сервер на Node.js и фронтенд-приложение на React. Docker-файлы представлены в приложении П.

Для бэкенд-сервера создан Dockerfile на базе образа `node:20-alpine`. В контейнере устанавливаются зависимости из `package.json`, копируется код приложения и конфигурационные файлы. Контейнер экспонирует порт 8080 и запускает сервер командой `node server.js`. Используются `volumes` для синхронизации кода с хост-системой, что обеспечивает `hot reload` при разработке.

Фронтенд-приложение также контейнеризировано на базе `node:20-alpine`. В контейнере устанавливаются зависимости и запускается `dev`-сервер Vite с флагом `host` для доступа извне контейнера. Контейнер работает на порту 3000 и использует `volumes data` для синхронизации исходного кода, что позволяет видеть изменения без пересборки образа.

База данных MySQL развернута в отдельном контейнере на основе официального образа `mysql:8.0`. Для совместимости используется плагин аутентификации `mysql_native_password`. Данные БД сохраняются в именном `volume mysql_data`, что обеспечивает персистентность данных между перезапусками контейнеров.

Оркестрация контейнеров выполняется через Docker Compose с файлом `docker-compose.yml`. Определена единая сеть `freelance_network` для взаимодействия сервисов. Бэкенд зависит от БД через `depends_on` с проверкой готовности через `healthcheck`. Переменные окружения настраиваются через `.env`-файл или используются значения по умолчанию.

Сборка и запуск выполняются командой `docker-compose up -d`, которая создает образы, запускает контейнеры в фоновом режиме и настраивает сеть. Для остановки используется `docker-compose down`, при необходимости с флагом `-v` для удаления `volumes`. Такая конфигурация обеспечивает изоляцию окружений, воспроизводимость развертывания и упрощает работу в команде.

3.4 Выводы по разделу

Архитектура серверной части базируется на принципах микросервисного разделения логики. Использование Node.js совместно с фреймворком NestJS обеспечивает высокую скорость обработки асинхронных запросов. Система аутентификации строится на многоуровневой проверке подлинности, где JWT токены обеспечивают передачу данных без состояния. Ассоциации в ORM Sequelize минимизируют риск возникновения аномалий при работе с реляционными данными. Интеграция Socket.IO создает канал связи для

событийного управления уведомлениями. Это позволяет серверу мгновенно оповещать исполнителей о новых заказах без опроса базы данных.

Клиентская часть приложения спроектирована с использованием компонентного подхода в библиотеке React. Применение TypeScript гарантирует типизацию передаваемых объектов и снижает количество ошибок на этапе компиляции. Использование Tailwind CSS упрощает стилизацию элементов и обеспечивает единообразие визуальных компонентов. Разделение бизнес логики и представления данных через отдельные сервисы повышает чистоту кода. Каждый модуль клиента изолирован, что облегчает проведение модульного тестирования. Маршрутизация на основе React Router позволяет динамически переключать виды приложения без полной перезагрузки страницы.

Механизм взаимодействия между клиентом и сервером унифицирован через единый API. Защищенные маршруты на фронтенде блокируют неавторизованные переходы, обращаясь к состоянию сессии пользователя. При интеграции с WebSocket фронтенд подписывается на специфические события, поступающие от сервера в реальном времени. Это создает бесшовную среду для коммуникации заказчиков и студентов. Использование хуков React для управления состоянием обеспечивает мгновенное обновление интерфейса после успешных ответов от сервера. Такая архитектура повышает отзывчивость системы при интенсивном пользовательском потоке.

Безопасность системы подкрепляется строгой политикой обработки пользовательских данных. Серверный слой реализует фильтрацию входящих запросов и валидацию передаваемых параметров на соответствие заданным схемам. Все пароли проходят через хеширование с использованием криптографически стойких алгоритмов. Сессионные токены хранятся с учетом требований к защите от перехвата и подмены данных.

Масштабируемость архитектуры закладывает основу для развития функциональных модулей в будущем. Модульная структура позволяет добавлять новые бизнес-процессы без глубокой модификации текущего программного кода. Система легко адаптируется под изменение требований заказчика за счет слабой связанности компонентов. Использование контейнеризации в перспективе упростит развертывание проекта на различных средах исполнения. Применение стандарта ES6 и актуальных версий библиотек обеспечивает высокую производительность и поддержку современных стандартов безопасности веб приложений.

4 Тестирование веб-приложения

Тестирование веб-приложения направлено на проверку корректности работы реализованного функционала, выявление ошибок и оценку стабильности системы при различных сценариях взаимодействия пользователей. В рамках данного раздела рассматриваются методы и подходы, использованные при проверке серверной и клиентской частей приложения, а также результаты тестирования ключевых модулей: аутентификации, профилей, заказов, услуг, чатов, уведомлений и административной панели. Особое внимание уделяется проверке надежности API, корректности обработки данных, устойчивости интерфейса и работоспособности механизмов обмена сообщениями в режиме реального времени. Проведённое тестирование позволяет убедиться, что разработанное приложение соответствует функциональным требованиям и обеспечивает необходимый уровень качества и безопасности.

4.1 Тестирование модуля регистрации и аутентификации

Первоначально была проверена клиентская валидация данных в форме регистрации. Основная цель – убедиться, что система корректно реагирует на ошибки ввода и предоставляет пользователю исчерпывающие подсказки.

Была проведена проверка валидации данных в форме регистрации. Цель заключалась в том, чтобы убедиться, что интерфейс корректно реагирует на ошибки ввода и информирует пользователя о нарушениях правил. Одним из сценариев стало тестирование обработки слабого пароля. На рисунке 4.1 показано, что при вводе пароля, не соответствующего требованиям, отображается предупреждение, не позволяя продолжить регистрацию до исправления.

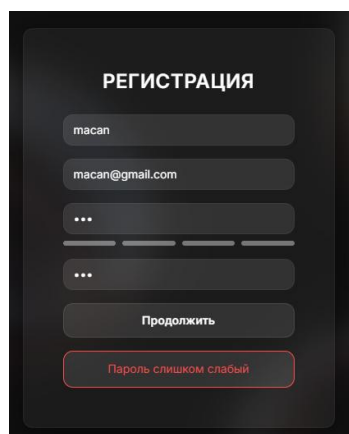
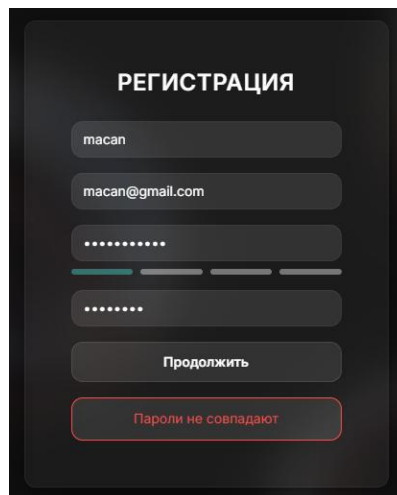


Рисунок 4.1 – Обработка ошибки со слабым паролем

				ДП 04.00.ПЗ						
	ФИО	Подпись	Дата							
Разраб.	Глухова Д.В.			4 Тестирование веб-приложения	Лит.		Лист		Листов	
Пров.	Уласевич Н.И.					У		1	14	
					БГТУ 1-40 05 01, 2026					
Н. контр.	Николайчук А.Н.									
Утв.	Блинова Е.А.									

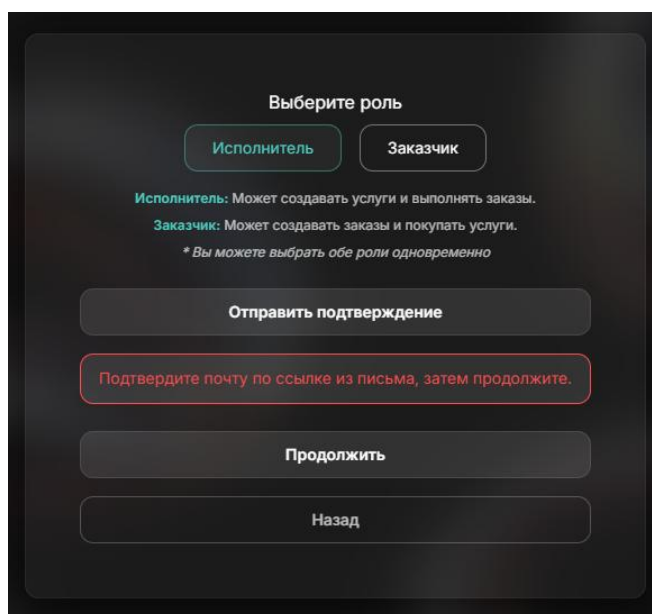
Следующим тестируемым сценарием стало несовпадение паролей в полях «Пароль» и «Подтверждение пароля». На рисунке 4.2 видно, что приложение корректно реагирует на данную ошибку и выводит сообщение о том, что введенные значения не совпадают. Это предотвращает создание аккаунта с непреднамеренно введенным паролем и повышает общую надёжность системы.



The screenshot shows a registration form titled "РЕГИСТРАЦИЯ" on a dark background. It contains input fields for "macan", "macan@gmail.com", and two password fields. The second password field has a red border and a red error message "Пароли не совпадают" (Passwords do not match) below it. A "Продолжить" (Continue) button is visible below the password fields.

Рисунок 4.2 – Обработка несовпадения паролей

Важной частью тестирования стала проверка процесса подтверждения электронной почты. При попытке продолжить регистрацию без прохождения этапа email-верификации система корректно блокирует дальнейшие действия и отображает соответствующее уведомление, что показано на рисунке 4.3. Это обеспечивает обязательность подтверждения адреса электронной почты, что повышает безопасность и защищённость пользовательских данных.



The screenshot shows a registration form titled "Выберите роль" (Choose a role) on a dark background. It has two buttons: "Исполнитель" (Executor) and "Заказчик" (Client). Below them, there is text explaining the roles: "Исполнитель: Может создавать услуги и выполнять заказы." and "Заказчик: Может создавать заказы и покупать услуги." followed by a note: "* Вы можете выбрать обе роли одновременно". Below this, there is a button "Отправить подтверждение" (Send confirmation) and a red-bordered box with the text "Подтвердите почту по ссылке из письма, затем продолжите." (Confirm email via link from email, then continue). At the bottom, there are buttons "Продолжить" (Continue) and "Назад" (Back).

Рисунок 4.3 – Попытка продолжения регистрации без подтверждения почты

На рисунке 4.4 представлен сценарий попытки регистрации пользователя, email которого уже существует в базе данных. Система корректно

определяет конфликт уникальности и информирует пользователя, что учетная запись с указанным адресом уже зарегистрирована. Таким образом, предотвращается создание дубликатов и обеспечивается целостность данных.

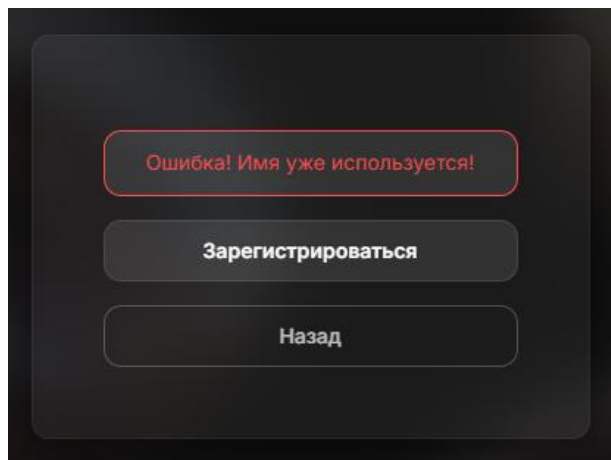


Рисунок 4.4 – Попытки регистрации существующего пользователя

Процесс авторизации также был протестирован на корректную обработку ошибок. На рисунке 4.5 отражена ситуация, когда пользователь вводит неверный пароль при попытке входа. В этом случае сервер возвращает сообщение об ошибке, а клиентское приложение корректно отображает уведомление, информируя пользователя о некорректных данных, что крайне важно для предотвращения возможного несанкционированного доступа.

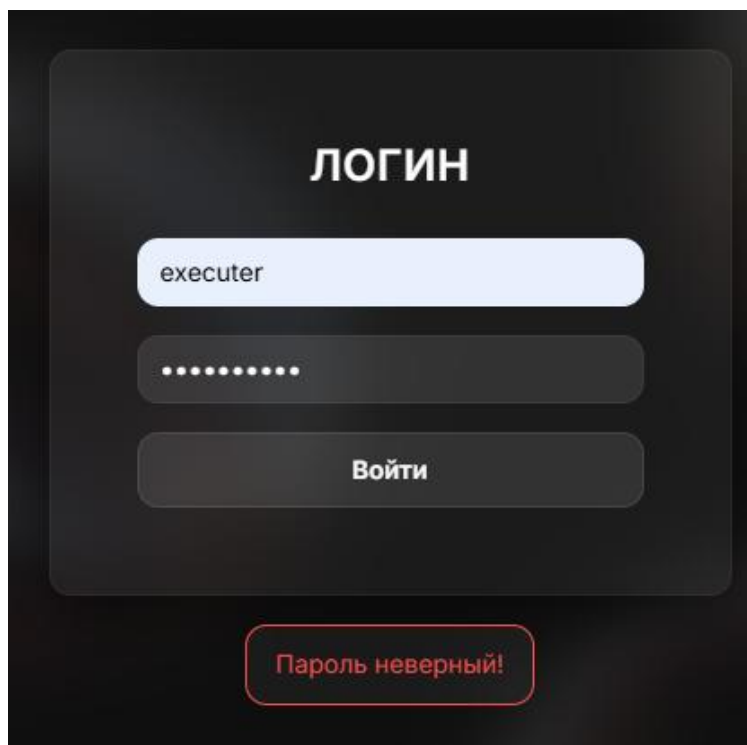


Рисунок 4.5 – Попытка входа под неправильным паролем

В завершение был протестирован сценарий входа под несуществующим пользователем. Как показано на рисунке 4.6, при вводе email, который

отсутствует в системе, приложение возвращает стандартное сообщение об ошибке авторизации. Подобный подход не раскрывает сведения о существовании или отсутствии конкретного пользователя, что является хорошей практикой с точки зрения безопасности.

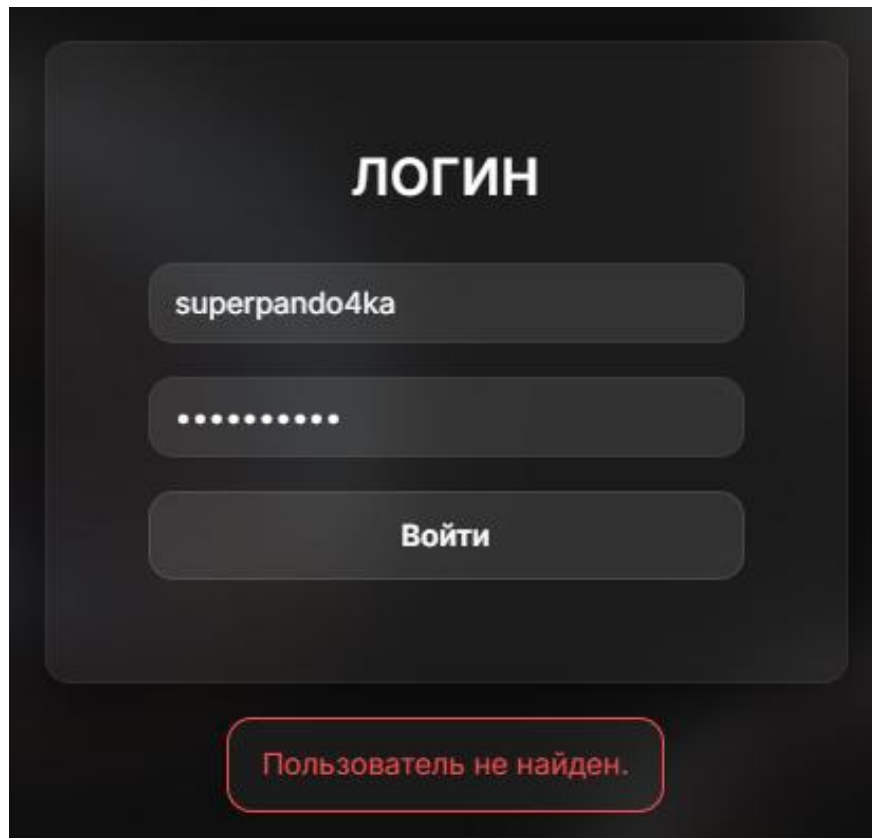


Рисунок 4.6 – Попытка входа под несуществующим пользователем

В результате проведенного тестирования модуля регистрации и аутентификации было подтверждено, что система корректно обрабатывает все основные и пограничные сценарии, связанные с созданием и доступом к учетной записи пользователя. Клиентская и серверная валидация работают согласованно, своевременно информируя пользователя о допущенных ошибках и предотвращая некорректный ввод данных. Механизм подтверждения электронной почты функционирует надёжно, обеспечивая дополнительный уровень безопасности. Процедура авторизации устойчиво реагирует на попытки входа с неверными или несуществующими учетными данными. Модуль регистрации и аутентификации соответствует предъявляемым требованиям и обеспечивает необходимый уровень надежности и безопасности.

4.2 Тестирование создания заказов и услуг

Процесс создания заказов и услуг является одним из ключевых элементов функциональности веб-приложения, поскольку он обеспечивает взаимодействие между пользователями платформы – заказчиками и исполнителями. В рамках тестирования была проведена серия проверок, направленных на

оценку корректности работы форм ввода, механизмов валидации и обработки ошибок как на стороне клиента, так и на стороне сервера.

Первоначально была протестирована клиентская валидация формы создания заказа. Целью являлась проверка того, что система не позволяет отправить форму, если обязательные поля не заполнены, а также корректно отображает сообщения об ошибках. Как показано на рисунке 4.7, при попытке сохранения заказа без указания названия, описания или других ключевых параметров, приложение предупредило пользователя о необходимости заполнить обязательные поля. Наличие всплывающей подсказки подтверждает, что валидация работает корректно и предотвращает отправку некорректных данных.

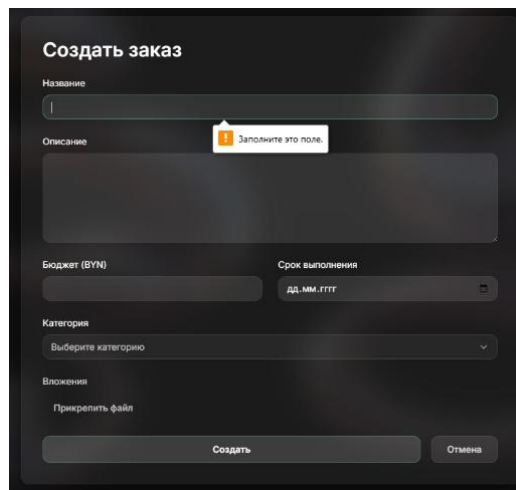


Рисунок 4.7 – Попытка создания заказа без заполнения обязательных полей

Особое внимание также уделялось процессу создания услуг. Форма создания услуги включает выбор категории, которая является обязательным параметром и определяет принадлежность услуги к определённому типу деятельности. Как видно на рисунке 4.8, при попытке сохранить услугу без выбранной категории система корректно вывела сообщение об ошибке и заблокировала дальнейшее действие.

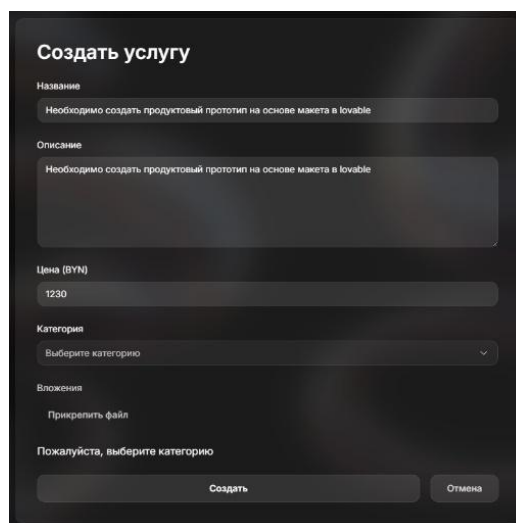


Рисунок 4.8 – Попытка создания услуги без выбора категории

Протестирована корректная обработка опциональных полей, таких как бюджет или срок выполнения заказа, а также цена для услуги. Система не дает пользователю ввести буквы алфавита в поля, где предполагаются цифры.

Проведённые проверки подтверждают, что формы создания заказов и услуг оснащены необходимыми механизмами защиты от неверного ввода, а ошибки отображаются пользователю в понятном и информативном виде. Благодаря этому система обеспечивает удобство взаимодействия, снижает вероятность допущения ошибок и поддерживает высокое качество данных в базе.

4.3 Тестирование системы рейтингов

Проверка системы рейтингов началась с валидации вводимых оценок. Было подтверждено, что приложение корректно блокирует неверные значения, позволяя пользователю выбирать только цифры в диапазоне от 1 до 5. На рисунке 4.9 представлен диапазон значений для выбора оценки.

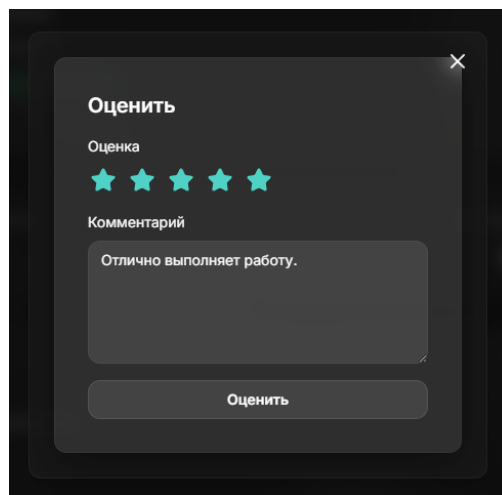


Рисунок 4.9 – Диапазон оценки

Далее тестировалось отображение добавленных оценок в профиле пользователя. При добавлении нового рейтинга система автоматически обновляет и показывает среднее значение, что позволяет другим пользователям видеть актуальный уровень оценок. Это подтверждает корректность работы механизма агрегирования рейтингов. Результат представлен на рисунке 4.10.

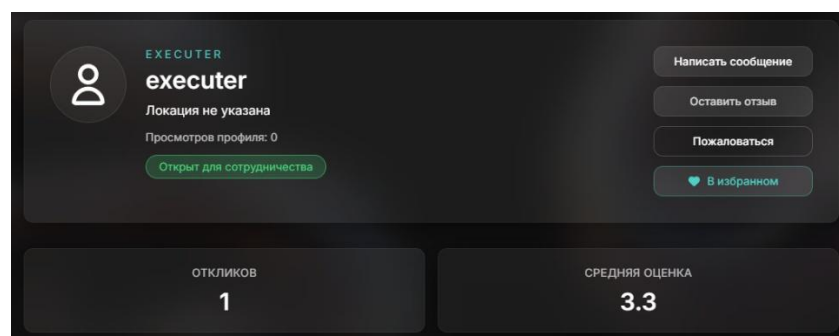


Рисунок 4.10 – Корректное отображение средней оценки пользователя

Также была проверена визуализация отзывов. Система корректно отображает комментарии, связанные с оценками, включая дату и автора, что обеспечивает прозрачность и удобство восприятия информации о пользователе.

Результат отображения отзывов представлен на рисунке 4.11.

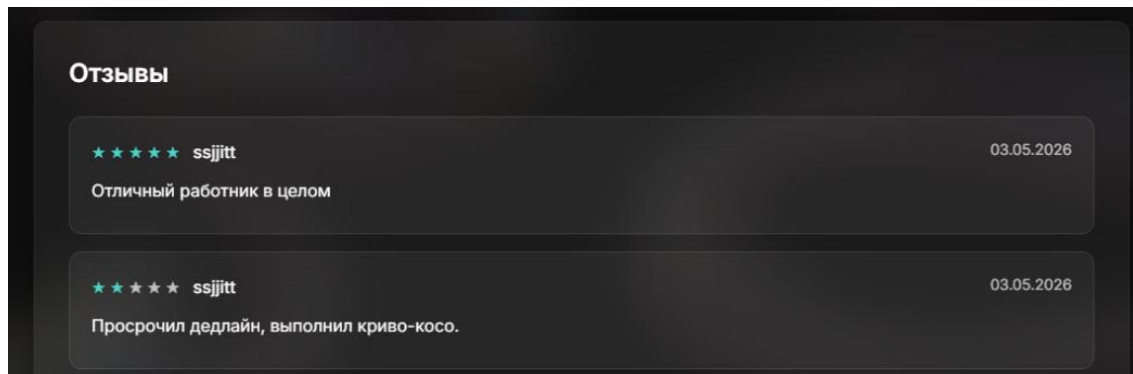


Рисунок 4.11 – Корректное отображение отзывов

В результате проведенного тестирования можно заключить, что модуль рейтингов работает стабильно: оценки валидируются, средние значения пересчитываются корректно, а отзывы отображаются корректно и удобно.

4.4 Тестирование системы откликов

Проверка полей для ввода числовых значений показала, что система корректно ограничивает ввод. При попытке ввести буквы в поле цены система блокирует такой ввод, позволяя пользователю вводить только допустимые числовые значения. Попытка ввода представлена на рисунке 4.12.

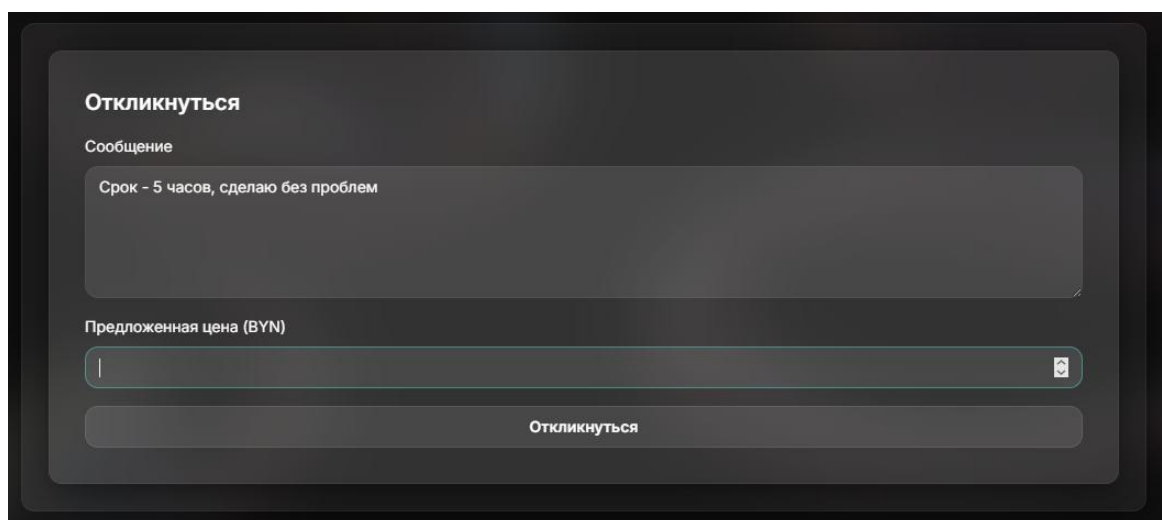


Рисунок 4.12 – Попытка ввода букв вместо цифр в поле цены

Дополнительно была протестирована функциональность откликов на заказы и услуги. В случае попытки отмены отклика пользователю отображается всплывающее окно с подтверждением и отменой действия.

В процессе тестирования было подтверждено, что система корректно обрабатывает оба сценария: ввод числовых данных ограничен, а действия пользователя, которые требующие подтверждения, сопровождаются всплывающими модальными окнами, которое представлено на рисунке 4.13.

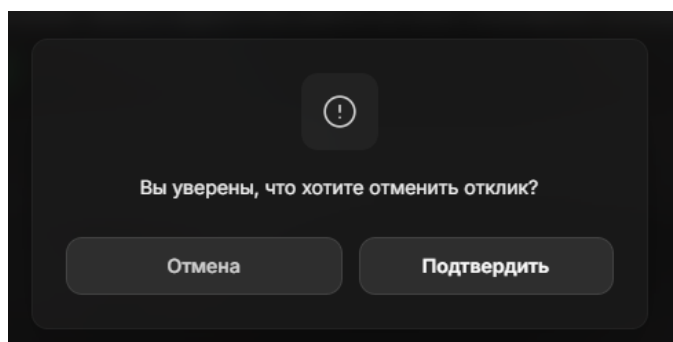


Рисунок 4.13 – Попытка отмены отклика с окном подтверждения

В результате проверок можно сделать вывод, что интерфейс корректно контролирует ввод данных и защищает пользователя от непреднамеренных действий, обеспечивая надежность и удобство работы с формами и откликами.

4.5 Тестирование системы избранных

Проверка функциональности системы добавления в избранное началась с тестирования добавления элементов. Пользователь может отметить другого пользователя как избранное, после чего элемент появляется в отдельном списке избранного. Отображение профиля пользователя до добавления в избранное представлено на рисунке 4.14.

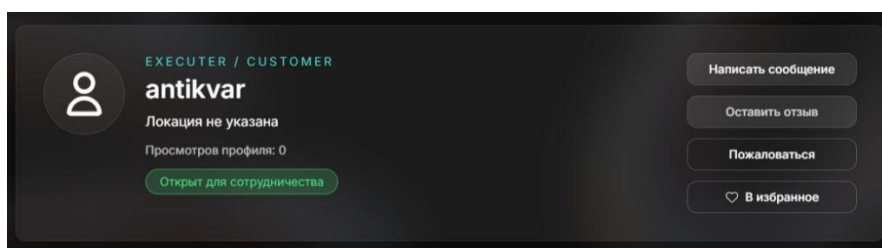


Рисунок 4.14 – Исходное состояние до добавления в избранное

Интерфейс корректно отображает подтверждение действия: изменение иконки и подсветку элемента, что позволяет пользователю видеть, что выбранный объект успешно добавлен, что представлено на рисунке 4.15.

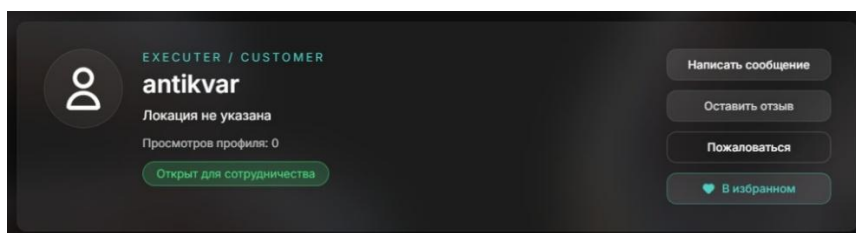


Рисунок 4.15 – Изменение иконки при добавлении в избранное

Следующий этап включал тестирование удаления элементов из избранного. При выборе функции удаления необходимо подтвердить действие, что показано на рисунке 4.16, элемент исчезает из списка, а интерфейс обновляется мгновенно, отображая актуальное состояние. Это позволяет пользователю легко управлять своей коллекцией избранных заказов и услуг, исключая возможность путаницы или сохранения устаревших данных.

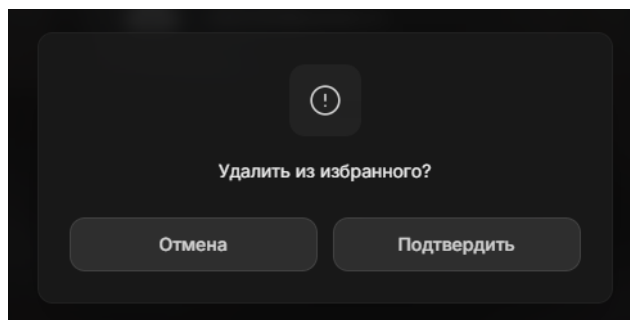


Рисунок 4.16 – Подтверждение удаления из избранного

Отображение актуального состояния после удаления из избранного представлено на рисунке 4.17.

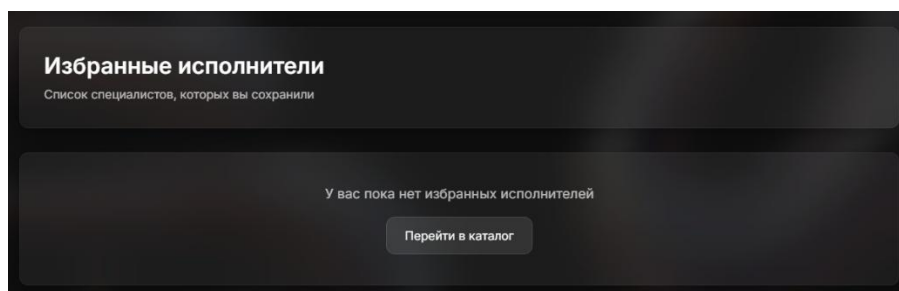


Рисунок 4.17 – Отображение актуального состояния после удаления

Было также проверено добавление и удаление нескольких элементов одновременно. Система корректно обрабатывает множественные операции, обеспечивая стабильное отображение всех элементов в списке и предотвращая дублирование или потерю данных при массовых действиях.

В результате проведенного тестирования можно заключить, что функциональность «Избранного» работает корректно: пользователи имеют возможность добавлять, удалять и просматривать выбранные элементы без ошибок, а интерфейс обеспечивает наглядное подтверждение всех действий, что повышает удобство взаимодействия с приложением.

4.6 Тестирование каталога

Тестирование каталога с фильтрацией и сортировкой началось с проверки корректного отображения всех доступных заказов и услуг при загрузке страницы. Пользователю предоставляется полный список элементов, каждый из которых отображается с основными данными: названием, категорией, ценой или бюджетом, а также кратким описанием. Это позволяет сразу

ознакомиться с доступными предложениями без необходимости дополнительной настройки фильтров. Каталог представлен на рисунке 4.18.

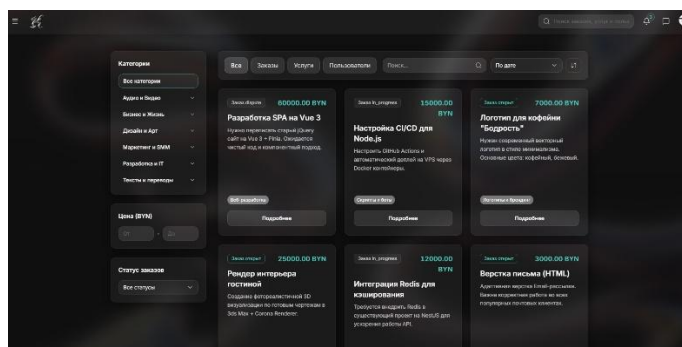


Рисунок 4.18 – Каталог

Далее была проведена проверка работы фильтров по категориям. При выборе одной или нескольких категорий каталог корректно обновляется, показывая только элементы, соответствующие выбранным критериям. Интерфейс мгновенно реагирует на изменения, обеспечивая пользователю быстрый доступ к релевантным данным. Результат показан на рисунке 4.19.

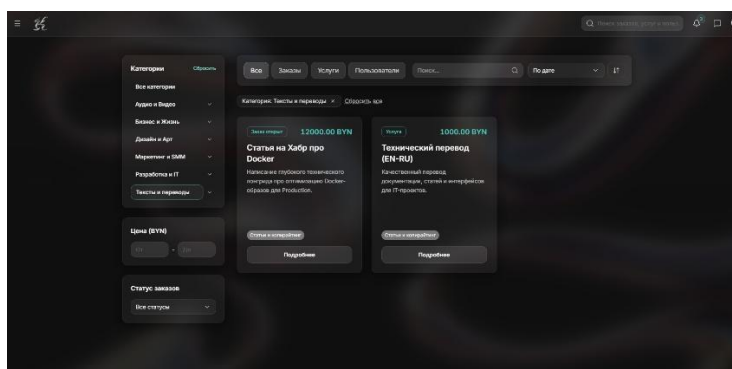


Рисунок 4.19 – Фильтрация по категории

Также была протестирована комбинированная фильтрация, включающая категорию, диапазон цен или бюджета и другие параметры. Система правильно обрабатывает все условия одновременно, исключая элементы, не соответствующие выбранным фильтрам. Пример представлен на рисунке 4.20

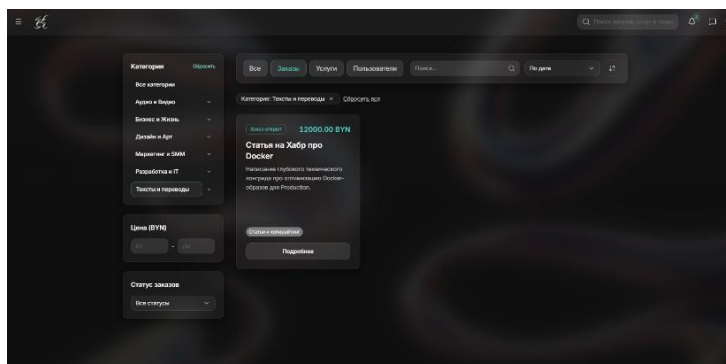


Рисунок 4.20 – Комбинированная фильтрация

В результате тестирования можно сделать вывод, что функциональность каталога с фильтрацией работает надежно: пользователи могут эффективно искать и отслеживать интересные их заказы и услуги, а интерфейс обеспечивает удобное взаимодействие и наглядное отображение результатов.

4.7 Тестирование функционала администратора

Тестирование функционала администратора началось с проверки возможности отправки массовых уведомлений пользователям. Администратор вводит текст уведомления, выбирает получателей и отправляет сообщение. Ввод данных для отправки уведомлений представлен на рисунке 4.21.

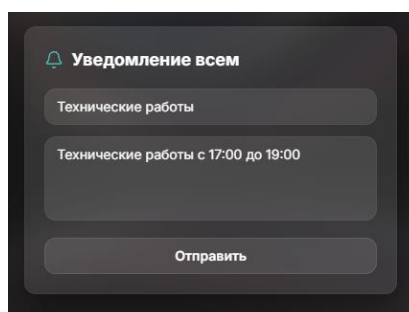
The screenshot shows a dark-themed modal window titled "Уведомление всем" (Notify all) with a bell icon. It contains two text input fields: the first is labeled "Технические работы" (Technical work) and the second is labeled "Технические работы с 17:00 до 19:00" (Technical work from 17:00 to 19:00). At the bottom is a button labeled "Отправить" (Send).

Рисунок 4.21 – Заполнение текста уведомлений

Система корректно обрабатывает данные и отображает уведомления в общем списке, что подтверждает успешную доставку и видимость сообщений для целевой аудитории. Доставка уведомлений представлена на рисунке 4.22.

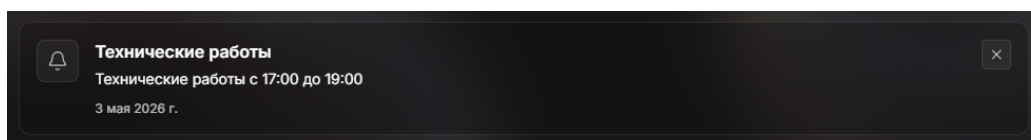
The screenshot shows a notification card in a list. It has a bell icon, the title "Технические работы" (Technical work), the description "Технические работы с 17:00 до 19:00" (Technical work from 17:00 to 19:00), and the date "3 мая 2026 г." (May 3, 2026). There is a close button (X) in the top right corner.

Рисунок 4.22 – Корректное отображения оповещения в списке

Следующий этап тестирования включал управление категориями. Администратор создаёт новую категорию, и система проверяет корректность её добавления в базу данных. Создание категории представлено на рисунке 4.23.

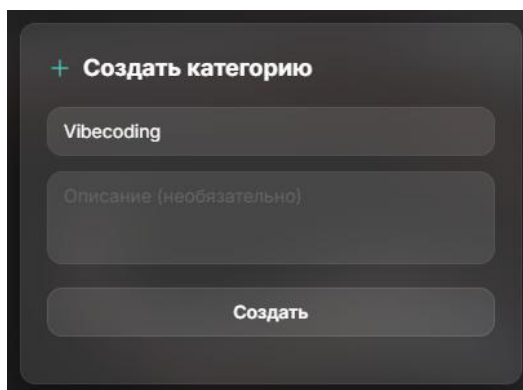
The screenshot shows a dark-themed modal window titled "+ Создать категорию" (Create category). It contains two text input fields: the first is labeled "Vibecoding" and the second is labeled "Описание (необязательно)" (Description (optional)). At the bottom is a button labeled "Создать" (Create).

Рисунок 4.23 – Создание категории

После создания категория становится доступной в соответствующих списках на фронтенде, что позволяет пользователям выбирать созданную категорию при создании своих заказов или услуг.

Доступность созданной категории представлена на рисунке 4.24.

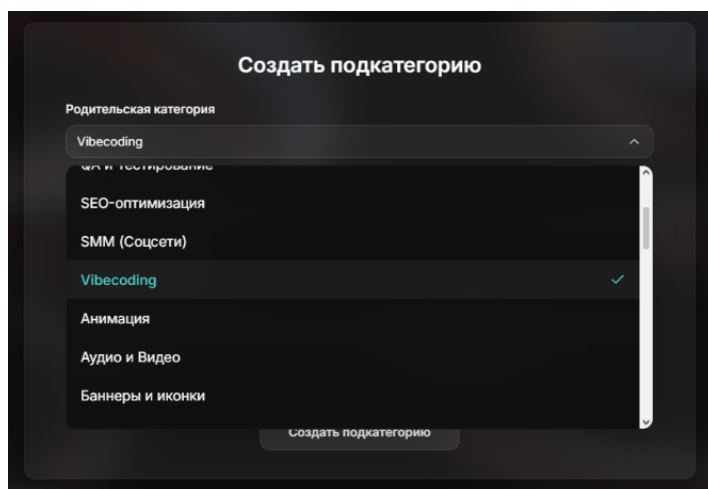


Рисунок 4.24 – Проверка доступности созданной категории

Далее была проверена работа статистики за месяц. После выполнения операций администратора, данные отображаются в календаре, представленном на рисунке 4.25 с правильными значениями, что позволяет оперативно отслеживать активность пользователей и результаты взаимодействий. Статистика обновляется в реальном времени и корректно отражает все события.

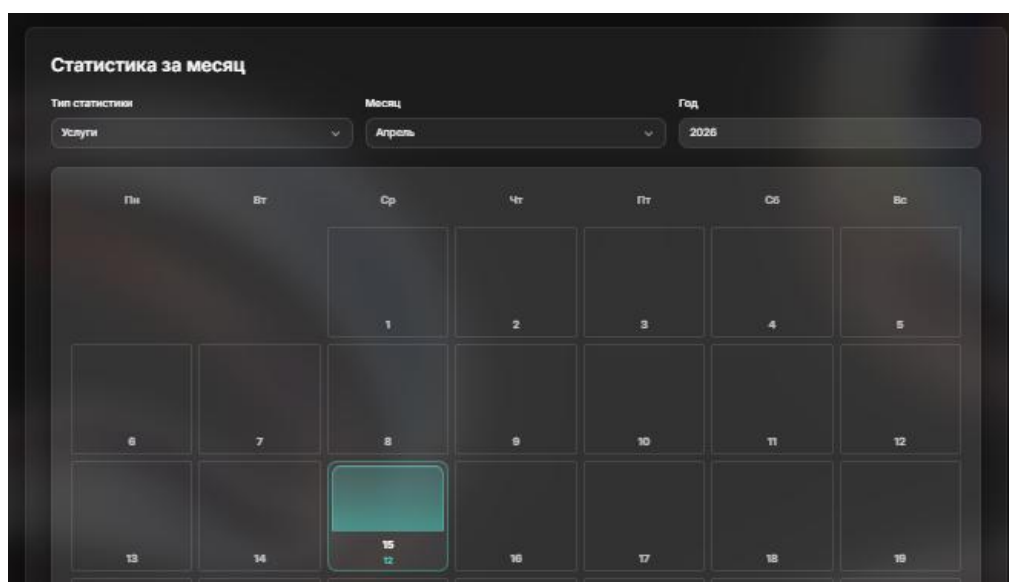


Рисунок 4.25 – Корректное отображение статистики за месяц в календаре

Тестирование механизма блокировки и разблокировки пользователей началось с проверки функционала блокировки аккаунта администратором. После выбора пользователя и активации опции «Блокировать», система корректно изменяет статус пользователя в базе данных, что предотвращает его дальнейший вход в приложение. Результат представлена на рисунке 4.26.

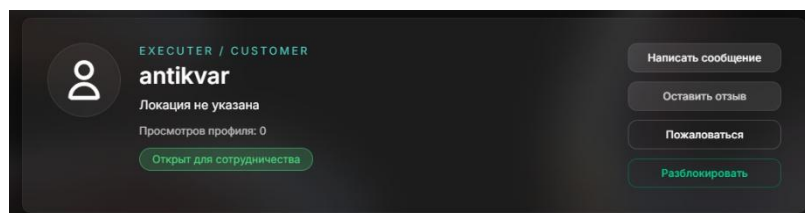


Рисунок 4.26 – Блокировка пользователя

Следующим шагом стало тестирование разблокировки пользователя. Администратор снимает блокировку, и система корректно обновляет статус в базе данных. После этого пользователь может успешно войти в систему, получить доступ к защищённым маршрутам и продолжить работу с приложением без ограничений. Результат представлен на рисунке 4.27.

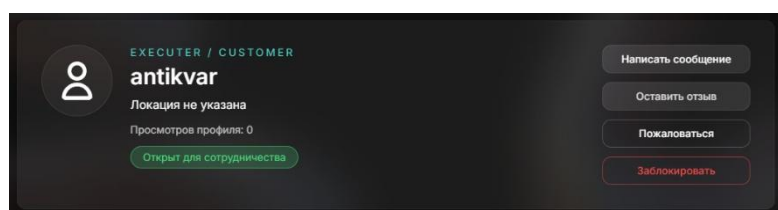


Рисунок 4.27 – Разблокировка пользователя

Далее была проверена реакция интерфейса пользователя на блокировку. Попытка доступа к защищённым разделам приложения заблокированным пользователем приводит к перенаправлению на страницу с уведомлением о блокировке. Это гарантирует, что заблокированный пользователь не имеет возможности обойти ограничения и получить доступ к ресурсам. Пример уведомления, которое видит пользователь при блокировке, представлен на рисунке 4.28.

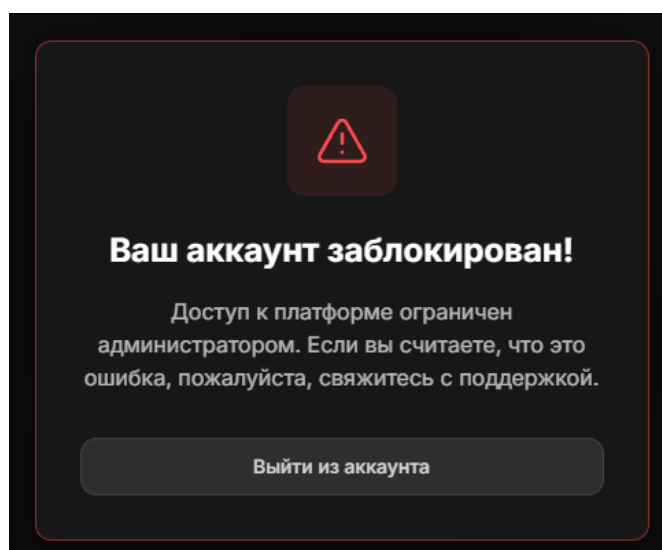


Рисунок 4.28 – Уведомление о блокировке

В итоге тестирование показало, что механизм блокировки и разблокировки пользователей работает корректно: администратор может управлять

доступом, система блокирует доступ заблокированного пользователя, а после снятия блокировки права пользователя полностью восстанавливаются.

4.8 Вывод по разделу

В ходе комплексного тестирования веб-приложения были проверены все ключевые модули. Основное внимание уделялось процессу создания и редактирования заказов и услуг, работе системы откликов, рейтингов и избранного, функционалу каталога с фильтрацией, а также административным функциям, включая массовую рассылку уведомлений, управление категориями и блокировку пользователей. Были проведены проверки корректности отображения данных, обработки ошибок и ограничений доступа по ролям.

Тестирование подтвердило, что интерфейс приложения стабильно реагирует на действия пользователя: данные корректно сохраняются и отображаются в соответствующих разделах, система уведомлений и рейтингов функционирует корректно, фильтры каталога работают как задумано, а возможность добавления и удаления элементов в избранное реализована без ошибок. Особое внимание уделялось поведению при некорректном вводе данных, граничных и ошибочных ситуациях, включая попытки создания ресурсов без необходимых прав или полей, что обеспечило надежность проверки и валидации на фронтенде и серверной части.

Проверка производительности и взаимодействия фронтенда с бэкендом показала, что приложение обеспечивает быстрый отклик и стабильность при различных сценариях, включая массовые операции и многопользовательскую активность. Были подтверждены корректность работы защищенных маршрутов, обработка ошибок и устойчивость к некорректным действиям пользователя. Механизмы безопасности, такие как проверка JWT-токенов, хеширование паролей, контроль ролей и блокировка пользователей, обеспечивают защиту данных и предотвращение несанкционированного доступа.

Проведенное тестирование показало, что веб-приложение отличается высокой надежностью, безопасностью и удобством использования. Критические функции работают корректно, ошибки обрабатываются должным образом, а интерфейс предоставляет пользователю интуитивно понятный и стабильный опыт работы. Результаты подтверждают готовность системы к эксплуатации в реальных условиях и уверенность в реализации функционала.

5 Руководство пользователя

Данное руководство описывает работу с системой фриланс-маркетплейса. Оно предназначено для пользователей, которые хотят использовать платформу для поиска исполнителей или выполнения заказов.

5.1 Руководство пользователя для роли Исполнитель

Возможности этих пользователей схожи, за исключением создаваемых объявлений, поэтому рассмотрение функционала будет базироваться на исполнителе, так фактически то же самое имеет возможность делать и заказчик.

Пользователь может зарегистрироваться в системе с ролью «Исполнитель». При регистрации пользователь может указать логин, email и пароль. Пользователь может создать пароль длиной не менее 6 символов, содержащий заглавные буквы, цифры и специальные символы. После заполнения данных пользователь может выбрать роль «Исполнитель» или обе роли одновременно, если планирует также создавать заказы. Затем пользователь может подтвердить email по ссылке из письма. Регистрация представлена на рисунке 5.1.

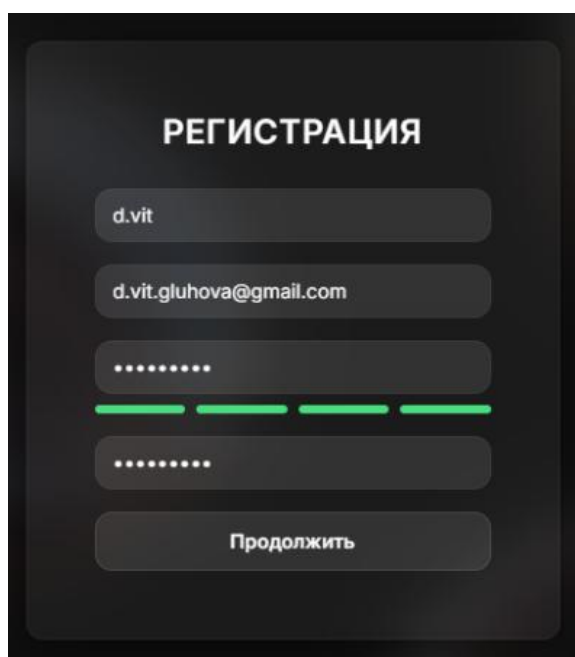


Рисунок 5.1 – Регистрация пользователя

После подтверждения электронной почты можно завершить регистрацию и войти в систему, форма логина представлена на рисунке 5.2.

				ДП 05.00.ПЗ			
	ФИО	Подпись	Дата				
Разраб.	Глухова Д.В.			5 Руководство пользователя	Лит.	Лист	Листов
Пров.	Уласевич Н.И.				У	1	19
					БГТУ 1-40 05 01, 2026		
Н. контр.	Николайчук А.Н.						
Утв.	Блинова Е.А.						

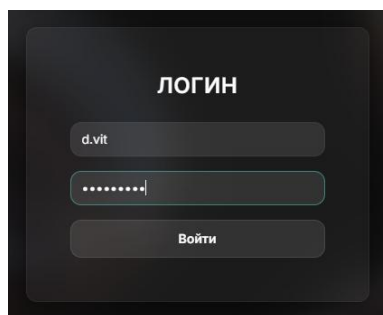


Рисунок 5.2 – Вход в систему

После входа пользователь попадает на главную страницу, где может видеть статистику платформы, популярные категории, лучших исполнителей и заказчиков, а также ленту активности. Для исполнителей на главной отображаются открытые заказы, которые пользователь может просмотреть и откликнуться. Главная страница представлена на рисунке 5.3.

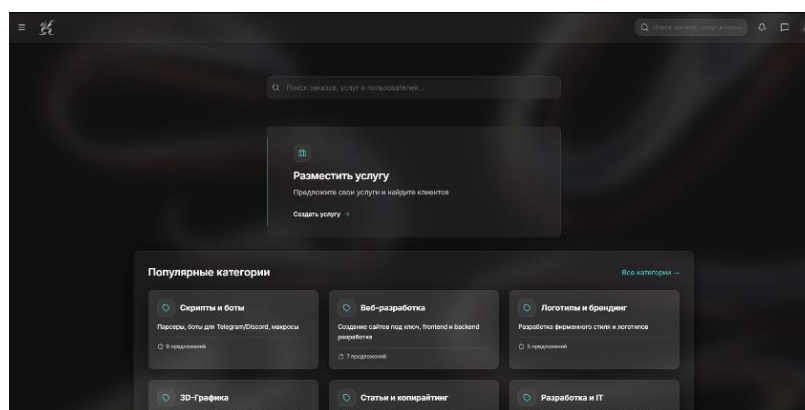


Рисунок 5.3 – Главная страница

При переходе в раздел «Профиль» через меню навигации, в редакторе профиля можно заполнить личные данные: имя, фамилию, страну и город, образование, контакты (телефон, email, сайт), информацию о себе. Добавить навыки через специальное поле: введите навык и нажмите Enter, чтобы добавить тег. Навыки помогают заказчикам находить вас по специализации. Профиль представлен на рисунке 5.4.

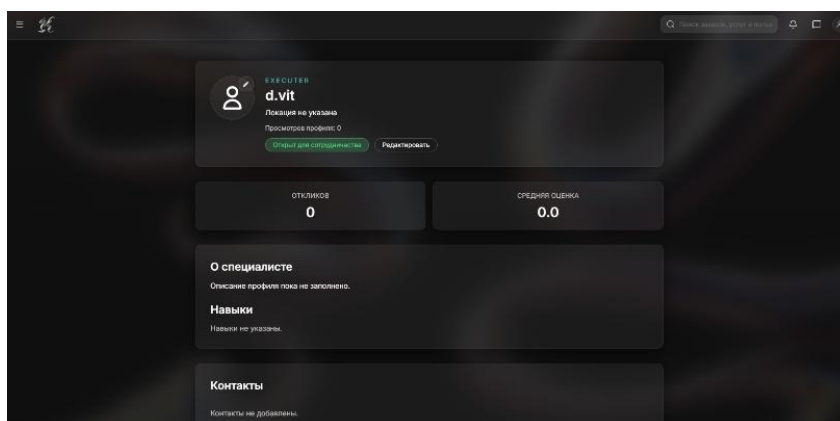


Рисунок 5.4 – Профиль пользователя

Пользователь может добавить контакты в разделах «Мессенджеры», «Социальные сети» и «Другие контакты». Пользователь может указать платформу и имя пользователя или ссылку. Это упрощает связь с заказчиками. Добавление контактов представлено на рисунке 5.5.

Рисунок 5.5 – Добавление контактов

После заполнения профиля пользователь может создать резюме. Резюме – отдельный документ, который виден заказчикам и повышает шансы на получение заказов. В форме резюме пользователь может указать название, описание, опыт работы, образование, навыки (через теги) и ссылку на портфолио. После создания резюме отправляется на модерацию администратором. После одобрения оно становится видимым в профиле. Пользователь может активировать или скрыть резюме через переключатель «Активно/Скрыто». Создание резюме представлено на рисунке 5.6.

Рисунок 5.6 – Добавление резюме

Как исполнитель пользователь может создавать услуги – готовые предложения, которые заказчики могут приобрести. Пользователь может перейти в раздел создания услуги через меню или кнопку на главной. В форме пользователь может указать название, подробное описание, цену и выбрать категорию. Категория обязательна и определяет, где услуга будет отображаться в каталоге. После создания услуга появляется в каталоге и доступна для просмотра. Создание услуги показано на рисунке 5.7.

Рисунок 5.7 – Создание услуги

Пользователь может редактировать услугу: изменить название, описание, цену или категорию. Пользователь может удалить услугу через раздел «Мои заказы и услуги», где отображаются все услуги пользователя. При удалении система запрашивает подтверждение. Услуги помогают получать заказы без поиска: заказчики находят услуги пользователя в каталоге и могут сразу их приобрести или связаться для обсуждения деталей. Чем больше услуг пользователь создаст в разных категориях, тем выше его видимость и шансы на получение заказов. Редактирование услуги представлено на рисунке 5.8.

Рисунок 5.8 – Редактирование услуги

Пользователь может открыть каталог через меню навигации. В каталоге отображаются заказы, услуги и пользователи. Для исполнителей наиболее интересны заказы. Каталог, который содержит услуги, заказы, пользователей, сортировки и фильтры представлен на рисунке 5.9.

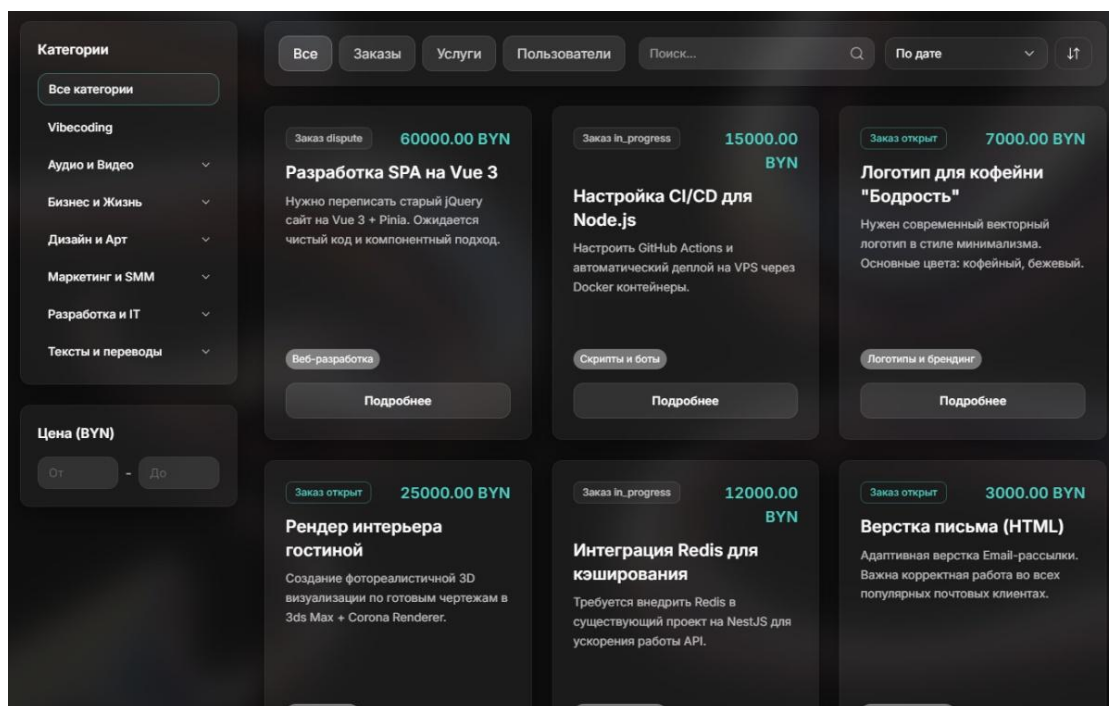


Рисунок 5.9 – Страница каталога

Пользователь может использовать фильтры для поиска: ввести ключевые слова в строку поиска, выбрать категорию в боковой панели, указать диапазон бюджета. Пользователь может сортировать результаты: по дате создания, по цене, по названию. Фильтры представлены на рисунке 5.10.

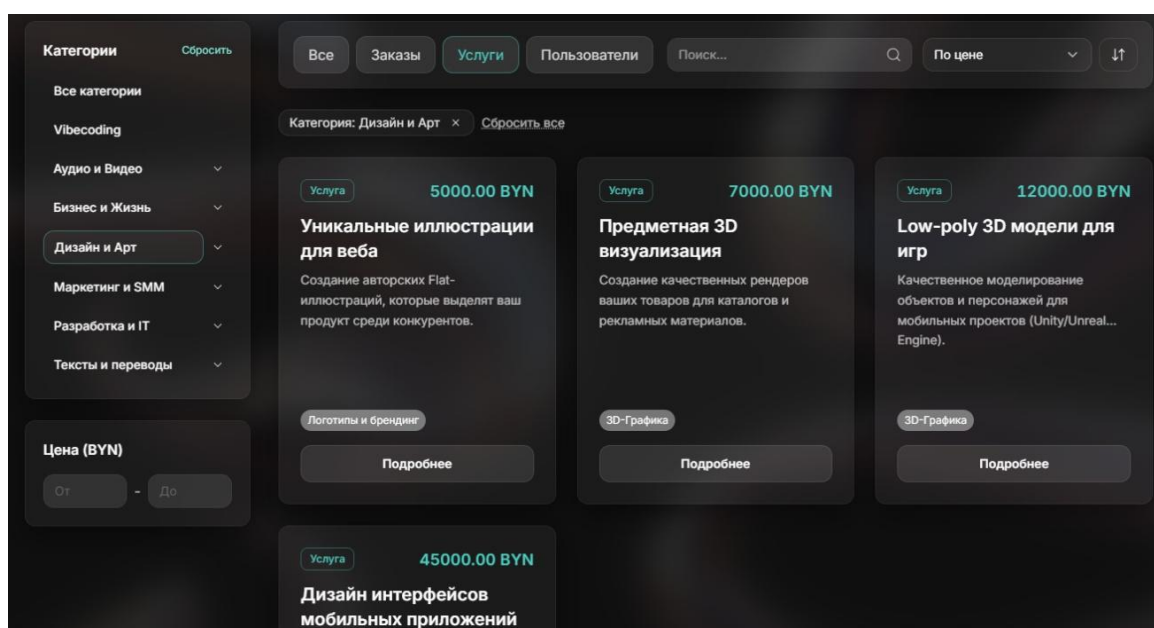


Рисунок 5.10 – Фильтры каталога

При выборе категории система автоматически показывает заказы из этой категории и всех её подкатегорий. Например, при выборе «Веб-разработка» пользователь может видеть заказы по «Frontend», «Backend», «Fullstack» и другим подкатегориям. Результат фильтрации представлен на рисунке 5.11.

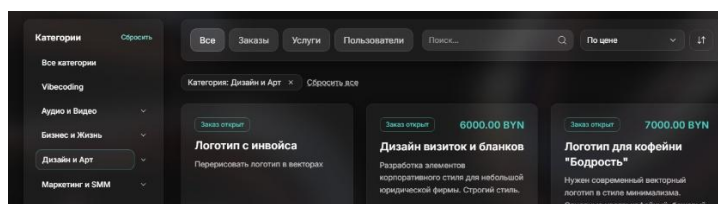


Рисунок 5.11 – Фильтрация по категории

Пользователь может открыть интересующий заказ, кликнув на карточку. На странице заказа отображаются название, описание, бюджет, срок выполнения, категория, информация о заказчике, статус заказа и количество откликов. Одна из карточек заказа представлена на рисунке 5.12.

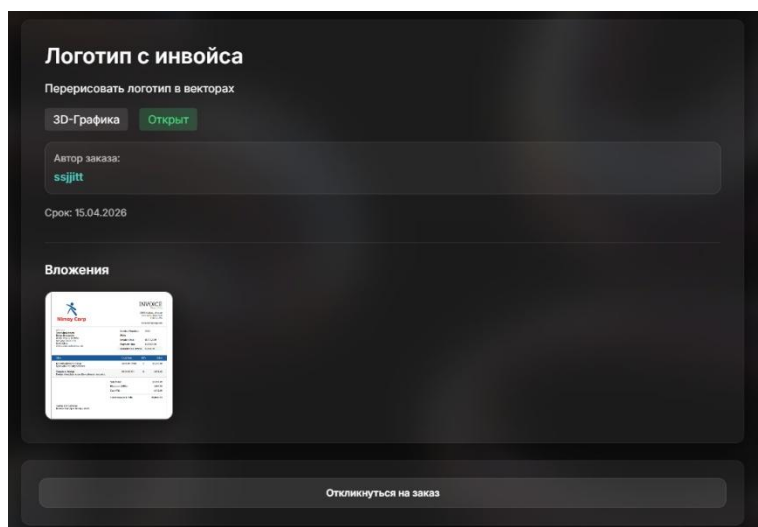


Рисунок 5.12 – Карточка заказа

Если заказ открыт, и пользователь еще не откликнулся, он может видеть форму отклика. В форме пользователь может указать сообщение заказчику (описать подход, опыт, почему он подходит) и предложенную цену, если хочет предложить другую сумму. Поля опциональны, но заполнение повышает шансы на одобрение. Отклик на заказ представлен на рисунке 5.13.

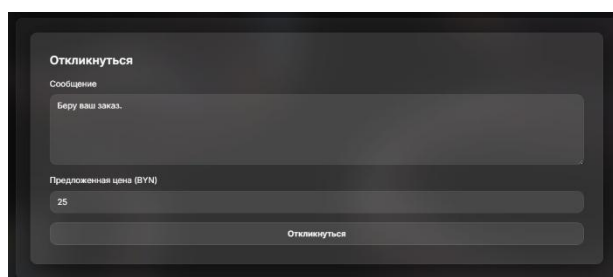


Рисунок 5.13 – Отклик на заказ

Статус отклика отображается на странице заказа: «pending» (ожидает рассмотрения), «approved» (принят), «rejected» (отклонен).

В разделе «Мои заказы и услуги» пользователь может видеть все созданные услуги. Страница представлена на рисунке 5.14. Здесь пользователь может просмотреть каждую услугу, перейти к редактированию или удалению. Услуги отображаются с основной информацией: название, категория, цена, дата создания. Пользователь на этой странице может перейти к редактированию или просмотру на странице услуги.

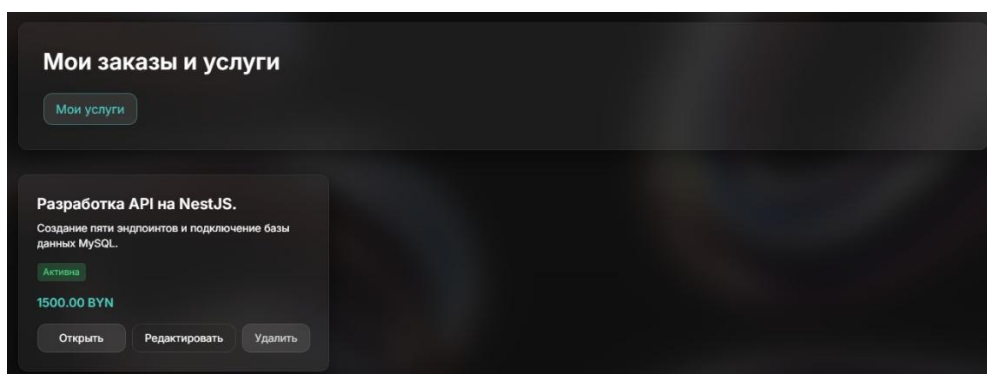


Рисунок 5.14 – Мои заказы и услуги

После завершения заказа или услуги пользователь может оставить отзыв заказчику. На странице завершенного заказа или услуги отображается форма отзыва. Пользователь может выбрать оценку от 1 до 5 звезд и при желании добавить текстовый комментарий. Отзыв влияет на рейтинг заказчика и помогает другим исполнителям оценить надежность. Список отзывов для одного из пользователей представлен на рисунке 5.15.

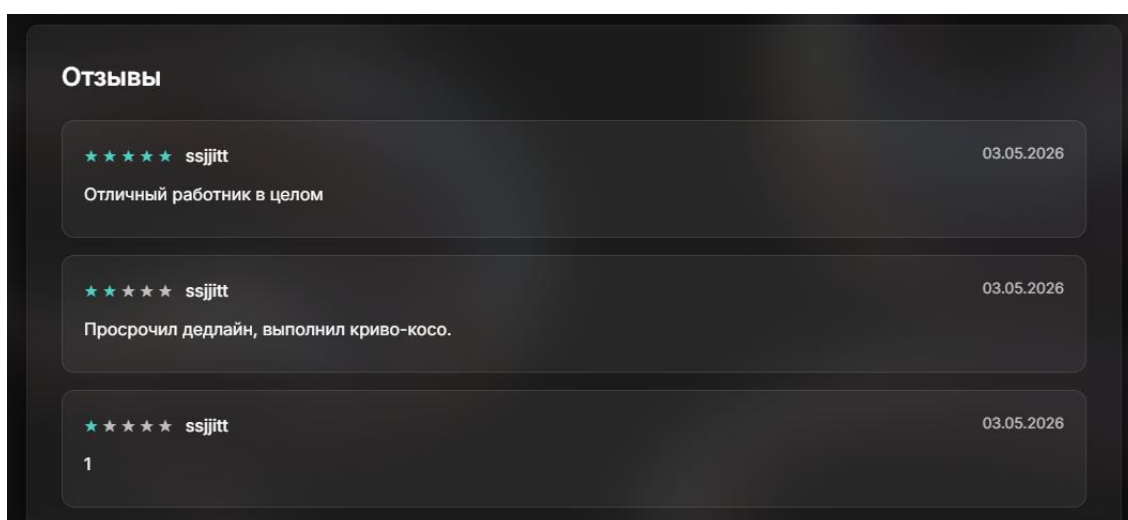


Рисунок 5.15 – Отзыв о пользователе

Пользователь может добавлять заказы и услуги в избранное для быстрого доступа. На странице заказа или услуги пользователь может нажать кнопку «Добавить в избранное». Элемент сохранится в списке избранного,

доступном через меню навигации. Пользователь, добавленный в избранное, представлен на рисунке 5.16.

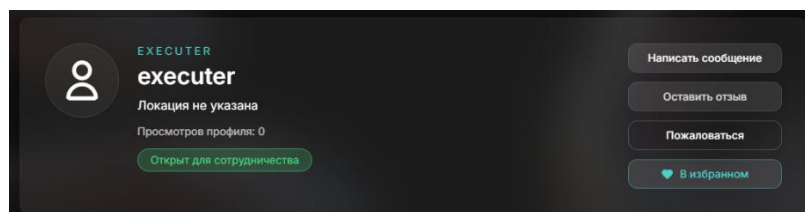


Рисунок 5.16 – Добавление в избранное

В избранном пользователь может видеть все сохраненные элементы с основной информацией. Это удобно для отслеживания интересных заказов или услуг, к которым пользователь хочет вернуться позже. Пользователь может удалить элемент из избранного в любой момент. Пользователь, который был удален из избранного, представлен на рисунке 5.17.

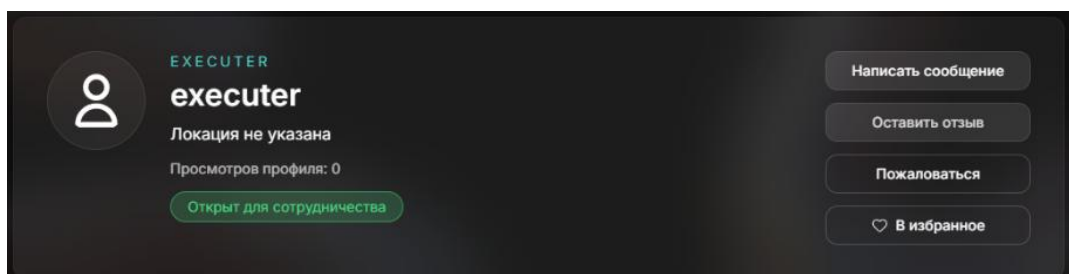


Рисунок 5.17 – Удаление из избранного

Система поддерживает обмен сообщениями. При просмотре профиля другого пользователя, исполнитель может начать переписку. Чаты доступны через раздел «Чаты» в меню навигации, где пользователь может видеть все активные переписки. В чате пользователь может видеть историю сообщений и отправлять новые. Сообщения доставляются в реальном времени, что позволяет оперативно обсуждать детали проекта. Чат помогает согласовывать требования, сроки, бюджет и другие вопросы без использования внешних мессенджеров. Чаты пользователя с сообщениями представлены на рисунке 5.18.

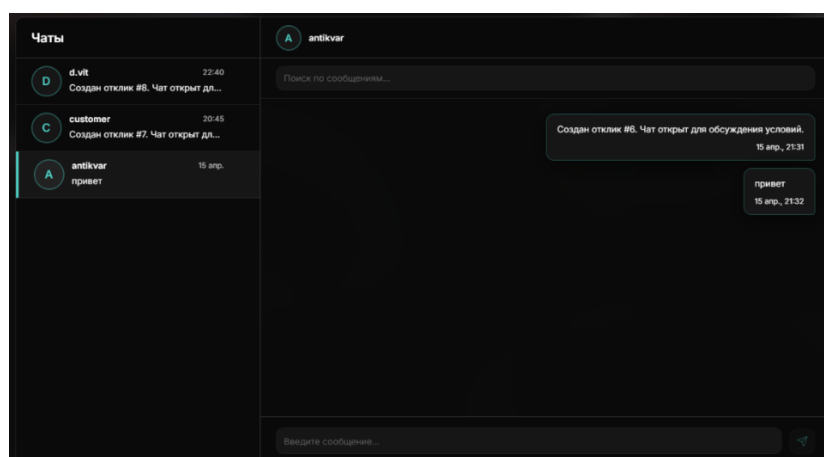


Рисунок 5.18 – Чаты пользователя

Система отправляет уведомления, представленные на рисунке 5.19, о важных событиях: одобрении или отклонении отклика, новых сообщениях в чате, изменениях статуса заказа, новых отзывах и других действиях, связанных с деятельностью пользователя.

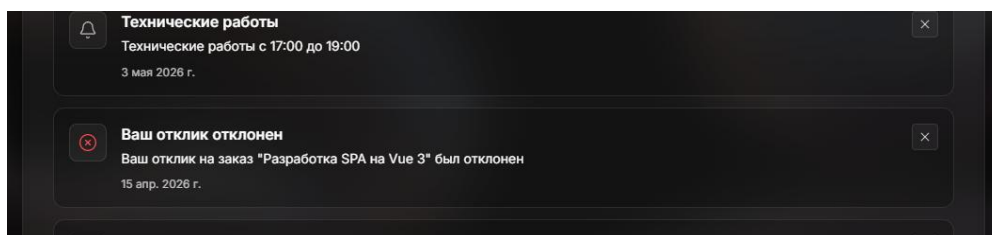


Рисунок 5.19 – Страница уведомлений

Уведомления отображаются в специальном разделе через меню навигации. Пользователь может просмотреть все уведомления, просмотреть непросчитанные уведомления и отметить, как прочитанное. Это помогает не пропускать важные события и оперативно реагировать на действия заказчиков.

5.2 Руководство пользователя для роли Заказчик

Пользователь может зарегистрироваться в системе с ролью «Заказчик». При регистрации можно указать логин, email и пароль. Пароль должен быть не менее 6 символов, содержать заглавные буквы, цифры и специальные символы. После заполнения данных можно выбрать роль «Заказчик» или обе роли одновременно, если планируется также выполнять заказы. Затем нужно подтвердить email по ссылке из письма. После подтверждения можно завершить регистрацию и войти в систему.

После входа пользователь попадает на главную страницу, которая представлена на рисунке 5.20, где видит статистику платформы, популярные категории, лучших исполнителей и заказчиков. Для заказчиков на главной отображаются доступные услуги, которые можно просмотреть и приобрести.

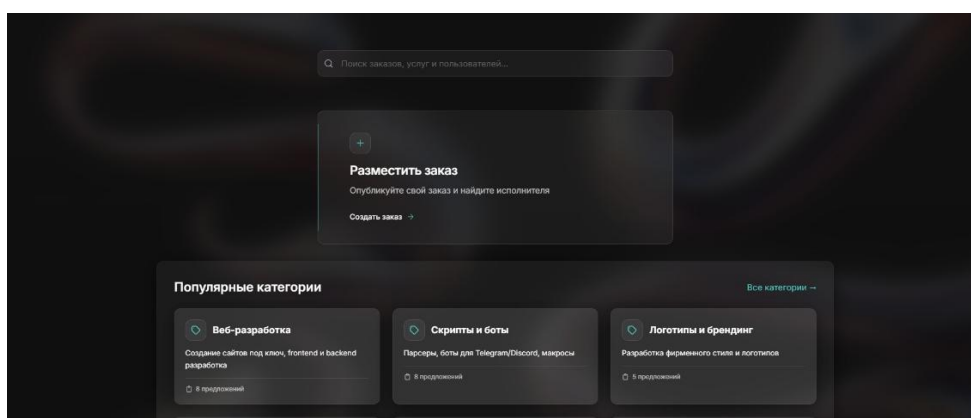


Рисунок 5.20 – Главная страница

При переходе в раздел «Профиль» через меню навигации можно заполнить личные данные, добавить навыки и контакты.

Как заказчик пользователь может создавать заказы – описание задач, которые нужно выполнить. Можно перейти в раздел создания заказа через меню или кнопку на главной. Создание заказа представлено на рисунке 5.21. В форме можно указать название, подробное описание, бюджет в рублях (опционально), срок выполнения (опционально) и выбрать категорию. Категория обязательна и определяет, где заказ будет отображаться в каталоге.

Скриншот формы «Создать заказ» с темной темой. В форме есть следующие поля: «Название» (текстовое поле), «Описание» (текстовое поле), «Бюджет (BYN)» (текстовое поле), «Срок выполнения» (календарный селектор с датой «Дд.Мм.Гггг»), «Категория» (выпадающий список с подсказкой «Выберите категорию»), «Вложения» (ссылка «Прикрепить файл»). Внизу находятся кнопки «Создать» и «Отмена».

Рисунок 5.21 – Создание заказа

После создания заказ появляется в каталоге и доступен для просмотра и для откликов заинтересованных исполнителей.

Пользователь может редактировать заказы: изменить название, описание, бюджет, срок выполнения или категорию, это представлено на рисунке 5.22. Можно удалить заказ через раздел «Мои заказы и услуги», где отображаются заказы пользователя. При попытке удалении система запрашивает подтверждение для разрешения или отмены действия.

Скриншот формы «Редактировать заказ» с темной темой. В форме есть следующие поля: «Название» (текстовое поле со значением «Разработка SPA на Vue 3»), «Описание» (текстовое поле со значением «Нужно переписать старый jQuery сайт на Vue 3 + Pinia. Ожидается чистый код и компонентный подход»), «Бюджет (BYN)» (текстовое поле со значением «60000,00»), «Срок выполнения» (календарный селектор с датой «04.05.2026»), «Категория» (выпадающий список со значением «Веб-разработка»), «Статус заказа» (выпадающий список со значением «Выберите статус»), «Вложения» (ссылка «Прикрепить файл»). Внизу находятся кнопки «Обновить» и «Отмена».

Рисунок 5.22 – Редактирование заказа

Пользователь может открыть каталог через меню навигации. В каталоге отображаются заказы, услуги и пользователи. Для заказчиков наиболее

интересны услуги и исполнители. Можно использовать фильтры для поиска: ввести ключевые слова в строку поиска, выбрать категорию в боковой панели, указать диапазон. Можно сортировать по дате создания, цене или названию.

Пользователь может открыть интересующую услугу, кликнув на карточку. На странице услуги отображаются название, описание, цена, категория, информация об исполнителе, рейтинг исполнителя и портфолио.

Если услуга доступна, пользователь может видеть форму отклика. В форме можно указать сообщение исполнителю и предложенную цену, если нужно обсудить другую сумму. Отклик представлен на рисунке 5.23.

The screenshot shows a dark-themed web interface for a service listing. At the top, the title 'Разработка API на NestJS.' is displayed in white. Below it, a subtitle reads 'Создание пяти эндпоинтов и подключение базы данных MySQL.' There are two status buttons: 'Веб-разработка' (grey) and 'Активна' (green). The author of the service is listed as 'd.vit' in a light blue font. The price is shown as 'Цена: 1500.00 BYN' in green. A section titled 'Фото и файлы' contains a thumbnail image of a flowchart. At the bottom, a grey box contains the text 'Вы откликнулись на эту услугу' and 'Статус: На рассмотрении'. To the right of this box is a button labeled 'Отменить отклик'.

Рисунок 5.23 – Отклик на услугу

Статус отклика отображается на странице услуги: «pending» (ожидает рассмотрения), «approved» (принят), «rejected» (отклонен).

В разделе «Мои заказы и услуги» пользователь может видеть все созданные заказы. Здесь можно просмотреть каждый заказ, перейти к редактированию или удалению. Заказы отображаются с основной информацией: название, категория, бюджет, дата создания, статус, количество откликов. Можно быстро перейти к редактированию или просмотру на странице заказа.

На странице своего заказа заказчик может видеть все отклики от исполнителей. Для каждого отклика отображаются имя исполнителя, сообщение, предложенная цена (если указана) и статус. Заказчик может одобрить или отклонить отклик. При одобрении статус заказа меняется, и исполнитель получает уведомление. Заказчик может одобрить несколько откликов, если это необходимо. Управление откликами представлено на рисунке 5.24.

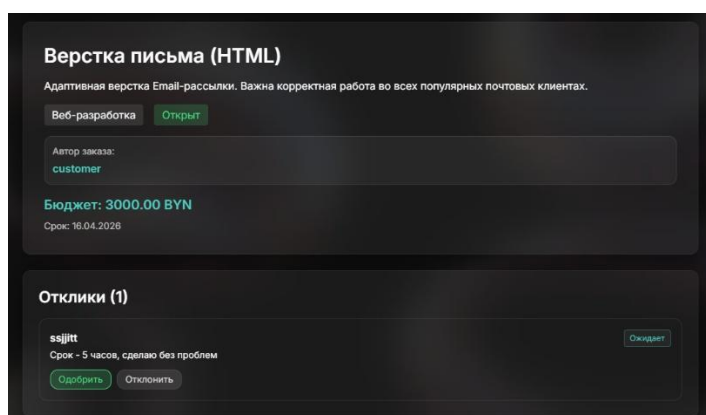


Рисунок 5.24 – Управление откликами на заказ

После одобрения отклика заказчик может завершить заказ, когда работа выполнена. На странице заказа есть кнопка «Завершить заказ», которая меняет статус на «Завершен», что показано на рисунке 5.25. После завершения заказа становится доступна возможность оставить отзыв исполнителю.

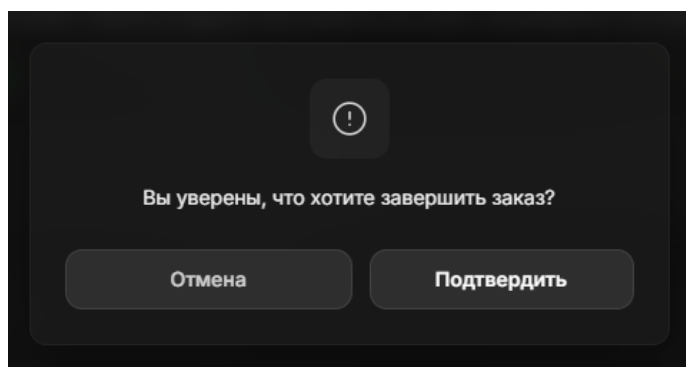


Рисунок 5.25 – Завершение заказа

После завершения заказа пользователь может оставить отзыв исполнителю. На странице завершенного заказа отображается форма отзыва. Пользователь может выбрать оценку от 1 до 5 звезд и при желании добавить текстовый комментарий. Отзывы с оценками представлены на рисунке 5.26.

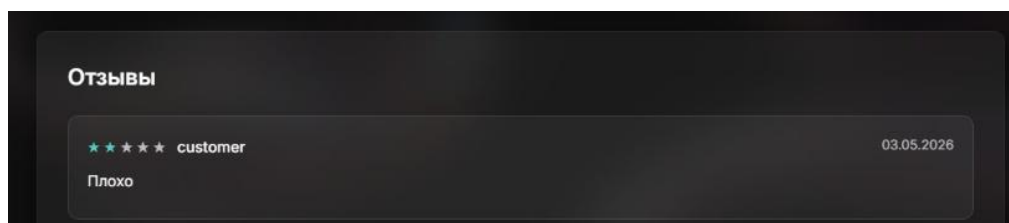


Рисунок 5.26 – Отзыв о пользователе

Пользователь может добавлять заказы и услуги в избранное для быстрого доступа. В избранном можно видеть все сохраненные элементы с основной информацией и удалить элемент из избранного в любой момент.

Система отправляет уведомления, которые показаны на рисунке 5.27, о важных событиях: новых откликах на заказы, одобрении или отклонении

отклика на услугу, новых сообщениях в чате, изменениях статуса заказа, новых отзывах и других действиях, связанных с деятельностью пользователя.

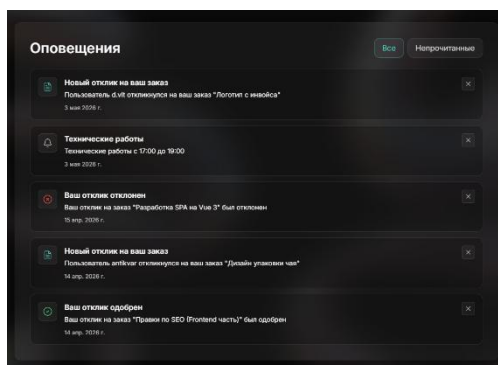


Рисунок 5.27 – Страница уведомлений

Система поддерживает обмен сообщениями. При просмотре профиля другого пользователя или при одобрении отклика заказчик может начать переписку. Чаты доступны через раздел «Чаты» в меню навигации. В чате можно видеть историю сообщений и отправлять новые. Сообщения доставляются в реальном времени, что позволяет оперативно обсуждать детали проекта.

5.3 Руководство пользователя для роли Администратор

Пользователь с ролью администратора получает доступ к панели управления после входа. Администратор видит расширенную главную страницу с дополнительными разделами, недоступными обычным пользователям. На главной странице администратор может видеть статистику платформы, инструменты для управления системой и быстрые действия для модерации.

На главной странице администратор может видеть общую статистику платформы, которая отображается в виде анимированных карточек с ключевыми показателями. Пользователь может видеть общее количество пользователей, зарегистрированных на платформе, общее количество созданных заказов, общее количество созданных услуг и количество активных заказов, которые находятся в работе. Статистика обновляется автоматически и отображается с анимацией при загрузке страницы, что позволяет администратору быстро оценить состояние платформы, показано на рисунке 5.28.

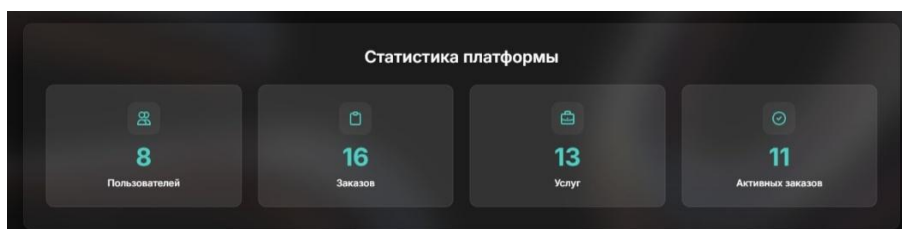


Рисунок 5.28 – Статистика платформы

Администратор может просматривать статистику за конкретный месяц в виде интерактивного календаря. Администратор может выбрать тип

статистики из выпадающего списка: заказы, услуги, выполненные заказы или новые регистрации. Пользователь может выбрать месяц из списка всех двенадцати месяцев года и указать год для просмотра исторических данных. Календарь отображает каждый день месяца в виде ячейки, где высота цветного столбца соответствует количеству событий в этот день. Администратор может навести курсор на любой день, чтобы увидеть точное количество событий в подсказке. Максимальное значение отображается внизу календаря для удобства оценки масштаба данных. Календарь представлен на рисунке 5.29.

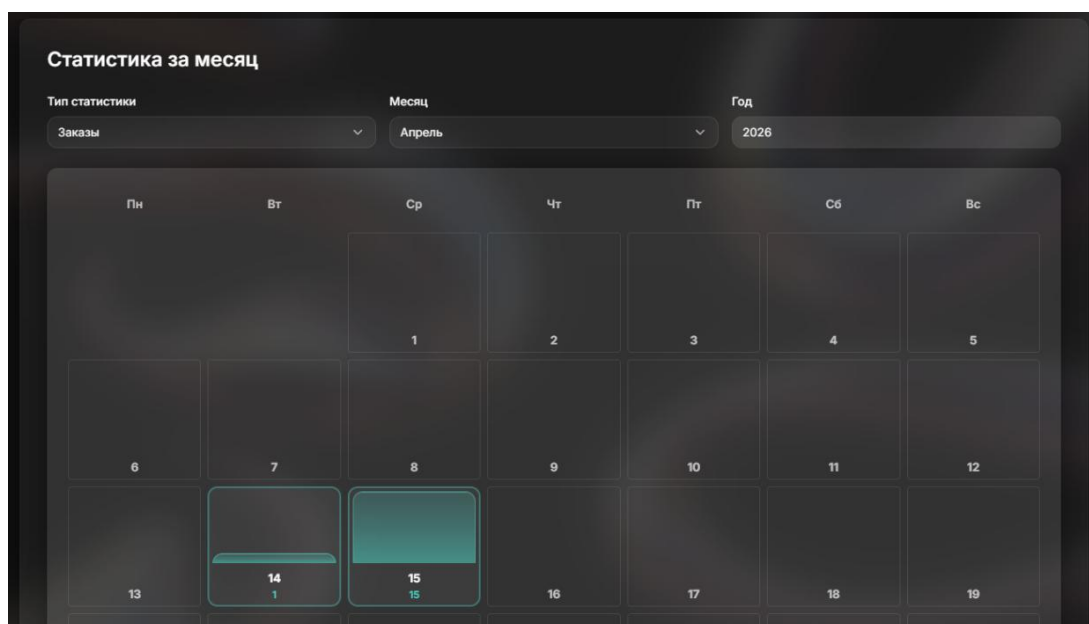


Рисунок 5.29 – Календарь статистики

Администратор может заблокировать пользователя, нажав кнопку «Заблокировать» рядом с его именем. При блокировке пользователь теряет доступ к системе и не может войти в свой аккаунт. Заблокированный пользователь не может создавать заказы или услуги, откликаться на проекты или выполнять любые другие действия на платформе. Кнопка представлена на рисунке 5.30.

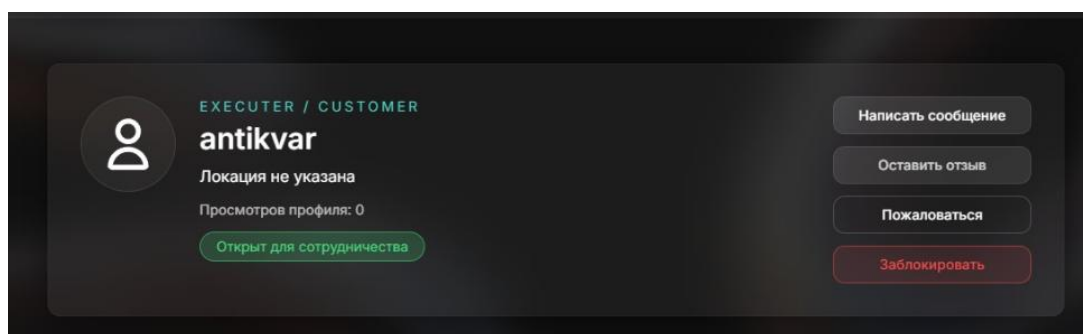


Рисунок 5.30 – Кнопка для блокировки пользователя

Администратор может разблокировать заблокированного пользователя в любой момент, нажав кнопку «Разблокировать», что восстановит доступ пользователя к веб-приложению. Блокировка и разблокировка выполняются

мгновенно, и изменения применяются сразу после подтверждения действия. Кнопка представлена на рисунке 5.31.

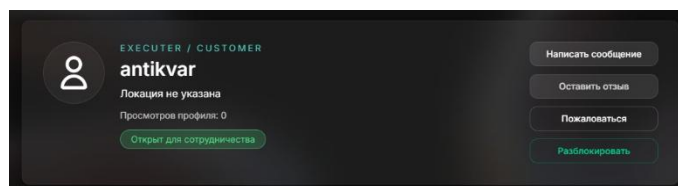


Рисунок 5.31 – Кнопка для разблокировки пользователя

Администратор может создавать категории для организации заказов и услуг на платформе. Администратор может перейти в раздел управления категориями в панели администратора и заполнить форму создания категории. После заполнения формы администратор может нажать кнопку «Создать категорию», и новая категория сразу появляется в системе и становится доступной для выбора при создании заказов и услуг, создание показано на рисунке 5.32.

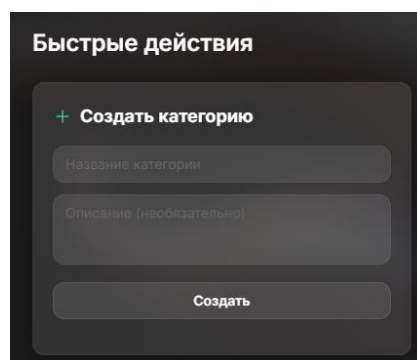


Рисунок 5.32 – Создание категории

Администратор может создавать подкатегории для существующих категорий, что позволяет организовать контент детально. При выборе родительской категории в каталоге автоматически отображаются все заказы и услуги из этой категории и всех её подкатегорий. Создание показано на рисунке 5.33.

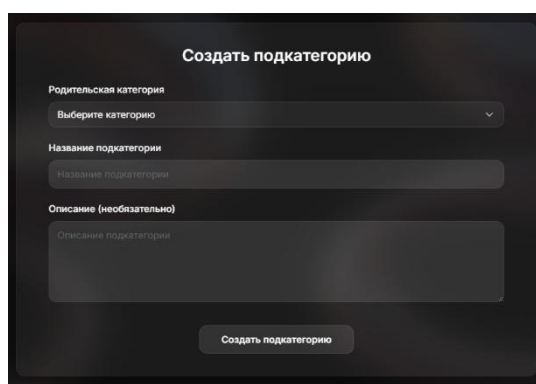


Рисунок 5.33 – Создание подкатегории

Администратор может просматривать все резюме, которые были созданы исполнителями и ожидают модерации. Пользователь может видеть

список резюме на главной странице в разделе быстрых действий, где отображаются резюме, ожидающие проверки. Администратор может одобрить резюме, нажав кнопку «Одобрить» рядом с резюме. После одобрения резюме становится видимым в профиле исполнителя и может быть просмотрено заказчиками при выборе исполнителя для своих проектов. Одобренное резюме помогает заказчикам оценить квалификацию исполнителя и принять решение о сотрудничестве. После одобрения резюме автоматически удаляется из списка ожидающих модерации, и исполнитель получает уведомление о том, что его резюме было одобрено. Вид быстрого действия для одобрения резюме в списке быстрого доступа представлен на рисунке 5.34.

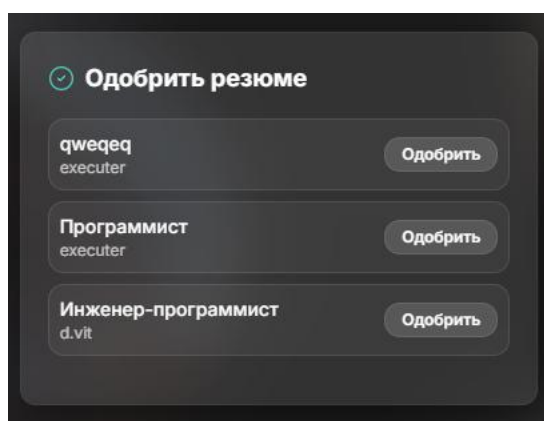


Рисунок 5.34 – Одобрение резюме

Администратор может поддерживать связь с пользователями платформы через систему уведомлений. Пользователь может отправлять уведомления всем зарегистрированным пользователям, информируя их о важных событиях, обновлениях платформы, изменениях в правилах или других значимых новостях. Форма отправки уведомлений представлена на рисунке 5.35.

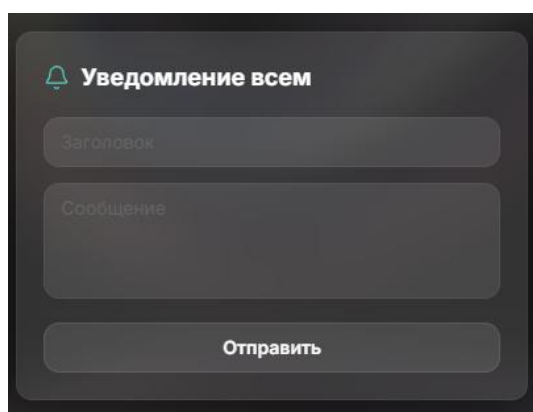


Рисунок 5.35 – Форма отправки уведомлений

Уведомления доставляются всем пользователям одновременно и отображаются в их разделе уведомлений, где они могут прочитать сообщение в удобное время. Это позволяет администратору эффективно информировать

большое количество пользователей о важных событиях без необходимости индивидуальной коммуникации с каждым пользователем.

5.4 Руководство пользователя для роли Менеджер

Пользователь с ролью менеджера после входа в систему перенаправляется на панель менеджера, которая служит центральной точкой доступа ко всем полномочиям модерации и поддержки. В отличие от заказчика и исполнителя, менеджер не видит стандартную домашнюю витрину маркетплейса при открытии главной страницы: интерфейс сразу ориентирован на оперативный контроль очередей обращений, споров, пользователей и контента. На панели отображаются агрегированные показатели с возможностью перехода в соответствующий раздел одним щелчком, что позволяет менеджеру за секунды оценить нагрузку на поддержку и модерацию. Общий вид панели менеджера представлен на рисунке 5.36.

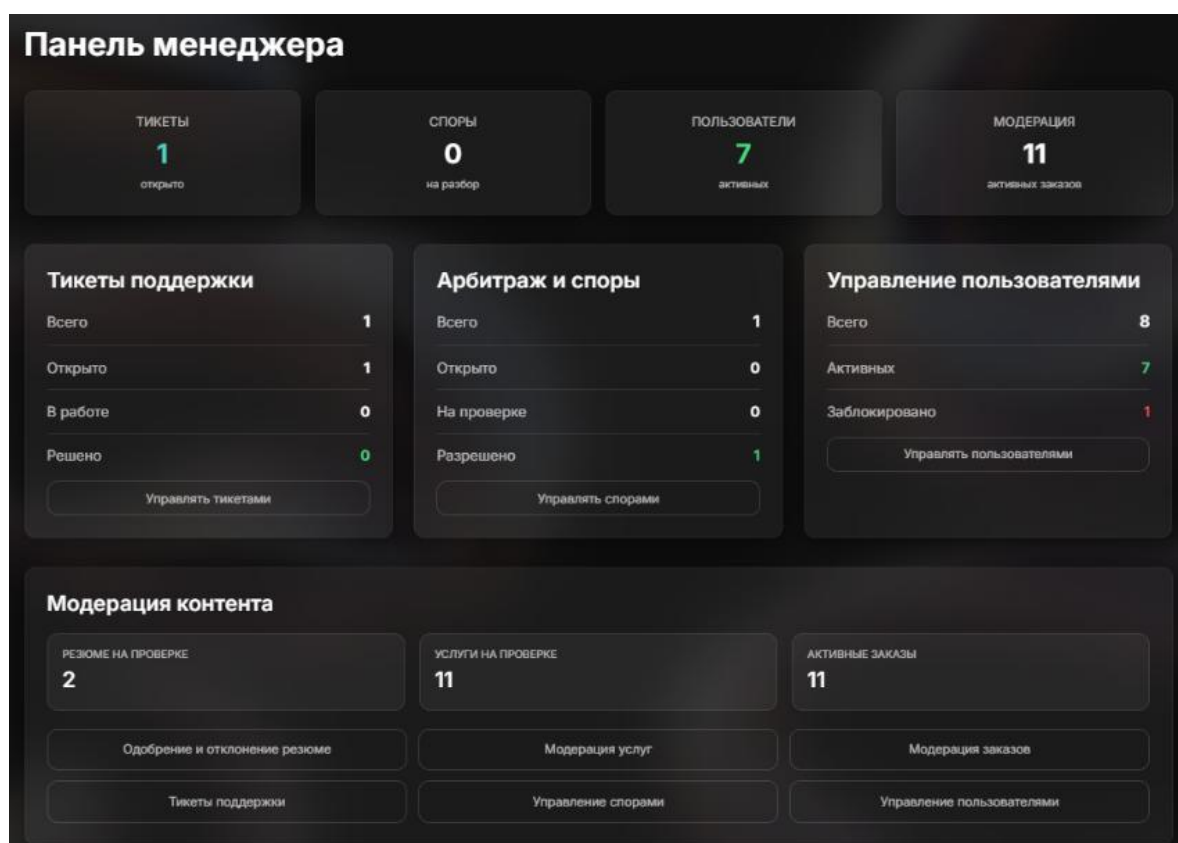


Рисунок 5.36 – Панель менеджера

Раздел «Тикеты помощи» содержит таблицу всех обращений пользователей в поддержку. Менеджер может отфильтровать записи по статусу: все, открытые, в работе, решённые, закрытые. Внутри карточки менеджер изменяет статус жизненного цикла обращения и сохраняет изменения, фиксируя ход работы с обращением. Вид списка тикетов представлен на рисунке 5.37.

ID	Тема	Категория	Приоритет	Статус	От кого	Дата	Действие
1	Жалоба на service #14	Нарушение	medium	open	customer	15.04.2026	Открыть

Рисунок 5.37 – Тикеты помощи

Раздел «Споры и арбитраж» предназначен для разрешения конфликтов между заказчиком и исполнителем по заказам. Таблица споров поддерживает фильтрацию по статусам: открытый, на разборе, решённый, закрытый. Для споров, ещё не переведённых в финальное состояние, в списке доступны быстрые действия: зафиксировать исход в пользу заказчика, в пользу исполнителя или назначить исход «возврат» без перехода на отдельную страницу. Интерфейс списка споров с быстрыми кнопками представлен на рисунке 5.38.

ID	Заказ	Заказчик	Исполнитель	Причина	Статус	Решение	Дата	Действия
1	Заказ #1	customer	executor	почему отклонили	Решен	Выиграл заказчик	15.04.2026	Открыть

Рисунок 5.38 – Список споров

Раздел «Модерация услуг» предназначен для работы с услугами, ожидающими проверки. Перечень полей и состав действий с услугами для менеджера на экране представлен на рисунке 5.39.

ID	Услуга	Категория	Исполнитель	Цена	Статус	Дата	Действия
27	1	Vibecoding	sojllt	—	На модерации	03.05.2026	Просмотр, Одобрить, Скрыть, Удалить
26	Идуй	Анимация	sojllt	123,00 BYN	Активно	03.05.2026	Просмотр, Скрыть, Удалить
25	Разработка API на NestJS.	Веб-разработка	d.vlt	1 500,00 BYN	Активно	03.05.2026	Просмотр, Скрыть, Удалить
13	Frontend разработка на React/Next.js	Веб-разработка	executor	50 000,00 BYN	Неактивно	15.04.2026	Просмотр, Показать в каталоге, Удалить

Рисунок 5.39 – Модерация услуг

Роль менеджера в системе охватывает линию поддержки (тикеты), арбитраж по заказам, защиту сообщества (блокировка пользователей) и

контентную модерацию, при этом все функции сгруппированы вокруг единой панели с понятной навигацией и показателями загрузки очередей.

5.5 Выводы по разделу

Руководства пользователя охватывают основные роли: исполнитель, заказчик, администратор и менеджер. Каждая роль имеет свой набор функций и возможностей, которые позволяют пользователям эффективно работать на платформе в соответствии со своими задачами и потребностями.

Исполнители могут создавать и публиковать услуги с указанием названия, описания, цены и категории. Система позволяет редактировать и удалять свои услуги, управлять их статусом. Исполнители могут создавать резюме с информацией об опыте, образовании, навыках и портфолио, что помогает заказчикам оценить их компетенции. Через каталог исполнители ищут подходящие заказы, фильтруют по категориям, бюджету и другим параметрам, просматривают детали заказов и откликаются с предложением цены и сообщением. Система отслеживает статусы откликов (ожидание, принят, отклонен), позволяет просматривать историю взаимодействий с заказчиками и получать уведомления о новых заказах и изменениях статусов. Исполнители могут общаться с заказчиками через встроенную систему чатов, отправлять и получать сообщения в реальном времени, просматривать рабочий профиль с отзывами и рейтингами, управлять личным профилем, добавлять контакты и навыки, загружать аватары и обновлять информацию о себе.

Для заказчиков система предоставляет инструменты для создания заказов, поиска исполнителей, управления проектами, рассмотрения откликов и оценки качества работы. Заказчики могут эффективно решать свои задачи, находить подходящих исполнителей и получать качественные результаты от профессиональных специалистов.

Для администраторов система предоставляет инструменты для управления платформой, модерации контента, контроля пользователей и анализа активности. Администраторы могут поддерживать порядок на платформе, обеспечивать качество контента и способствовать развитию системы.

Для менеджеров система предоставляет инструменты оперативной модерации и сопровождения платформы без полномочий глобальной настройки, типичных для администратора. Менеджеры работают с заказами и услугами: проверяют публикации, одобряют их для отображения в каталоге, при необходимости скрывают материалы с указанием причины.

Все роли взаимодействуют друг с другом, создавая экосистему, где каждый участник может эффективно выполнять свои задачи. Система обеспечивает безопасность, прозрачность и удобство использования для всех типов пользователей, что способствует созданию доверительной среды и успешному взаимодействию между участниками платформы.

6 Технико-экономическое обоснование проекта

Основной целью экономического раздела является обоснование целесообразности разработки программного средства, созданного в рамках дипломного проекта. В данном разделе рассчитываются затраты на разработку веб-приложения, полная себестоимость, отпускная цена, прибыль и показатели экономической эффективности.

Разработка программных средств требует трудовых, материальных и финансовых ресурсов. Поэтому проект необходимо оценивать не только с технической, но и с экономической точки зрения.

6.1 Общая характеристика программного средства

В результате выполнения дипломного проекта разработано веб-приложение «Маркетплейс фриланс-заказов для студентов». Программное средство представляет собой нишевую платформу для взаимодействия студентов-исполнителей, заказчиков, администраторов и менеджеров. Приложение предназначено для поиска заказов, публикации услуг, формирования резюме, подачи откликов, коммуникации между участниками и модерации пользовательской активности.

Приложение поддерживает следующие роли: гость, администратор, заказчик, исполнитель и менеджер.

Для роли гостя предусмотрены следующие функции:

- регистрация и авторизация;
- просмотр каталога заказов, услуг и профилей;
- поиск, фильтрация и сортировка материалов каталога.

Для роли заказчика предусмотрены следующие функции:

- создание, редактирование и удаление заказов;
- просмотр профилей исполнителей и каталога услуг;
- рассмотрение откликов, общение с исполнителем в чате;
- оценка выполненной работы и написание отзывов.

Для роли исполнителя предусмотрены следующие функции:

- создание и редактирование резюме;
- создание и публикация услуг;
- поиск подходящих заказов;
- подача откликов и ведение переписки с заказчиком.

Для ролей администратора и менеджера реализованы функции контроля и сопровождения платформы:

				ДП 06.00.ПЗ						
	ФИО	Подпись	Дата							
Разраб.	Глухова Д.В.			6 Технико-экономическое обоснование проекта			Лит.	Лист	Листов	
Провер.	Уласевич Н.И.						У	1	14	
Консульт.	Соболевский А.С.						БГТУ 1-40 05 01, 2026			
Н. контр.	Николайчук А.Н.									
Утв.	Блинова Е.А.									

- управление категориями;
- просмотр статистики;
- модерация резюме, заказов и услуг;
- рассмотрение жалоб;
- блокировка и разблокировка пользователей.

Серверная часть реализована на Node.js с использованием Express.js. Клиентская часть выполнена на React.js. Для хранения данных используется MySQL, для обмена сообщениями в реальном времени применяется Socket.IO, а для развёртывания используется Docker.

Способ монетизации программного средства предполагает продажу разработанного веб-приложения с передачей имущественных прав заказчику, что соответствует характеру дипломного проекта: продукт является завершённым программным средством, которое может быть передано учебной организации, центру карьеры или коммерческому заказчику для последующего внедрения и сопровождения. В перспективе заказчик может развивать дополнительную монетизацию через комиссию за выполненные заказы или платное продвижение услуг, но эти прогнозные доходы не включаются в базовый расчет, чтобы не завышать экономический эффект без подтвержденной статистики.

6.2 Исходные данные для проведения расчетов

Источниками исходных данных, необходимых для проведения расчетов, являются методические рекомендации по экономическому обоснованию программных средств, структура разработанного приложения, календарный план дипломного проекта и экспертная оценка трудоемкости работ. Исходные данные для расчета приведены в таблице 6.1.

Таблица 6.1 – Исходные данные для расчета

Наименование показателя	Условное обозначение	Норматив
1	2	3
Норматив дополнительной заработной платы, %	Ндз	20
Ставка отчислений в Фонд социальной защиты населения, %	Нфсзн	34
Ставка отчислений в БРУСП «Белгосстрах», %	Нбгс	0,6
Норматив прочих прямых затрат, %	Нпз	16
Норматив накладных расходов, %	Ннр	28
Норматив расходов на сопровождение и адаптацию, %	Нрса	8
Ставка НДС, %	Нндс	20
Ставка налога на прибыль, %	Нп	20

Продолжение таблицы 6.1

1	2	3
Среднее количество рабочих дней в месяце	Дм	21,5
Стоимость одного машино-часа, руб.	Смч	0,12

Данные таблицы 6.1 используются для расчета затрат и эффективности разрабатываемого веб-приложения.

6.3 Методика обоснования цены

Стоимостная оценка веб-приложения у разработчика предполагает определение затрат по следующим статьям:

- основная и дополнительная заработная плата исполнителей;
- отчисления в ФСЗН и БРУСП «Белгосстрах»;
- расходы на материалы;
- расходы на специальное оборудование и платные услуги;
- расходы на оплату машинного времени;
- прочие прямые затраты;
- накладные расходы;
- расходы на сопровождение и адаптацию.

На основании этих затрат рассчитывается себестоимость программного средства. Затем определяется отпускная цена с учетом плановой прибыли и налога на добавленную стоимость, подход позволяет оценить целесообразность разработки без необоснованного завышения прогнозных доходов.

6.4 Расчет затрат времени на разработку программного средства

В таблице 6.2 приведены работы, выполненные для создания веб-приложения, должности исполнителей и затраты рабочего времени. Расчет выполнен при условии разработки программного средства в организации.

Таблица 6.2 – Затраты рабочего времени на разработку ПС

Содержание работ	Исполнитель	Количество исполнителей	Затраты рабочего времени, дней
1	2	3	4
Анализ предметной области и аналогов фриланс-платформ	бизнес-аналитик	1	4
Формирование функциональных требований и ролей пользователей	бизнес-аналитик	1	4
Планирование состава работ и структуры проекта	продакт-менеджер	1	3
Проектирование пользовательских сценариев и интерфейса	UX/UI дизайнер	1	10

Продолжение таблицы 6.2

1	2	3	4
Проектирование базы данных и связей между сущностями	администратор базы данных	1	6
Разработка серверной части и REST API	backend-разработчик	1	14
Разработка авторизации, ролей и разграничения доступа	backend-разработчик	1	4
Разработка чатов и уведомлений в реальном времени	backend-разработчик	1	4
Разработка клиентской части веб-приложения	frontend-разработчик	1	14
Разработка каталога, заказов, услуг, откликов и профилей	frontend-разработчик	1	8
Тестирование интерфейса и пользовательских сценариев	тестировщик	1	5
Тестирование серверной логики и обработки ошибок	тестировщик	1	5
Подготовка руководства пользователя	продакт-менеджер	1	2
Всего			83

С учетом анализа предметной области, проектирования, разработки клиентской и серверной частей, реализации чатов, уведомлений, административных функций и тестирования, затраты рабочего времени составили 83 рабочих дня.

6.5 Расчет основной заработной платы

Для определения величины основной заработной платы проведена детальная оценка уровня оплаты труда специалистов, привлекаемых к проектированию и разработке веб-приложения. Данный анализ базируется на актуальных аналитических данных рынка труда, собранных со специализированных рекрутинговых платформ и статистических обзоров в сфере информационных технологий. Такой подход позволяет установить объективные и конкурентоспособные тарифные ставки, соответствующие квалификационному уровню исполнителей и специфике выполняемых ими задач на каждом этапе жизненного цикла создания программного продукта.

В процессе планирования трудозатрат количество рабочих дней переводится в месяцы из расчета 21,5 рабочего дня в месяц. Это значение является нормативным среднемесячным балансом времени, что полностью соответствует стандартному производственному календарю для условий пятидневной рабочей недели с полной занятостью. Применение единого нормативного измерителя позволяет стандартизировать расчеты, исключить влияние календарной сезонности на оценку трудоемкости и обеспечить высокую точность прогнозирования издержек. На основе полученных стоимостных и временных показателей основная заработная плата отдельного специалиста рассчитывается по формуле (6.1).

$$\text{Соз}_i = \text{Сзп}_i \cdot \text{Траз}_i \cdot \text{Краз}_i, \quad (6.1)$$

где Соз_i – основная заработная плата работника i -й должности, руб.;

Сзп_i – средняя месячная заработная плата работника i -й должности, руб.;

Траз_i – время работы работника i -й должности, месяцев;

Краз_i – количество работников i -й должности, человек.

Таблица 6.3 – Расчет основной заработной платы

Вид работ	Продолжительность работы, месяцев	Месячная заработная плата, руб.	Количество разработчиков, чел.	Основная заработная плата, руб.
Продакт-менеджер	0,23	2 600,90	1	604,86
Бизнес-аналитик	0,37	2 653,80	1	987,46
UX/UI дизайнер	0,47	2 604,30	1	1 211,30
Администратор базы данных	0,28	2 700,00	1	753,49
Backend-разработчик	1,02	2 900,80	1	2 968,26
Frontend-разработчик	1,02	2 799,00	1	2 864,09
Тестировщик	0,47	2 455,70	1	1 142,19
Всего				10 531,65

Исходя из расчета основной заработной платы, основная заработная плата составит 10 531,65 руб.

6.6 Расчет дополнительной заработной платы

Дополнительная заработная плата включает выплаты, предусмотренные законодательством о труде, и определяется по нормативу в процентах к основной заработной плате, формула для вычисления 6.2.

$$\text{Сдз} = \text{Соз} \cdot \text{Ндз} / 100, \quad (6.2)$$

где Соз – основная заработная плата, руб.;

Ндз – норматив дополнительной заработной платы, %.

$$\text{Сдз} = 10\,531,65 \cdot 20 / 100 = 2\,106,33 \text{ руб.}$$

Размер дополнительной заработной платы составит 2 106,33 руб.

6.7 Расчет отчислений в Фонд социальной защиты населения и по обязательному страхованию

Отчисления в Фонд социальной защиты населения и по обязательному страхованию от несчастных случаев на производстве и профессиональных

заболеваний определяются в процентном отношении к фонду основной и дополнительной заработной платы, формула 6.3.

$$Сфсзн = (Соз + Сдз) \cdot Нфсзн / 100, \quad (6.3)$$

где Нфсзн – норматив отчислений в ФСЗН, %;

Соз – основная заработная плата, руб.;

Сдз – дополнительная заработная плата.

$$Сфсзн = (10\,531,65 + 2\,106,33) \cdot 34 / 100 = 4\,296,91 \text{ руб.}$$

По формуле 6.4 определяется размер обязательных страховых взносов, которые организация направляет в Белорусское республиканское унитарное страховое предприятие «Белгосстрах». Данный платеж покрывает риски, связанные с несчастными случаями на производстве и профессиональными заболеваниями сотрудников. Размер отчислений напрямую зависит от фонда оплаты труда и присвоенного организации класса профессионального риска.

$$Сбгс = (Соз + Сдз) \cdot Нбгс / 100, \quad (6.4)$$

где Нбгс – ставка отчислений по страхованию, %.

$$Сбгс = (10\,531,65 + 2\,106,33) \cdot 0,6 / 100 = 75,83 \text{ руб.}$$

Таким образом, отчисления в ФСЗН составят 4 296,91 руб., а отчисления в БРУСП «Белгосстрах» – 75,83 руб.

6.8 Расчет расходов на материалы

Структура расходов на материалы для данного проекта была сформирована исходя из необходимости обеспечения непрерывного процесса разработки и строгого соответствия стандартам технической документации. Общая сумма материальных затрат отражает рациональное использование ресурсов, необходимых для сопровождения жизненного цикла программного средства на этапе его создания. Расходы на материалы включают затраты, связанные с подготовкой документации, печатью промежуточных материалов и резервным копированием рабочих файлов, расходы приведены в таблице 6.4.

Таблица 6.4 – Материальные затраты на разработку ПС

Вид материальных затрат	Фактические затраты, руб.
Бумага офисная для подготовки и печати материалов	12,00
Тонер для картриджа	8,00
Канцелярские принадлежности	6,50
Накопитель и резервное копирование рабочих материалов	18,00
Всего	44,50

Значительную часть статьи расходов составляют затраты на организацию резервного копирования рабочих файлов. В условиях разработки веб-приложения надежное хранение данных и создание контрольных точек версионирования проекта являются важными аспектами.

Расходы на бумажную документацию и расходные материалы для печати продиктованы необходимостью подготовки промежуточных отчетов, технических описаний и ведения сопроводительной документации, которая требуется для фиксации архитектурных решений и этапов разработки.

Дополнительные расходы, связанные с приобретением канцелярских принадлежностей, были направлены на обеспечение рабочего процесса инструментами, необходимыми для оперативного фиксирования идей, планирования задач и ведения текущих записей.

Исходя из расчетов, сумма расходов на материалы для подготовки документации, печати промежуточных материалов и резервным копированием рабочих файлов составит 44,50 руб.

6.9 Расходы на специальное оборудование и платные услуги

При разработке веб-приложения использовались бесплатные инструменты и библиотеки: Node.js, Express.js, React.js, MySQL, Socket.IO и Docker. Специальное оборудование, платные лицензии и услуги сторонних организаций для выполнения проекта не приобретались. Поэтому расходы равны 0,00 руб.

6.10 Расчет расходов на оплату машинного времени

Расходы на оплату машинного времени являются неотъемлемой частью себестоимости разработки, так как они отражают интенсивность использования вычислительных мощностей на протяжении всего жизненного цикла проекта, показатель включает в себя затраты на электроэнергию, амортизационные отчисления оборудования и техническую эксплуатацию, необходимые для полноценного функционирования среды разработки, включая проектирование, написание программного кода, комплексное тестирование и отладку веб-приложения. Количество машино-часов принято исходя из 6 часов работы компьютера в течение каждого рабочего дня разработки.

$$C_{\text{мв}} = \text{Драз} \cdot T_{\text{д}} \cdot C_{\text{мч}}, \quad (6.5)$$

где Драз – количество рабочих дней разработки;

$T_{\text{д}}$ – среднее количество часов работы компьютера в день;

$C_{\text{мч}}$ – стоимость одного машино-часа, руб.

$$C_{\text{мв}} = 83 \cdot 6 \cdot 0,12 = 59,76 \text{ руб.}$$

На основе данной формулы осуществляется аккумулярование прямых материально-технических затрат, непосредственно связанных с обеспечением непрерывного процесса проектирования. Расчет указанных издержек

позволяет детально обосновать долю эксплуатационных расходов в общей структуре себестоимости программного средства.

6.11 Расчет прочих прямых затрат

Сумма прочих прямых затрат определяется произведением основной заработной платы на норматив прочих затрат в целом по организации.

$$\begin{aligned} C_{пз} &= C_{оз} \cdot Н_{пз} / 100, \\ C_{пз} &= 10\,531,65 \cdot 16 / 100 = 1\,685,06 \text{ руб.} \end{aligned} \quad (6.6)$$

6.12 Расчет накладных расходов

Накладные расходы учитывают затраты на организацию работ, управление разработкой, содержание рабочего места и использование общих ресурсов организации. По формуле 6.7 определяются накладные расходы.

$$\begin{aligned} C_{нр} &= C_{оз} \cdot Н_{нр} / 100, \\ C_{нр} &= 10\,531,65 \cdot 28 / 100 = 2\,948,86 \text{ руб.} \end{aligned} \quad (6.7)$$

6.13 Сумма расходов на разработку программного средства

Сумма расходов на разработку программного средства определяется как сумма основной и дополнительной заработных плат, отчислений на социальные нужды, расходов на материалы, расходов на специальное оборудование и платные услуги, а также расходов на оплату машинного времени, прочих прямых затрат и накладных расходов.

$$\begin{aligned} C_p &= C_{оз} + C_{дз} + C_{фсзн} + C_{бгс} + C_m + C_{сопу} + C_{мв} + C_{пз} + C_{нр}. \\ C_p &= 10\,531,65 + 2\,106,33 + 4\,296,91 + 75,83 + \\ &+ 44,50 + 0,00 + 59,76 + 1\,685,06 + 2\,948,86 = 21\,748,91 \text{ руб.} \end{aligned} \quad (6.8)$$

Общая сумма расходов на разработку составит 21 748,91 руб.

6.14 Расходы на сопровождение и адаптацию

После завершения разработки программное средство требует адаптации к условиям внедрения, настройки окружения, проверки работы в целевой инфраструктуре и первичного сопровождения пользователей. Расходы на сопровождение и адаптацию рассчитываются по формуле 6.9.

$$\begin{aligned} C_{рса} &= C_p \cdot Н_{рса} / 100. \\ C_{рса} &= 21\,748,91 \cdot 8 / 100 = 1\,739,91 \text{ руб.} \end{aligned} \quad (6.9)$$

6.15 Полная себестоимость

Полная себестоимость определяется как сумма расходов на разработку и расходов на сопровождение и адаптацию.

$$C_{\text{п}} = C_{\text{р}} + C_{\text{рса}}. \quad (6.10)$$

$$C_{\text{п}} = 21\,748,91 + 1\,739,91 = 23\,488,82 \text{ руб.}$$

6.16 Определение цены и оценка эффективности

Для разрабатываемого программного средства прогнозная рыночная цена определяется на основе комплексного подхода, сочетающего в себе учет полной себестоимости разработки, планового уровня рентабельности, а также глубокого маркетингового анализа стоимостных параметров аналогичных веб-платформ, присутствующих на современном рынке информационных технологий. Применение подобной методологии позволяет не только полностью компенсировать понесенные затраты и обеспечить необходимую норму прибыли, но и сформировать отпускную стоимость, коммерчески привлекательную для потенциальных заказчиков.

Для объективного определения окончательной цены реализации и минимизации рисков неверного ценового позиционирования продукта возникает необходимость произвести детальный сравнительный анализ стоимости аналогичных программных приложений. В качестве основного аналитического инструмента для решения данной задачи выступает мониторинг актуальных данных ведущих специализированных сайтов-агрегаторов услуг по разработке программного обеспечения, которые аккумулируют текущие рыночные предложения и позволяют оценить средние бюджеты на создание схожих программных систем.

Первым ресурсом для анализа является сайт <https://megagroup.ru>. Выбрав все необходимые характеристики, выходит, что цена разработки веб-приложения составит 804432 рос. руб., что составляет 30983,50 руб. (по курсу Национального банка Республики Беларусь на 30.05.2026). На рисунке 6.1 представлен результат работы сайта-агрегатора.

Выберите тариф	
☐ Лендинг	5 450 P
☐ Лендинг (Индивидуальный дизайн)	59 950 P
☐ Сайт Бизнес (Базовое отраслевое решение)	5 450 P
☐ Сайт Бизнес (Минимальный дизайн на выбор)	6 450 P
☐ Сайт Бизнес (Индивидуальный дизайн)	60 000 P
☐ Интернет-магазин (Минимальный)	5 450 P
☐ Интернет-магазин (Стандартный)	5 450 P
☐ Интернет-магазин (Максимальный)	35 000 P
☒ Интернет-магазин (Индивидуальный дизайн)	60 000 P

Итого: 804 432 P

[Оставить заявку](#)

Рисунок 6.1 – Расчет стоимости сайт <https://megagroup.ru>

Вторым ресурсом является сайт <https://varanika.by>. Выбрав все необходимые характеристики, выходит, что цена разработки составит 31250 руб. На рисунке 6.2 представлен результат работы сайта-агрегатора.

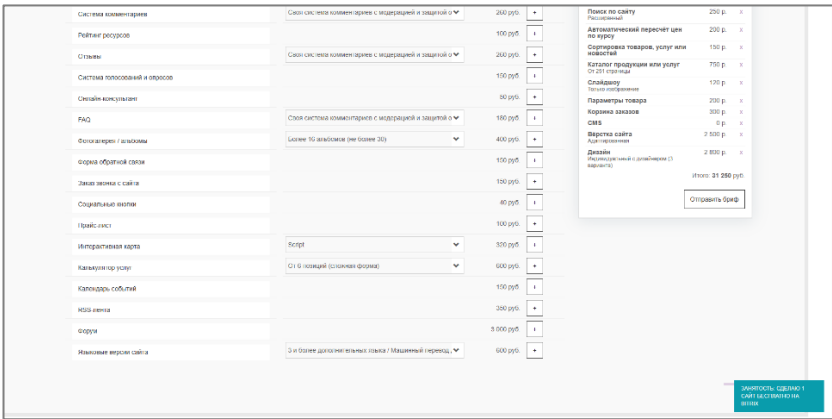


Рисунок 6.2 – Расчет стоимости сайт <https://varanika.by>

Третьим ресурсом для анализа является сайт <https://web-industry.pro>. Выбрав все необходимые характеристики, выходит, что цена разработки веб-приложения составит 1280506 рос. руб., что составляет 49319,96 руб. (по курсу Национального банка Республики Беларусь на 30.05.2026). На рисунке 6.3 представлен результат работы сайта-агрегатора.

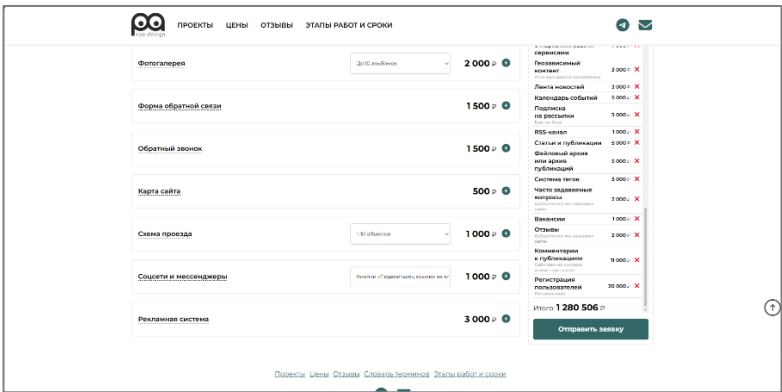


Рисунок 63 – Расчет стоимости сайт <https://web-industry.pro>

Средняя цена разработки приложения с аналогичным функционалом рассчитывается как среднее арифметическое указанных выше цен:

$$Ц_{ср} = \frac{(30983,50 + 31250 + 49319,96)}{3} = 37184,49 \text{ руб.}$$

В ходе проведения анализа была определена стоимость разработки аналогичного программного продукта. Средняя цена разработки аналогичного приложения составляет приблизительно 37184,49 рублей.

Так как полная себестоимость разработки и поддержки ПС составляет 23488,82 руб., а среднерыночная стоимость аналогичных решений равна 37184,49 руб., прогнозную отпускную цену с НДС целесообразно установить

на уровне 36642,56 руб. Данное решение позволяет полностью покрыть издержки производства, получить запланированную норму прибыли и одновременно обеспечить высокую конкурентоспособность программного средства за счет цены, которая находится чуть ниже среднерыночного уровня.

Цена программного средства без НДС рассчитывается по формуле 6.11:

$$Ц_{\text{без НДС}} = \frac{Ц_p}{(Ц_{\text{ндс}} + 100) / 100}, \quad (6.11)$$

$$Ц_{\text{без НДС}} = 36\,642,56 \cdot (100 / 120) = 30\,535,47$$

Прибыль от реализации рассчитывается по формуле 6.12:

$$П_{\text{пс}} = Ц_{\text{рын}} - С_{\text{п}}, \quad (6.12)$$

$$П_{\text{пс}} = 30\,535,47 - 23\,488,82 = 7\,046,65 \text{ руб.}$$

Рентабельность разрабатываемого продукта по формуле 6.13:

$$Р_{\text{пс}} = П_{\text{пс}} / С_{\text{п}} \cdot 100, \quad (6.13)$$

$$Р_{\text{пс}} = (7\,046,65 / 23\,488,82) \times 100 = 30,00 \, \%.$$

Чистая прибыль рассчитывается по формуле (6.14):

$$ЧП = П_{\text{пс}} \cdot (1 - Н_{\text{п}} / 100), \quad (6.14)$$

$$ЧП = 7\,046,65 \times (1 - 20/100) = 5\,637,32 \text{ руб.}$$

На основании расчетов видно, что проект является экономически целесообразным, так как основные показатели эффективности выше нуля, а отпускная цена обоснована текущей рыночной конъюнктурой.

6.17 Конкурентные преимущества программного средства

Разрабатываемое веб-приложение относится к классу фриланс-платформ. В качестве аналогов можно рассматривать Fiverr, Freelancer и Upwork, которые также обеспечивают взаимодействие заказчиков и исполнителей. При этом разработанное программное средство имеет более узкую и понятную целевую аудиторию: студентов, которым необходимо получить первый коммерческий опыт, и заказчиков, которым требуется доступный исполнитель для выполнения разовых задач.

Основными преимуществами веб-приложения являются:

- низкий барьер входа для студентов-исполнителей;
- наличие резюме, портфолио, рейтингов и отзывов в одном профиле;
- поддержка заказов и готовых услуг в едином каталоге;
- встроенный чат для согласования требований и сроков;
- модерация резюме, заказов, услуг и жалоб;
- разделение прав доступа между администратором и менеджером;

– возможность развёртывания в инфраструктуре с помощью Docker.

Таким образом, проект является специализированной платформой для профессиональной интеграции студентов.

6.18 Выводы по разделу

Результаты расчетов представлены в таблице 6.5. Полная себестоимость разработки веб-приложения «Маркетплейс фриланс-заказов для студентов» составит 23 488,82 руб. При продаже программного средства с передачей прав заказчику отпускная цена без НДС составит 30 535,47 руб., а отпускная цена с НДС – 36 642,56 руб. Прибыль от реализации программного средства составит 7 046,65 руб., чистая прибыль – 5 637,32 руб., уровень рентабельности – 30,00 %.

Таблица 6.5 – Результаты расчетов

Наименование показателя	Значение показателя
Время разработки, дней	83
Основная заработная плата, руб.	10 531,65
Дополнительная заработная плата, руб.	2 106,33
Отчисления в Фонд социальной защиты населения, руб.	4 296,91
Отчисления в БРУСП «Белгосстрах», руб.	75,83
Расходы на материалы, руб.	44,50
Расходы на специальное оборудование и платные услуги, руб.	0,00
Расходы на оплату машинного времени, руб.	59,76
Прочие прямые затраты, руб.	1 685,06
Накладные расходы, руб.	2 948,86
Себестоимость разработки программного средства, руб.	21 748,91
Расходы на сопровождение и адаптацию, руб.	1 739,91
Полная себестоимость, руб.	23 488,82
Отпускная цена без НДС, руб.	30 535,47
Отпускная цена с НДС, руб.	36 642,56
Прибыль от реализации программного средства, руб.	7 046,65
Чистая прибыль, руб.	5 637,32
Уровень рентабельности, %	30,00

На основании полученных данных можно сделать однозначный вывод, что разработка данного программного средства является экономически целесообразной и высокоэффективной. Проект имеет полностью завершённый функциональный состав, четко обоснованную стратегию ценообразования, а все ключевые финансово-экономические показатели эффективности стабильно превышают нулевую отметку, что гарантирует полную окупаемость инвестиций и финансовую устойчивость разработки.

Заключение

Разработка приложения была успешно завершена, в результате чего создана полнофункциональная, модульная и масштабируемая платформа, предназначенная для эффективного взаимодействия Исполнителей и Заказчиков. Успех проекта базируется на четком следовании принципам разделения ответственности и использовании современного технологического стека, обеспечивающего как надежность, так и высокую производительность.

Проект реализован с четким архитектурным разделением на серверную (backend) и клиентскую (frontend) части. Серверная часть, построенная на Node.js с использованием фреймворка Express.js и ORM Sequelize для работы с базой данных MySQL, организована по строго модульному принципу. Это означает, что каждый функциональный блок (аутентификация, управление заказами, чаты, профили) инкапсулирован в отдельную директорию, что значительно упрощает процессы сопровождения кода, локализации ошибок и последующего масштабирования системы.

Клиентская часть, разработанная на React с использованием TypeScript для повышения надежности и статической типизации, а также Tailwind CSS для быстрой и адаптивной стилизации, обеспечивает интуитивно понятный и современный пользовательский интерфейс. Взаимодействие с API реализовано через слой специализированных сервисов, что четко отделяет логику представления от логики взаимодействия с данными.

Ключевым технологическим достижением является полная интеграция Socket.IO (WebSocket), что позволило реализовать функциональность real-time коммуникации. Это критически важно для мгновенной доставки сообщений в чатах и оперативного получения уведомлений о событиях, обеспечивая интерактивный пользовательский опыт, невозможный при использовании только REST API.

Надежность и безопасность системы гарантируются за счет реализации многоуровневой системы аутентификации на основе JWT и строгой авторизации на основе ролей. Кроме того, внедрен механизм проверки блокировки аккаунта, который принудительно прерывает доступ заблокированным пользователям, поддерживая целостность и безопасность платформы.

Проект успешно достиг всех поставленных целей, создав функциональную, безопасную и масштабируемую основу для развития платформы.

				ДП 00.00.ПЗ						
	ФИО	Подпись	Дата							
Разраб.	Глухова Д.В.			Заключение			Лит.	Лист	Листов	
Пров.	Уласевич Н.И.						У	1	1	
							БГТУ 1-40 05 01, 2026			
Н. контр.	Николайчук А.Н.									
Утв.	Блинова Е.А.									

Список использованных источников

- 1 Node.js: документация [сайт]. – URL: <https://nodejs.org/api/all.html> (дата обращения: 24.01.2026).
- 2 JavaScript: документация [сайт]. – URL: <https://js-docs.vercel.app> (дата обращения: 24.01.2026).
- 3 Fiverr: freelance services marketplace [сайт]. – URL: <https://www.fiverr.com> (дата обращения: 25.01.2026).
- 4 Freelance: биржа фриланса [сайт]. – URL: <https://freelance.ru/> (дата обращения: 25.01.2026).
- 5 Upwork: фриланс-биржа [сайт]. – URL: <https://www.upwork.com/> (дата обращения: 25.01.2026).
- 6 Express: документация [сайт]. – URL: <https://js-docs.vercel.app> (дата обращения: 29.01.2026).
- 7 MySQL: документация [сайт]. – URL: <https://dev.mysql.com/doc/> (дата обращения: 29.01.2026).
- 8 Sequelize ORM: документация [сайт]. – URL: <https://sequelize.org/docs/v6/> (дата обращения: 29.01.2026).
- 9 React: документация [сайт]. – URL: <https://react.dev/learn> (дата обращения: 29.01.2026).
- 10 Geeksforgeeks: документация use-case диаграмм [сайт]. – URL: <https://www.geeksforgeeks.org/system-design/use-case-diagram/> (дата обращения: 29.01.2026).
- 11 Geeksforgeeks: документация схемы развертывания [сайт]. – URL: <https://www.geeksforgeeks.org/system-design/deployment-diagram-unified-modeling-languageuml/> (дата обращения: 30.01.2026).
- 12 Docker: документация [сайт]. – URL: <https://docs.docker.com/> (дата обращения: 30.01.2026).

				ДП 00.00.ПЗ						
	ФИО	Подпись	Дата							
Разраб.	Глухова Д.В.			Список использованных источников	Лит.		Лист	Листов		
Пров.	Уласевич Н.И.					У	1	1		
					БГТУ 1-40 05 01, 2026					
Н. контр.	Николайчук А.Н.									
Утв.	Блинова Е.А.									

ПРИЛОЖЕНИЕ А

Схема архитектуры приложения

ПРИЛОЖЕНИЕ Б
Диаграмма вариантов использования

ПРИЛОЖЕНИЕ В
Логическая схема базы данных

ПРИЛОЖЕНИЕ Г
Блок-схема обработки и маршрутизации сообщения

ПРИЛОЖЕНИЕ Д
Блок-схема процесса создания заказа

ПРИЛОЖЕНИЕ Е

Модели серверной части

```
const { PASSWORD } = require("../../config/db.config");
module.exports = (sequelize, Sequelize) => {
  const User = sequelize.define("users", {
    username: {
      type: Sequelize.STRING
    },
    email: {
      type: Sequelize.STRING
    },
    password: {
      type: Sequelize.STRING
    },
    isBlocked: {
      type: Sequelize.BOOLEAN,
      defaultValue: false
    },
    lastSeen: {
      type: Sequelize.DATE,
      allowNull: true
    }
  });
  return User;}
```

Листинг 1 – Модель пользователя

```
module.exports = (sequelize, Sequelize) => {
  const Role = sequelize.define("roles", {
    id: {
      type: Sequelize.INTEGER,
      primaryKey: true
    },
    name: {
      type: Sequelize.STRING
    }
  });
  return Role;
}
```

Листинг 2 – Модель роли пользователя

ПРИЛОЖЕНИЕ Ж

Реализация создания отклика на заказ или услугу

```

const existing = await Application.findOne({
  where: {
    userId: currentUser.id,
    [Op.or]: [
      { orderId: orderId || null },
      { serviceId: serviceId || null }
    ]
  }
});

if (existing) {
  return res.status(400).send({ message: "Вы уже отклик-
нулись на этот заказ/услугу" });
}

const application = await Application.create({
  userId: currentUser.id,
  orderId: orderId || null,
  serviceId: serviceId || null,
  message: message || null,
  proposedPrice: proposedPrice || null
});

const applicationWithRelations = await
Application.findByPk(application.id, {
  include: [
    { model: User, as: "user", attributes: ["id",
"username", "email"] },
    { model: Order, as: "order" },
    { model: Service, as: "service" }
  ]
});

if (orderId) {
  const order = await Order.findByPk(orderId);
  if (order) {
    await createNotification(
      order.customerId,
      'new_application',
      'Новый отклик на ваш заказ',
      `Пользователь ${currentUser.username} отклик-
нулся на ваш заказ "${order.title}"`,
      application.id,
      'application'
    );
  }
}

```


ПРИЛОЖЕНИЕ И

Система маршрутизации клиентской части

```

return (
  <Router>
    <Navbar />
    <Routes>
      <Route path="/login" element={<Login />} />
      <Route path="/register" element={<Register />} />
      <Route path="/verify/:token" element={<Verify />} />
      <Route path="/verify" element={<Verify />} />
      <Route
        path="/catalog"
        element={
          <ProtectedRoute allowPublic={true}>
            <Catalog />
          </ProtectedRoute>
        }
      />
      <Route
        path="/orders/:id"
        element={
          <ProtectedRoute allowPublic={true}>
            <OrderDetail />
          </ProtectedRoute>
        }
      />
      <Route
        path="/services/:id"
        element={
          <ProtectedRoute allowPublic={true}>
            <ServiceDetail />
          </ProtectedRoute>
        }
      />
      <Route
        path="/profile/:id"
        element={
          <ProtectedRoute allowPublic={true}>
            <WorkProfile />
          </ProtectedRoute>
        }
      />

      <Route
        path="/"
        element={
          <ProtectedRoute>
            <Home />
          </ProtectedRoute>
        }
      />
    </Routes>
  </Router>
)

```

ПРИЛОЖЕНИЕ К

Сервис администратора

```
import axios from "axios";
const API_URL = "http://localhost:8080/";
const getAuthHeaders = () => {
  const user = localStorage.getItem("user");
  if (user) {
    const userData = JSON.parse(user);
    return {
      Authorization: `Bearer ${userData.accessToken}`,
    };
  }
  return {};
};
const blockUser = (userId: number) => {
  return axios.put(API_URL + `admin/users/${userId}/block`, {}, {
    headers: getAuthHeaders(),
    withCredentials: true,
  });
};
const unblockUser = (userId: number) => {
  return axios.put(API_URL + `admin/users/${userId}/unblock`, {}, {
    headers: getAuthHeaders(),
    withCredentials: true,
  });
};
const getAllUsers = () => {
  return axios.get(API_URL + "admin/users", {
    headers: getAuthHeaders(),
    withCredentials: true,
  });
};
const AdminService = {
  blockUser,
  unblockUser,
  getAllUsers,
};
export default AdminService;
```

ПРИЛОЖЕНИЕ Л

WebSocket-коммуникация и управление состоянием

```
import { io, Manager, Socket } from "socket.io-client";
import AuthService from "../auth.service";
class WebSocketService {
  private socket: Socket | null = null;
  private reconnectAttempts = 0;
  private maxReconnectAttempts = 5;
  private isConnecting = false;
  connect() {
    const currentUser = AuthService.getCurrentUser();
    if (!currentUser) {
      return;
    }
    if (this.socket?.connected || this.isConnecting) {
      return;
    }
    if (this.socket && !this.socket.connected) {
      this.socket.disconnect();
      this.socket = null;
    }
    this.isConnecting = true;
    const token = currentUser.accessToken;
    this.socket = io("http://localhost:8080", {
      auth: {
        token: token,
      },
      transports: ["websocket", "polling"],
      reconnection: true,
      reconnectionDelay: 1000,
      reconnectionDelayMax: 5000,
      reconnectionAttempts: this.maxReconnectAttempts,
      timeout: 10000,
      forceNew: false,
    });
    this.socket.on("connect", () => {
      console.log("WebSocket connected");
      this.reconnectAttempts = 0;
      this.isConnecting = false;
    });
  }
}
```

ПРИЛОЖЕНИЕ М

Docker-файлы для сборки проекта

```
FROM node:20-alpine
WORKDIR /app
COPY package*.json ./
RUN npm install
COPY backend/ ./backend/
COPY config/ ./config/
WORKDIR /app/backend
EXPOSE 8080
CMD ["node", "server.js"]
```

Листинг 1 – Docker-файл для backend части

```
FROM node:20-alpine
WORKDIR /app
COPY package*.json ./
RUN npm install
COPY . .
EXPOSE 3000
CMD ["npm", "run", "dev", "--", "--host"]
```

Листинг 2 – Docker-файл для frontend части

ПРИЛОЖЕНИЕ Н
Экранная форма главной страницы

				ДП 06.00.ГЧ			
				Таблица технико-экономических показателей	Лит.		
	ФИО	Подпись	Дата		у		
Разраб.	Глухова Д.В.						
Провер.	Уласевич Н.И.						
					Лист 1		Листов 1
					БГТУ 1-40 05 01, 2026		
Н. контр.	Николайчук А.Н.						
Утв.	Блинова Е.А.						

ПРИЛОЖЕНИЕ П
Таблица технико-экономических показателей

				<i>ДП 07.00.ГЧ</i>			
	ФИО	Подпись	Дата	<i>Экранная форма главной страницы</i>	Лит.		
					у		
Разраб.	<i>Глухова Д.В.</i>						
Провер.	<i>Уласевич Н.И.</i>						
					Лист	1	Листов 1
Н. контр.	<i>Николайчук А.Н.</i>				<i>БГТУ 1-40 05 01, 2026</i>		
Утв.	<i>Блинова Е.А.</i>						